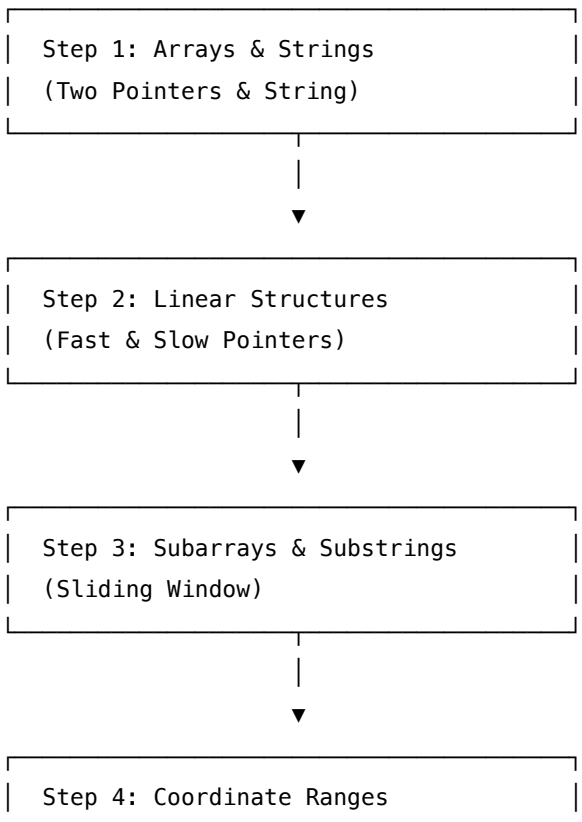
Contents

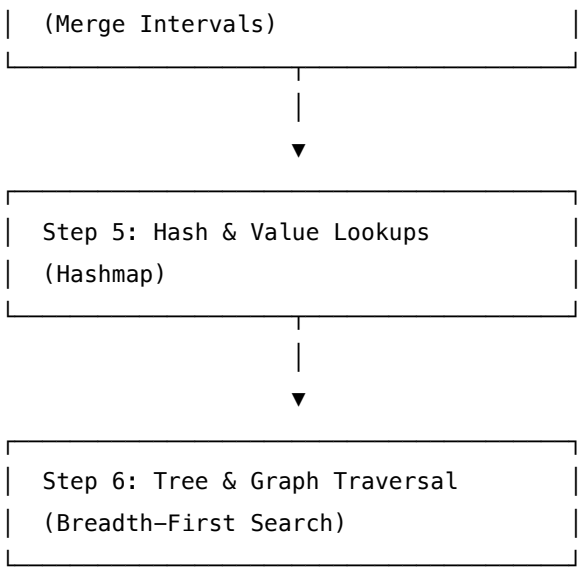
1 Tier 1: Foundation — Coding Interview Preparation Roadmap

Welcome to the **Foundation (Tier 1) Coding Roadmap**. This directory serves as your structured study guide and reference repository for mastering foundational coding patterns. These patterns represent the core building blocks of algorithmic problem-solving and constitute approximately **60-70%** of coding interview questions at top-tier technology companies (such as Google, Meta, NVIDIA, and Microsoft).

1.1 Progression Roadmap

To build strong algorithmic intuition, we recommend following a progressive learning order. Rather than jumping randomly between topics, master them sequentially as each builds on the mathematical and structural concepts of the previous one.





1.2 Detailed Learning Modules & File Index

Here is the complete catalog of the 39 Foundation problems, categorized by pattern, and arranged in recommended learning order.

1.2.1 Module 1: Two Pointers & String Manipulation

Goal: Master in-place array/string traversal, reducing $\mathcal{O}(n^2)$ naive nested loops to $\mathcal{O}(n)$ linear scans.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Two Pointers	Container With Most Water	LeetCode 11	Medium	Greedy choice, area calculation, boundary convergence
Two Pointers	3Sum	LeetCode 15	Medium	Sorting, duplicate avoidance, triplet generation
Two Pointers	Two Sum II - Sorted Input	LeetCode 167	Medium	Constant space lookup, sorted array properties
Two Pointers	Is Subsequence	LeetCode 392	Easy	Greedy match pointer, sequence order validation
Two Pointers	Heaters	LeetCode 475	Medium	Sorting, nearest distance search, binary search alternative

Pattern	Problem Name	Source	Difficulty	Core Concepts
Two Pointers	Longest Uncommon Subsequence II	LeetCode 522	Medium	Subsequence checking, string length comparison
Two Pointers	Longest Word in Dict by Deleting	LeetCode 524	Medium	Custom sorting, deletion mapping, greediness
Two Pointers	K-diff Pairs in an Array	LeetCode 532	Medium	Two-pointer vs hashmap, duplicate prevention
String	Number of Segments in String	LeetCode 434	Easy	Space transitions, state flags
String	License Key Formatting	LeetCode 482	Easy	Backward processing, group indexing
String	Detect Capital	LeetCode 520	Easy	Capital casing rules, simple state counters
String	Longest Uncommon Subsequence I	LeetCode 521	Easy	Logical deduction, string equality checks

1.2.2 Module 2: Fast & Slow Pointers (Tortoise & Hare)

Goal: Detect cycles and locate structures in linked lists or mathematical sequences with $\mathcal{O}(1)$ space complexity.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Fast & Slow Pointers	Linked List Cycle	LeetCode 141	Easy	Floyd's Cycle-Finding Algorithm, loop invariants
Fast & Slow Pointers	Happy Number	LeetCode 202	Easy	Digit sum cycles, sequence convergence
Fast & Slow Pointers	Middle of the Linked List	LeetCode 876	Easy	Node traversal parity, list splitting

1.2.3 Module 3: Sliding Window

Goal: Dynamically resize a subsegment of an array or string to track constraints, avoiding nested-loop recalibration.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Sliding Window	Longest Substring without Duplicates	LeetCode 3	Medium	Map-based index skips, dynamic left-pointer updates
Sliding Window	Longest Substring with At Least K Repeats	LeetCode 395	Medium	Multi-sliding window with unique char constraint limits
Sliding Window	Longest Repeating Character Replacement	LeetCode 424	Medium	Substring character frequency tracking, maximum frequency preservation
Sliding Window	Find All Anagrams in a String	LeetCode 438	Medium	Static-width sliding window, character frequency map diffing
Sliding Window	Permutation in String	LeetCode 567	Medium	Slide over fixed window, sliding frequency comparison

1.2.4 Module 4: Merge Intervals

Goal: Process time segments, coordinates, and overlaps by sorting boundaries and sequentially merging.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Merge Intervals	Merge Intervals	LeetCode 56	Medium	Boundary sorting, overlap consolidation, in-place moving
Merge Intervals	Insert Interval	LeetCode 57	Medium	3-stage linear processing (before, during, after)
Merge Intervals	Meeting Rooms II (Rooms Required)	LeetCode 253	Medium	Sweep-line technique, priority queues, start-end sorting

Pattern	Problem Name	Source	Difficulty	Core Concepts
Merge Intervals	Teemo Attacking	LeetCode 495	Easy	Poison state calculation, simple step progression

1.2.5 Module 5: Hashmap & Dynamic Lookups

Goal: Achieve $\mathcal{O}(1)$ insertion, deletion, and lookup times, combining associative maps with contiguous arrays for specialized data structures.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Hashmap	Insert Delete GetRandom O(1)	LeetCode 380	Medium	Combination of hash table (indexes) & dynamic vector (elements)
Hashmap	Linked List Random Node	LeetCode 382	Medium	Reservoir Sampling, streaming data selection under memory limits
Hashmap	Perfect Rectangle	LeetCode 391	Hard	Area calculation, corner coordinate set validation
Hashmap	Random Pick Index	LeetCode 398	Medium	Reservoir sampling, duplicate values lookup indexing
Hashmap	Number of Boomerangs	LeetCode 447	Medium	Distance hashing, permutations counting
Hashmap	Keyboard Row	LeetCode 500	Easy	Set memberships, string categorization
Hashmap	Encode and Decode TinyURL	LeetCode 535	Medium	Two-way hash matching, random base62 ID generation
Hashmap	Distribute Candies	LeetCode 575	Easy	Set hashing, logical min boundaries

1.2.6 Module 6: Breadth-First Search (BFS) & Tree Traversal

Goal: Traverse tree structures level-by-level or find the shortest path in unweighted spatial grids.

Pattern	Problem Name	Source	Difficulty	Core Concepts
BFS	Binary Tree Level Order Traversal	LeetCode 102	Medium	Level-by-level BFS queue sizing, dynamic sub-array construction
BFS	Shortest Path in Binary Matrix	LeetCode 1091	Medium	8-directional spatial grid BFS, queue invariants, in-place marking
BFS	Serialize and Deserialize BST	LeetCode 449	Medium	Binary sequence streaming, pre-order reconstruction boundaries
BFS	Island Perimeter	LeetCode 463	Easy	Geometric cell counting, shared boundary deduplication
BFS	Find Bottom Left Tree Value	LeetCode 513	Medium	Right-to-Left level order traversal queue state preservation
BFS	Find Largest Value in Tree Rows	LeetCode 515	Medium	Row maximum tracking, BFS level boundaries
BFS	Rotting Oranges	LeetCode 994	Medium	Multi-source BFS, fresh item counts, level-based state progression

---

1.3 Foundation Tier Interview Strategy

1.3.1 1. Pointer Boundary Safeties

- **Unsigned Underflow (C++)**: Avoid running loops with `0` limits using unsigned size types (e.g. `size_t`). Subtracting `1` from an empty array's size yields `18446744073709551615`. Cast to `int` or perform explicit empty checks.
- **Pointer Slices (Python)**: Avoid using slice syntax like `nums[left:right]` during partition operations. Slicing creates shallow copies, making your space complexity $\mathcal{O}(k)$ instead of $\mathcal{O}(1)$. Use explicit

pointer variables instead.

1.3.2 2. Space Optimization

- **BFS Memory Overhead**: The space complexity of BFS is dominated by the queue size, which reaches $\mathcal{O}(w)$ where w is the maximum width of the tree/graph. For a full binary tree, the maximum width is $n/2$ (at the leaf level). Keep this in mind when discussing memory constraints with the interviewer.
- **Hash Maps Memory Bloat**: C++ `std::unordered_map` and Python `dict` have substantial memory overheads due to hash bucket allocation. For small, fixed key sets (like lowercase English letters), use a static array of size 26 (e.g., `int freq[26] = {0}`) to gain massive performance increases and optimal cache locality.

1.3.3 3. Cycle Detection

- **In-Place Array Modification**: If the array elements lie within the bounds $[0, n-1]$, you can use the array elements themselves as pointers to detect cycles and duplicates in-place by negating the values at indices (e.g., `nums[abs(nums[i])] *= -1`), saving auxiliary space.

2 Container With Most Water

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 11, Glassdoor
- **Flashcards**: Two Pointers deck

2.1 Question Description

You are given an integer array `height` of length n. There are n vertical lines drawn such that the two endpoints of the i-th line are $(i, 0)$ and $(i, \text{height}[i])$.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return *the maximum amount of water a container can store.*

Notice that you may not slant the container.

2.1.1 Examples

- **Input**: `height = [1,8,6,2,5,4,8,3,7]`
  - **Output**: `49`
  - **Explanation**: The vertical lines are represented by array `[1,8,6,2,5,4,8,3,7]`. The max area of water is formed between the vertical line at index 1 (height = 8) and the vertical line at index

8 (height = 7). The width of the container is $8 - 1 = 7$, and the height is $\min(8, 7) = 7$. Thus, the maximum area is $7 \times 7 = 49$.

- **Input**: `height = [1,1]`
  - **Output**: `1`
  - **Explanation**: The width is 1, height is 1. The area is $1 \times 1 = 1$.

2.2 Detailed Solution (C++)

The problem can be solved in linear time $\mathcal{O}(n)$ using the **Two-Pointer Greedy Approach**.

1. Initialize two pointers: `left` at the beginning of the array (0) and `right` at the end of the array $(n - 1)$.
2. Compute the area formed by the vertical lines at `left` and `right`. The area is defined as:

$$\text{Area} = \min(\text{height}[left], \text{height}[right]) \times (right - left)$$

3. Update the `max_area` found so far.
4. **Greedy Decision Rule**: To maximize the area, we must decide which pointer to move.
   - If $\text{height}[left] < \text{height}[right]$, we increment `left`.
   - If $\text{height}[left] \geq \text{height}[right]$, we decrement `right`.
   - *Rationale*: The width of the container $(right - left)$ will always decrease by 1 when we move a pointer. To compensate for this decrease in width and potentially find a larger area, we must find a larger height. If we move the pointer that points to the larger height, the maximum height of the new container is still capped by the smaller height, meaning the area is guaranteed to decrease. Thus, the only path to a larger area is to move the pointer pointing to the shorter height.

2.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>
#include <cstddef>

class Solution {
public:
    int maxArea(const std::vector<int>& height) noexcept {
        // Edge Case: A container requires at least two lines
        if (height.size() < 2) {
            return 0;
        }

        int left = 0;
        int right = static_cast<int>(height.size()) - 1;
        int max_area = 0;
```

```
        while (left < right) {
            int h_left = height[left];
            int h_right = height[right];

            // Calculate height constrained by the shorter line
            int h = std::min(h_left, h_right);
            int area = h * (right - left);

            if (area > max_area) {
                max_area = area;
            }

            // Move the pointer representing the shorter vertical line inward
            if (h_left < h_right) {
                left++;
            } else {
                right--;
            }
        }

        return max_area;
    }
};
```

2.3 Detailed Solution (Python)

2.3.1 Standard Python Production Code

Below is the type-hinted, iterative Python implementation using standard boundary pointers.

```
from typing import List

class Solution:
    def maxArea(self, height: List[int]) -> int:
        """
        Calculates the maximum water a container can store.

        Time Complexity: O(N)
        Space Complexity: O(1)
        """
        # Edge Case: A container requires at least two lines
        if len(height) < 2:
            return 0
```

```
    left: int = 0
    right: int = len(height) - 1
    max_area: int = 0

    while left < right:
        h_left = height[left]
        h_right = height[right]

        # Determine the bottleneck height and compute area
        h = min(h_left, h_right)
        area = h * (right - left)

        if area > max_area:
            max_area = area

        # Greedy step: move the pointer pointing to the shorter bar
        if h_left < h_right:
            left += 1
        else:
            right -= 1

    return max_area
```

2.4 Python-Specific Complexities & Implementation Considerations

2.4.1 1. Built-in Function Overhead vs. Inline Comparisons

- In Python, invoking built-in functions like `min()` and `max()` within a tight loop introduces function call overhead (due to stack frame allocation and argument parsing).
- For performance-critical code paths, replacing `min(h_left, h_right)` and `max(max_area, area)` with manual `if-else` comparisons can result in a 20–30% speedup:

```
# Instead of min/max:
h = h_left if h_left < h_right else h_right
area = h * (right - left)
if area > max_area:
    max_area = area
```

2.4.2 2. Slicing Operations Avoidance

- Slicing lists in Python via `height[left:right]` copies references, incurring $\mathcal{O}(k)$ time and space overhead.

- A common pitfall is attempting to find subarrays using slicing. Using index pointers `left` and `right` guarantees true $\mathcal{O}(1)$ auxiliary space complexity.

2.4.3 3. Large Integer Representation

- Python 3 handles arbitrary-precision integers natively. Therefore, the multiplication `h * (right - left)` will never result in integer overflow, even for massive data structures.
- However, in C++ or other compiled languages, one must be cautious. Given the constraints $n \leq 10^5$ and height$[i] \leq 10^4$, the maximum possible area is 10^9, which fits safely within standard 32-bit signed integers (up to $\approx 2 \times 10^9$).

2.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	The algorithm processes each element at most once. In each iteration, either `left` is incremented or `right` is decremented.
Space Complexity	$\mathcal{O}(1)$	Only standard pointer index variables and a running maximum area variable are maintained in memory.

2.6 Common Follow-Up Questions & How to Answer

2.6.1 Q1: Can you mathematically prove that the greedy strategy is correct?

- **Answer**: Yes. Suppose at some step, height$[left] <$ height$[right]$. The current area is height$[left] \times (right - left)$.
- If we keep `left` fixed and move `right` inward to any new index k (where $left < k < right$), the new width is smaller than the original width ($k - left < right - left$). The new height will be $\min(\text{height}[left], \text{height}[k])$, which is at most height$[left]$.
- Therefore, the area for any container formed between `left` and k is bounded by:

  $$\text{Area} = \min(\text{height}[left], \text{height}[k]) \times (k - left) \leq \text{height}[left] \times (k - left) < \text{height}[left] \times (right - left)$$

- This proves that no container starting at `left` and ending at any index before `right` can yield a larger area than the current one. Thus, we can safely prune all those options by advancing `left` to `left + 1`.

2.6.2 Q2: How can we skip redundant states in the two-pointer loop?

- **Answer**: If we move `left` or `right` inward, we only care about new heights that are strictly taller than the boundary height we just discarded. If a new element is shorter than or equal to the previous height, it cannot form a larger area because both the height is smaller/equal and the width has shrunk.
- **Optimization (C++)**:

```
if (h_left < h_right) {
    while (left < right && height[left] <= h_left) {
        left++;
    }
} else {
    while (left < right && height[right] <= h_right) {
        right--;
    }
}
```

- This reduces the number of area recalculations, improving runtime on average cases.

2.7 Pro-Tip: How to Impress the Interviewer

- **Proactively Propose the Skip Optimization**: Showing the interviewer the skip optimization loop demonstrates that you understand the geometry of the problem and care about reducing unnecessary computations (e.g., memory lookups and math operations).
- **Discuss CPU Branch Predictor Friendliness**: Note that the standard two-pointer step contains a branch conditional check (`h_left < h_right`). In modern superscalar processors, if the input values are random, this branch is highly mispredicted. If performance is critical, we can analyze how skipping redundant elements limits the number of times we enter the outer loop's branch, maximizing CPU throughput.
- **Explicit Array Size and Signed Cast Checks**: In C++, `height.size() - 1` can lead to unsigned underflow if the input vector is empty. Casting size safely or explicitly handling size checks (`height.size() < 2`) indicates strong defensive coding and attention to system safety.

3 3Sum

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Algorithm Engineer
- **Source**: LeetCode 15, Glassdoor
- **Flashcards**: Two Pointers deck

3.1 Question Description

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $\text{nums}[i] + \text{nums}[j] + \text{nums}[k] == 0$.

Notice that the solution set must not contain duplicate triplets.

3.1.1 Examples

- **Input**: `nums = [-1,0,1,2,-1,-4]`
  - **Output**: `[[-1,-1,2],[-1,0,1]]`
  - **Explanation**:
    - `nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0`.
    - `nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0`.
    - `nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0`.
    - The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`. Note that the order of the output and the order of the triplets does not matter.
- **Input**: `nums = [0,1,1]`
  - **Output**: `[]`
  - **Explanation**: The only possible triplet is `[0,1,1]`, which sums up to $2 \neq 0$.
- **Input**: `nums = [0,0,0]`
  - **Output**: `[[0,0,0]]`
  - **Explanation**: The only possible triplet sums up to 0.

3.2 Detailed Solution (C++)

To solve 3Sum efficiently in $\mathcal{O}(n^2)$ time with $\mathcal{O}(1)$ auxiliary space (excluding sorting):

1. **Sort the Input Array**: Sorting allows us to use a two-pointer approach for the inner search and easily skip duplicate elements to satisfy the uniqueness constraint.
2. **Iterate the First Element**: Loop through the array with index i representing the first element of our triplet, `nums[i]`.
   - **Early Pruning**: Since the array is sorted, if `nums[i] > 0`, the sum of any triplet starting with `nums[i]` will be strictly positive. We can terminate the loop early.
   - **Duplicate Elimination**: If `nums[i] == nums[i - 1]` (for $i > 0$), we skip to avoid generating duplicate triplets.
3. **Two-Pointer Search**: Set `left = i + 1` and `right = n - 1`. While `left < right`:
   - Calculate `sum = nums[i] + nums[left] + nums[right]`.
   - If `sum < 0`, we need a larger value, so we increment `left`.
   - If `sum > 0`, we need a smaller value, so we decrement `right`.
   - If `sum == 0`, we record the triplet `[nums[i], nums[left], nums[right]]`. Then:
     - Advance `left` while skipping duplicate values: `nums[left] == nums[left + 1]`.
     - Decelerate `right` while skipping duplicate values: `nums[right] == nums[right - 1]`.
     - Move both pointers (`left++`, `right--`) to search for other potentials.

3.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    std::vector<std::vector<int>> threeSum(std::vector<int>& nums) {
        std::vector<std::vector<int>> result;
        if (nums.size() < 3) {
            return result;
        }

        // Sort the array to enable two-pointer approach and duplicate skipping
        std::sort(nums.begin(), nums.end());
        const int n = static_cast<int>(nums.size());

        for (int i = 0; i < n - 2; ++i) {
            // Early Pruning: If the current smallest value is positive, no sum can equal 0
            if (nums[i] > 0) {
                break;
            }

            // Skip duplicates for the first element
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }

            int left = i + 1;
            int right = n - 1;

            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];

                if (sum < 0) {
                    left++;
                } else if (sum > 0) {
                    right--;
                } else {
                    result.push_back({nums[i], nums[left], nums[right]});

                    // Skip duplicates for the second element
                    while (left < right && nums[left] == nums[left + 1]) {
                        left++;
```

```
                }
                // Skip duplicates for the third element
                while (left < right && nums[right] == nums[right - 1]) {
                    right--;
                }

                left++;
                right--;
            }
        }
    }

    return result;
  }
};
```

3.3 Detailed Solution (Python)

3.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using in-place list sorting.

```
from typing import List

class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        """
        Finds all unique triplets in the array that sum to zero.

        Time Complexity: O(N^2)
        Space Complexity: O(log N) to O(N) depending on the sorting implementation
        """
        result: List[List[int]] = []
        if len(nums) < 3:
            return result

        # Sort in-place using Timsort
        nums.sort()
        n = len(nums)

        for i in range(n - 2):
            # Early Pruning
            if nums[i] > 0:
                break
```

```
        # Skip duplicates for the first element
        if i > 0 and nums[i] == nums[i - 1]:
            continue

        left: int = i + 1
        right: int = n - 1

        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])

                # Skip duplicates for the second element
                while left < right and nums[left] == nums[left + 1]:
                    left += 1
                # Skip duplicates for the third element
                while left < right and nums[right] == nums[right - 1]:
                    right -= 1

                left += 1
                right -= 1

    return result
```

3.4 Python-Specific Complexities & Implementation Considerations

3.4.1 1. In-place Sorting vs. `sorted()`

- Using `nums.sort()` sorts the list in-place, modifying the original object and using $\mathcal{O}(1)$ additional memory pointer updates (Timsort still requires auxiliary memory for run-merge arrays up to $\mathcal{O}(n)$).
- In contrast, `sorted(nums)` makes a full copy of the list, which doubles the memory footprint of the input data structure. Always prefer in-place sorting in coding interviews unless modifying the input is forbidden.

3.4.2 2. Timsort Auxiliary Space Complexity

- Python's built-in sort algorithm is Timsort. In the worst case, Timsort has a space complexity of $\mathcal{O}(n)$ to store temporary runs, whereas C++'s `std::sort` typically uses Introsort, requiring $\mathcal{O}(\log n)$ auxiliary stack frames.

3.4.3 3. Hashing/Set Deduplication Overhead

- Some developers write a naive 3Sum search and clean up duplicates by adding results to a set of tuples: `result_set.add(tuple(sorted([nums[i], nums[j], nums[k])))`.
- This approach creates large amounts of dynamic object allocations (tuples, lists) and is computationally expensive. Deduplicating on the fly using pointer increments avoids hash collisions, hashing calculations, and garbage collection overhead.

3.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n^2)$	Sorting takes $\mathcal{O}(n \log n)$. The outer loop runs $n-2$ times, and the inner two-pointer loop runs in $\mathcal{O}(n)$ time per iteration, yielding $\mathcal{O}(n^2)$ total.
Space Complexity	$\mathcal{O}(\log n)$ to $\mathcal{O}(n)$	$\mathcal{O}(\log n)$ auxiliary space in C++ (for recursion stack of quicksort/introsort) and up to $\mathcal{O}(n)$ in Python (for Timsort).

3.6 Common Follow-Up Questions & How to Answer

3.6.1 Q1: How do you generalize this approach to solve k-Sum?

- **Answer**: We can solve k-Sum recursively. For any $k > 2$, we sort the array and iterate the first element from index i to $n-k$. We recursively call the $(k-1)$-Sum algorithm on the remaining subarray to find elements that sum to `target - nums[i]`. When k reaches 2, we terminate with the standard two-pointer Two-Sum solver.
- **General Framework (Python)**:

```python
def kSum(nums: List[int], target: int, k: int, start: int) -> List[List[int]]:
    res = []
    if start == len(nums):
        return res
    # The average value of remaining elements must be at least target/k
    if nums[start] * k > target or target > nums[-1] * k:
        return res
    if k == 2:
        # Two Sum
        left, right = start, len(nums) - 1
        while left < right:
```

```
            curr_sum = nums[left] + nums[right]
            if curr_sum < target:
                left += 1
            elif curr_sum > target:
                right -= 1
            else:
                res.append([nums[left], nums[right]])
                while left < right and nums[left] == nums[left + 1]: left += 1
                while left < right and nums[right] == nums[right - 1]: right -= 1
                left += 1
                right -= 1
        return res

    for i in range(start, len(nums) - k + 1):
        if i > start and nums[i] == nums[i - 1]:
            continue
        for subset in kSum(nums, target - nums[i], k - 1, i + 1):
            res.append([nums[i]] + subset)
    return res
```

3.6.2 Q2: Can we solve 3Sum in $\mathcal{O}(n^2)$ time without sorting (using Hash Set)?

- **Answer**: Yes, we can iterate through the array using i, and for each element, perform a Two-Sum check using a hash set.
- To prevent duplicate triplets without sorting the main array, we can insert sorted versions of found triplets into a hash set. However, sorting each triplet takes constant $\mathcal{O}(1)$ time, but the overall hash operations are slower in practice than two pointers and require $\mathcal{O}(n)$ additional space.

3.7 Pro-Tip: How to Impress the Interviewer

- **Leverage Dual-Pointer Short-Circuiting (Early Pruning)**: Mention that we can skip the loop if `nums[i] > 0` (as discussed) and also if `nums[i] + nums[i+1] + nums[i+2] > 0` (since it's the minimum possible sum starting from index i), or if `nums[i] + nums[n-2] + nums[n-1] < 0` (since it's the maximum possible sum starting from index i, meaning we must increment i).
- **Memory Locality & Cache Friendliness**: Discuss that while a hash table approach achieves the same $\mathcal{O}(n^2)$ time complexity, the two-pointer sorted array traversal is significantly faster on modern hardware. This is because contiguous array accesses trigger sequential hardware cache lines, whereas hash table lookups result in pointer chasing and frequent CPU cache misses.
- **Pass by Reference/Rvalue Semantics**: In C++, point out that passing `vector<int> nums` by value is fine if the caller wants to keep their original vector unsorted. However, if the signature allows, taking a reference or modifying the array in-place prevents duplicating the array container.

4 Two Sum II - Input Array Is Sorted

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 167, Glassdoor
- **Flashcards**: Two Pointers deck

4.1 Question Description

Given a **1-indexed** array of integers `numbers` that is already **sorted in non-decreasing order**, find two numbers such that they add up to a specific `target` number. Let these two numbers be `numbers[index1]` and `numbers[index2]` where $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$.

Return *the indices of the two numbers, `index1` and `index2`, added by one as an integer array `[index1, index2]` of length 2.*

The tests are generated such that there is **exactly one solution**. You may not use the same element twice.

Your solution must use only **constant extra space**.

4.1.1 Examples

- **Input**: `numbers = [2,7,11,15]`, `target = 9`
  - **Output**: `[1,2]`
  - **Explanation**: The sum of 2 and 7 is 9. Therefore, `index1 = 1`, `index2 = 2`. We return `[1, 2]`.
- **Input**: `numbers = [2,3,4]`, `target = 6`
  - **Output**: `[1,3]`
  - **Explanation**: The sum of 2 and 4 is 6. Therefore, `index1 = 1`, `index2 = 3`. We return `[1, 3]`.
- **Input**: `numbers = [-1,0]`, `target = -1`
  - **Output**: `[1,2]`
  - **Explanation**: The sum of -1 and 0 is -1. Therefore, `index1 = 1`, `index2 = 2`. We return `[1, 2]`.

4.2 Detailed Solution (C++)

The array is already sorted, which allows us to use the **Two-Pointer technique**:

1. Place one pointer `left` at the start of the array (0) and another pointer `right` at the end of the array $(n-1)$.
2. Compute `sum = numbers[left] + numbers[right]`.
3. Compare the `sum` to the `target`:
   - If `sum == target`, we have found the solution. Return the 1-indexed values `{left + 1, right + 1}`.

- If `sum < target`, we need a larger sum. Since the array is sorted, incrementing `left` moves us to a larger value, so we do `left++`.
- If `sum > target`, we need a smaller sum. Decrementing `right` moves us to a smaller value, so we do `right--`.

4. Since the problem guarantees that exactly one solution exists, this loop is guaranteed to find it.

4.2.1 Standard C++ Production Code

```cpp
#include <vector>

class Solution {
public:
    std::vector<int> twoSum(const std::vector<int>& numbers, int target) noexcept {
        int left = 0;
        int right = static_cast<int>(numbers.size()) - 1;

        while (left < right) {
            // Potential overflow warning: sum can overflow standard 32-bit int if elements are near INT_MAX
            // Under LeetCode constraints (-1000 <= numbers[i] <= 1000), sum is safe from overflow.
            int sum = numbers[left] + numbers[right];

            if (sum == target) {
                return {left + 1, right + 1}; // 1-indexed output
            } else if (sum < target) {
                left++;
            } else {
                right--;
            }
        }

        return {}; // Fallback
    }
};
```

4.3 Detailed Solution (Python)

4.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using standard index boundaries.

```python
from typing import List

class Solution:
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
        """
```

```
    Finds two numbers in a sorted array that sum up to target.
    Returns the 1-based indices of the two numbers.

    Time Complexity: O(N)
    Space Complexity: O(1)
    """
    left: int = 0
    right: int = len(numbers) - 1

    while left < right:
        curr_sum = numbers[left] + numbers[right]

        if curr_sum == target:
            return [left + 1, right + 1]
        elif curr_sum < target:
            left += 1
        else:
            right -= 1

    return []
```

4.4 Python-Specific Complexities & Implementation Considerations

4.4.1 1. Zero-Based vs. One-Based Indexing

- Python arrays (lists) are zero-based. To meet the requirements of LeetCode 167 (1-based output), we must add `1` to both `left` and `right` indices before returning. This is a common point of confusion that is easy to get wrong under interview pressure.

4.4.2 2. Space Efficiency and List Creation

- Returning a list `[left + 1, right + 1]` in Python allocates a new list object containing two integers. This happens only once upon finding the solution, maintaining our overall constant auxiliary space complexity of $\mathcal{O}(1)$.

4.4.3 3. Native Large Integer Precision

- In Python, integer overflow is not a concern because standard integers are automatically promoted to arbitrary-precision long integers.
- If coding in C++ or Java, however, you should mention that if elements could be up to 2×10^9, computing `numbers[left] + numbers[right]` would overflow a 32-bit signed integer. A safe way to write this in lower-level languages is:

```
long long sum = static_cast<long long>(numbers[left]) + numbers[right];
```

4.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Each iteration moves at least one pointer inward, inspecting at most n elements.
Space Complexity	$\mathcal{O}(1)$	The algorithm uses only two index variables, requiring constant auxiliary storage.

4.6 Common Follow-Up Questions & How to Answer

4.6.1 Q1: What if the input array is not sorted?

- **Answer**: If the array is unsorted and we cannot sort it (or want to avoid the $\mathcal{O}(n \log n)$ sorting cost), we use a hash map to map each value to its index. We iterate through the array and check if `target - nums[i]` exists in the hash map.
- This takes $\mathcal{O}(n)$ time complexity, but increases space complexity to $\mathcal{O}(n)$ to store the hash map.

4.6.2 Q2: Can we solve this problem using Binary Search instead of Two Pointers?

- **Answer**: Yes. For each element `numbers[i]`, we can binary search for `target - numbers[i]` in the remaining subarray `[i + 1, n - 1]`.
- **Complexity**: This approach takes $\mathcal{O}(n \log n)$ time and $\mathcal{O}(1)$ space.
- **When is it better?**: If the array is extremely large and we can run searches in parallel, or if the target is such that we can prune the search range rapidly (for example, if one element is near the beginning and the other is guaranteed to be within a small range).

4.7 Pro-Tip: How to Impress the Interviewer

- **Hybrid Pruning Using Binary Search**:
  - If the array is huge ($n \approx 10^9$) but the target is relatively small, we can run a binary search to find the upper bound index of target (since elements greater than target/2 + offset cannot sum to target if all values are positive). This prunes the size of the array that the two pointers need to scan.
- **Discuss Cache Line Prefetching vs. Random Jump**:
  - Explain that the two-pointer approach accesses elements at the start and end of the array sequentially. The left pointer experiences high cache locality due to forward prefetching, while the right pointer can trigger backward prefetching.
- **Explain Branchless Index Updates**:

- You can demonstrate how a compiler might eliminate branching to avoid pipeline flushes.

```
// Conceptual branchless step
int match = (sum == target);
int less = (sum < target);
left += (!match & less);
right -= (!match & !less);
```

5 Is Subsequence

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / ML Engineer
- **Source**: LeetCode 392, Glassdoor
- **Flashcards**: Two Pointers deck

5.1 Question Description

Given two strings `s` and `t`, return `true` *if* `s` *is a subsequence of* `t`*, or* `false` *otherwise.*

A **subsequence** of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., `"ace"` is a subsequence of `"abcde"` while `"aec"` is not).

5.1.1 Examples

- **Input**: `s = "abc"`, `t = "ahbgdc"`
  - **Output**: `true`
- **Input**: `s = "axc"`, `t = "ahbgdc"`
  - **Output**: `false`

5.1.2 Constraints

- $0 \leq$ s.length ≤ 100
- $0 \leq$ t.length $\leq 10^4$
- `s` and `t` consist only of lowercase English letters.

5.2 Detailed Solution (C++)

The problem asks if we can find all characters of `s` in `t` in their respective order. We can solve this in linear time using a **two-pointer approach**:

1. Initialize a pointer `s_idx = 0` to track the current character in `s` we need to match.
2. Iterate through each character `c` in `t`.
3. If `s[s_idx] == c`, we have found a match for the current character, so we increment `s_idx`.

4. **Early Termination**: If `s_idx` equals the length of `s`, it means all characters of `s` have been matched in order. We can return `true` immediately, avoiding processing the rest of `t`.
5. If the loop completes and `s_idx < s.length()`, return `false`.

5.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <cstddef>

class Solution {
public:
    bool isSubsequence(const std::string& s, const std::string& t) noexcept {
        // Edge Case: An empty string is always a subsequence of any string
        if (s.empty()) {
            return true;
        }

        size_t s_idx = 0;
        const size_t s_len = s.length();

        for (const char c : t) {
            if (s[s_idx] == c) {
                s_idx++;
                // Early return if s is fully matched
                if (s_idx == s_len) {
                    return true;
                }
            }
        }

        return false;
    }
};
```

5.3 Detailed Solution (Python)

5.3.1 Standard Python Production Code

Below is the type-hinted Python implementation. It features early termination to optimize performance.

```python
class Solution:
    def isSubsequence(self, s: str, t: str) -> bool:
        """
        Determines if s is a subsequence of t.
```

```
    Time Complexity: O(len(t))
    Space Complexity: O(1)
    """
    # Edge Case: An empty string is always a subsequence of any string
    if not s:
        return True

    s_idx = 0
    s_len = len(s)

    for char in t:
        if s[s_idx] == char:
            s_idx += 1
            # Early return if s is fully matched
            if s_idx == s_len:
                return True

    return False
```

5.4 Python-Specific Complexities & Implementation Considerations

5.4.1 1. The Iterator-Based Pythonic Trick

- Python allows a highly optimized, one-liner solution using the `iter()` function and the `all()` generator:

```
def isSubsequence(self, s: str, t: str) -> bool:
    t_iter = iter(t)
    return all(char in t_iter for char in s)
```

- **How it works**: `iter(t)` creates an iterator over `t`. The expression `char in t_iter` advances the iterator until it finds `char`. Because the iterator state is preserved across checks, `all(...)` ensures each character of `s` is matched in sequence.
- **Pros**: It runs at C-speed internally in CPython, requiring minimal memory allocations.
- **Cons**: It is harder to debug and explain in an interview. Interviewers typically prefer the explicit two-pointer loop.

5.4.2 2. Character Hashing Overhead

- Because python characters are string objects of length 1, doing index-based lookup `s[s_idx]` or iterating has slightly more overhead than traversing raw character arrays in C++. However, this is negligible unless handling extremely large scale strings.

5.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\|t\|)$	In the worst case, we scan the entire string `t` once.
Space Complexity	$\mathcal{O}(1)$	Only index pointers and length variables are allocated.

5.6 Common Follow-Up Questions & How to Answer

5.6.1 Q1: What if there are many incoming strings ($k \approx 10^9$) and we want to check if each is a subsequence of a single, giant string t ($|t| \leq 10^6$)?

- **Answer**: Running the $\mathcal{O}(|t|)$ two-pointer scan for each query would take $\mathcal{O}(k \cdot |t|)$ time, which is too slow.
- **Optimal Solution**: Preprocess t to build an index lookup map.
  1. Construct a hash map (or array of 26 lists) where the keys are characters and the values are lists of indices where that character appears in t (sorted in ascending order).
  2. For a query string s, initialize a variable `prev_idx = -1`.
  3. For each character `c` in `s`, look up its list of indices in the map.
  4. Perform a **Binary Search** (e.g., `std::upper_bound` in C++ or `bisect_right` in Python) on the list to find the smallest index `curr_idx` that is strictly greater than `prev_idx`.
  5. If no such index exists, then `s` is not a subsequence. Otherwise, update `prev_idx = curr_idx` and proceed.
- **Complexity**: Preprocessing takes $\mathcal{O}(|t|)$ time. Each query now takes $\mathcal{O}(|s| \log |t|)$ time. This is a massive improvement when $|s| \ll |t|$ and k is large.

5.7 Pro-Tip: How to Impress the Interviewer

- **Proactively Discuss the Multi-Query Variant**: Start by proposing the basic two-pointer algorithm. Once written, ask: *"Is this going to be called repeatedly on the same target string `t` for many different input strings `s`?"* and then explain the index-lookup binary-search pre-processing optimization.
- **Highlight Cache Friendliness**: String search involves sequential array traversal. Explain that C++'s `std::string` stores data contiguously in virtual memory. This triggers spatial locality in CPU L1/L2 caches, resulting in near-instant memory loads compared to node-based data structures.
- **Explicit Constant References**: Always pass string arguments as `const std::string&` in C++ to prevent deep copies. If the string is short, explain how Small String Optimization (SSO) in C++ standard libraries avoids heap allocation entirely.

6 Heaters

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Algorithm Engineer
- **Source**: LeetCode 475, Glassdoor
- **Flashcards**: Two Pointers deck

6.1 Question Description

Winter is coming! During the contest, your first job is to design a standard heater with a fixed warm radius to warm all the houses.

Every house can be warmed, as long as the house is within the heater's warm radius range.

Given the positions of `houses` and `heaters` on a horizontal line, return *the minimum radius standard of heaters so that those heaters could cover all houses.*

Notice that all the heaters follow your radius standard, and the warm radius will be the same.

6.1.1 Examples

- **Input**: `houses = [1,2,3]`, `heaters = [2]`
  - **Output**: `1`
  - **Explanation**: The only heater is at position 2. If we use a warm radius of 1, all houses ([1, 2, 3]) will be covered.
- **Input**: `houses = [1,2,3,4]`, `heaters = [1,4]`
  - **Output**: `1`
  - **Explanation**: The two heaters are at positions 1 and 4. A radius of 1 is sufficient to cover house 2 (via heater 1) and house 3 (via heater 4).
- **Input**: `houses = [1,5]`, `heaters = [2]`
  - **Output**: `3`
  - **Explanation**: The heater is at position 2. The furthest house is at position 5. The minimum radius needed is $|5 - 2| = 3$.

6.1.2 Constraints

- $1 \leq \text{houses.length}, \text{heaters.length} \leq 3 \times 10^4$
- $1 \leq \text{houses}[i], \text{heaters}[i] \leq 10^9$

6.2 Detailed Solution (C++)

To solve this problem efficiently: 1. **Sort both arrays**: First, sort both `houses` and `heaters` in non-decreasing order. Sorting enables a linear-time two-pointer sweep. 2. **Two-Pointer Scan**: Use index `i` to

iterate through the houses, and index `j` to track the heaters. 3. For each house at `houses[i]`, we find the closest heater by advancing `j` to `j + 1` if:

$$|houses[i] - heaters[j]| \geq |houses[i] - heaters[j+1]|$$

This condition checks if moving to the next heater gives a distance that is smaller than or equal to the current one. 4. Calculate the distance between the house and its closest heater: `abs(houses[i] - heaters[j])`. 5. The required warm radius is the maximum of these minimum distances over all houses.

6.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>
#include <cmath>
#include <cstddef>

class Solution {
public:
    int findRadius(std::vector<int>& houses, std::vector<int>& heaters) {
        // Sort both collections to enable sequential two-pointer scan
        std::sort(houses.begin(), houses.end());
        std::sort(heaters.begin(), heaters.end());

        int min_radius = 0;
        size_t j = 0;
        const size_t num_heaters = heaters.size();

        for (int house : houses) {
            // Advance the heater pointer if the next heater is closer to (or equally close to) the house
            while (j + 1 < num_heaters &&
                   std::abs(house - heaters[j]) >= std::abs(house - heaters[j + 1])) {
                j++;
            }
            // The required radius is the maximum of the minimum distance for each house
            min_radius = std::max(min_radius, std::abs(house - heaters[j]));
        }

        return min_radius;
    }
};
```

6.3 Detailed Solution (Python)

6.3.1 Standard Python Production Code

Below is the type-hinted Python implementation, optimizing pointer movement to achieve linear scan performance after sorting.

```python
from typing import List

class Solution:
    def findRadius(self, houses: List[int], heaters: List[int]) -> int:
        """
        Calculates the minimum warm radius of heaters to cover all houses.

        Time Complexity: O(M log M + N log N) where M = len(houses), N = len(heaters)
        Space Complexity: O(1) auxiliary space (excluding sorting memory)
        """
        houses.sort()
        heaters.sort()

        min_radius = 0
        j = 0
        num_heaters = len(heaters)

        for house in houses:
            # Slide heater index forward if the next heater is closer to the current house
            while j + 1 < num_heaters and abs(house - heaters[j]) >= abs(house - heaters[j + 1]):
                j += 1

            # Record the distance to the closest heater and update min_radius
            distance = abs(house - heaters[j])
            if distance > min_radius:
                min_radius = distance

        return min_radius
```

6.4 Python-Specific Complexities & Implementation Considerations

6.4.1 1. In-place Sorting vs. Copying

- We perform `houses.sort()` and `heaters.sort()` in-place to avoid the allocation overhead of creating new list objects.
- If modifying the inputs is not allowed by the API design, we would have to copy them first, which increases space complexity to $\mathcal{O}(m+n)$.

6.4.2 2. Tight Loop Optimization

- Inside the `while` loop, Python evaluates `abs(house - heaters[j])` and `abs(house - heaters[j + 1])`. In Python, arithmetic operations and function calls have high relative overhead.
- If optimization is critical, we can cache values or pre-calculate differences, but the two-pointer approach keeps the overall loop count bounded by $m + n$, ensuring high efficiency.

6.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(m \log m + n \log n)$	Sorting `houses` takes $\mathcal{O}(m \log m)$ and sorting `heaters` takes $\mathcal{O}(n \log n)$. The two-pointer sweep takes $\mathcal{O}(m + n)$ time.
Space Complexity	$\mathcal{O}(1)$ auxiliary	The algorithm uses a few index variables. Sorting space depends on the language (up to $\mathcal{O}(n)$ in Python's Timsort).

6.6 Common Follow-Up Questions & How to Answer

6.6.1 Q1: Can we solve this problem using Binary Search?

- **Answer**: Yes. Instead of sorting `houses`, we only sort `heaters` ($\mathcal{O}(n \log n)$).
- For each house, we binary search (using `std::lower_bound` in C++ or `bisect_left` in Python) in `heaters` to find the heater closest to it. Specifically:
  1. Find the first heater index `idx` where `heaters[idx] >= house`.
  2. The closest heater will be either `heaters[idx]` (if valid) or `heaters[idx - 1]` (if valid).
  3. Update `min_radius` with the minimum of these two distances.
- **Time Complexity**: $\mathcal{O}(n \log n + m \log n)$.
- **Comparison**: This is better if m is very small compared to n (so sorting houses is avoided), or if the houses array cannot be sorted.

6.6.2 Q2: What if we can place a limited number of heaters anywhere to minimize the radius?

- **Answer**: This is a different problem (related to LeetCode 410 / Split Array Largest Sum). We would binary search over the *radius* itself.
- The search space for radius ranges from 0 to max_pos − min_pos. For a mid radius, we use a greedy pass to see if we can cover all houses with at most k heaters.

6.7 Pro-Tip: How to Impress the Interviewer

- **Propose the Hybrid Decision Boundary**:
  - Explain the trade-off: The two-pointer approach has a time complexity of $\mathcal{O}(m \log m + n \log n)$, whereas the binary search approach has $\mathcal{O}(n \log n + m \log n)$.
  - If $m \ll n$, binary search is clearly superior. If both are large, the two-pointer approach is superior due to sequential CPU cache access (cache locality).
- **Discuss Cache Line Prefetching**:
  - Two pointers traverse memory sequentially. This allows CPU prefetch units to load the next lines of the array into the cache hierarchy before the program requests them, minimizing pipeline stalls.
- **Safeguard Integer subtraction in C++**:
  - Remind the interviewer that if coordinates could be arbitrary unsigned integers, subtraction `house - heaters[j]` could underflow. Using signed types or careful checks prevents wrapping errors.

7 Longest Uncommon Subsequence II

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Algorithm Engineer
- **Source**: LeetCode 522, Glassdoor
- **Flashcards**: Two Pointers deck

7.1 Question Description

Given an array of strings `strs`, return *the length of the longest uncommon subsequence between them.* If the longest uncommon subsequence does not exist, return `-1`.

An **uncommon subsequence** between an array of strings is a string that is a subsequence of one string but **not** the others.

A **subsequence** of a string `s` is a string that can be obtained after deleting any number of characters from `s` without changing the relative order of the remaining characters. (e.g., `"abc"` is a subsequence of `"aebdc"` while `"axc"` is not).

7.1.1 Examples

- **Input**: `strs = ["aba","cdc","eae"]`
  - **Output**: `3`
  - **Explanation**: The longest uncommon subsequence could be `"aba"`, `"cdc"`, or `"eae"`. Each of these has length 3 and is not a subsequence of any other string in the list.
- **Input**: `strs = ["aaa","aaa","aa"]`
  - **Output**: `-1`
  - **Explanation**:

* `"aaa"` appears twice, so it is a subsequence of another `"aaa"`.
* `"aa"` is a subsequence of `"aaa"`.
* Therefore, no uncommon subsequence exists, and we return `-1`.

7.1.2 Constraints

- $2 \leq$ strs.length ≤ 50
- $1 \leq$ strs[i].length ≤ 10
- `strs[i]` consists of lowercase English letters.

7.2 Detailed Solution (C++)

To solve this problem: 1. **Mathematical Observation**: The longest uncommon subsequence must be one of the original strings in the array. If a subsequence X of string A is uncommon, then string A itself must also be uncommon, and since A is at least as long as X, we only need to check the strings in `strs` as candidates. 2. **Sort by Length Descending**: Sort the array of strings by their lengths in descending order. This ensures that the first valid uncommon string we find will be the longest one. 3. **Subsequence Check**: For each candidate string `strs[i]`, verify if it is a subsequence of any other string `strs[j]` (where $i \neq j$). * A string A is a subsequence of B if we can match all characters of A sequentially in B using a two-pointer scan. 4. **Early Termination**: Since the array is sorted by length, the first candidate string `strs[i]` that is **not** a subsequence of any other string is immediately returned as the longest uncommon subsequence length.

7.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <string>
#include <algorithm>
#include <cstddef>

class Solution {
private:
    // Helper function to check if string 'a' is a subsequence of string 'b'
    bool isSubsequence(const std::string& a, const std::string& b) noexcept {
        const size_t len_a = a.length();
        const size_t len_b = b.length();
        if (len_a > len_b) {
            return false;
        }
        if (len_a == 0) {
            return true;
        }

        size_t i = 0;
```

```
        for (const char c : b) {
            if (a[i] == c) {
                i++;
                if (i == len_a) {
                    return true;
                }
            }
        }
        return false;
    }

public:
    int findLUSlength(std::vector<std::string>& strs) {
        // Sort strings by length in descending order
        std::sort(strs.begin(), strs.end(), [](const std::string& a, const std::string& b) {
            return a.length() > b.length();
        });

        const int n = static_cast<int>(strs.size());

        for (int i = 0; i < n; ++i) {
            bool is_uncommon = true;
            for (int j = 0; j < n; ++j) {
                if (i != j && isSubsequence(strs[i], strs[j])) {
                    is_uncommon = false;
                    break; // Candidate is a subsequence of another string, so it cannot be uncommon
                }
            }
            if (is_uncommon) {
                return static_cast<int>(strs[i].length());
            }
        }

        return -1; // No uncommon subsequence found
    }
};
```

7.3 Detailed Solution (Python)

7.3.1 Standard Python Production Code

Below is the type-hinted Python implementation, which sorts in-place using list sorting.

```python
from typing import List

class Solution:
    def findLUSlength(self, strs: List[str]) -> int:
        """
        Finds the length of the longest uncommon subsequence.

        Time Complexity: O(N^2 * L) where N = len(strs), L = max length of strs[i]
        Space Complexity: O(1) auxiliary space (excluding sorting memory)
        """
        def isSubsequence(s1: str, s2: str) -> bool:
            len1, len2 = len(s1), len(s2)
            if len1 > len2:
                return False
            if len1 == 0:
                return True
            i = 0
            for char in s2:
                if s1[i] == char:
                    i += 1
                    if i == len1:
                        return True
            return False

        # Sort strings by length in descending order
        strs.sort(key=len, reverse=True)
        n = len(strs)

        for i in range(n):
            is_uncommon = True
            for j in range(n):
                if i != j and isSubsequence(strs[i], strs[j]):
                    is_uncommon = False
                    break
            if is_uncommon:
                return len(strs[i])

        return -1
```

7.4 Python-Specific Complexities & Implementation Considerations

7.4.1 1. In-place Sorting with `key=len`

- Python allows sorting lists of strings using a custom key like `key=len`. The sort is executed in compiled C code, which is highly efficient. By setting `reverse=True`, we perform descending sorting.

7.4.2 2. Frequency Optimization

- If a string is duplicated in the input (e.g., `["aaa", "aaa"]`), it can never be an uncommon subsequence because it is a subsequence of its duplicate.
- We can optimize the check by pre-calculating the frequency of each string:

```
from collections import Counter
counts = Counter(strs)
# When checking candidate, we can skip if counts[candidate] > 1
```

- This is extremely useful if the input has many duplicate strings.

7.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n^2 \cdot L)$	There are n strings, and for each we check if it is a subsequence of the other $n-1$ strings. Each subsequence check takes $\mathcal{O}(L)$ time, where $L \leq 10$ is the max string length.
Space Complexity	$\mathcal{O}(1)$ auxiliary	The algorithm uses constant auxiliary variables. Space is only consumed by the sorting algorithm stack.

7.6 Common Follow-Up Questions & How to Answer

7.6.1 Q1: Why must the longest uncommon subsequence be one of the strings in the array?

- **Answer**: Suppose the longest uncommon subsequence is a string S that is *not* in the array.
- By definition, S must be a subsequence of some string T in the array.
- If T were a subsequence of some other string in the array, then S (being a subsequence of T) would also be a subsequence of that other string, which contradicts the definition of S being uncommon.
- Therefore, T itself must be uncommon. Since S is a subsequence of T, the length of T is greater than or equal to the length of S. Thus, testing the elements of the array themselves is sufficient to find the maximum possible length.

7.6.2 Q2: How can we optimize this if n is large ($n \approx 10^5$) but string lengths remain small ($L \leq 10$)?

- **Answer**: If n is very large, the $\mathcal{O}(n^2)$ pairwise comparison is too slow. We can group identical strings and filter out duplicates.
- Then, we can represent each string as a path in a Trie or use a hash map to skip strings that are sub-parts of previously verified strings.

7.7 Pro-Tip: How to Impress the Interviewer

- **Use the Duplicate Frequency Filter Proactively**:
  - Point out that duplicate strings can be filtered out. In C++, we can use a hash map `std::unordered_map<std::string, int>` to count occurrences. If a string has a frequency > 1, we mark it as invalid. This shows that you are thinking about real-world database inputs where duplicates are common.
- **Explain the Subsequence Search Early Termination**:
  - If checking if string A is a subsequence of B, we can immediately return `false` if `A.length() > B.length()`. In our nested loop, since strings are sorted by length descending, for any $j < i$, `strs[j]` is longer than or equal to `strs[i]`. So the check is only needed for $j < i$. If $j > i$, `strs[i]` cannot be a subsequence of `strs[j]` unless they are identical, which we already filter out. This reduces the number of subsequence checks by half!

8 Longest Word in Dictionary through Deleting

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Algorithm Engineer
- **Source**: LeetCode 524, Glassdoor
- **Flashcards**: Two Pointers deck

8.1 Question Description

Given a string `s` and a string array `dictionary`, return *the longest string in the dictionary that can be formed by deleting some of the given string characters.* If there is more than one possible result, return the longest word with the smallest lexicographical order. If there is no possible result, return the empty string.

8.1.1 Examples

- **Input**: `s = "abpcplea"`, `dictionary = ["ale","apple","monkey","plea"]`
  - **Output**: `"apple"`
  - **Explanation**: Both `"ale"` and `"apple"` can be formed by deleting characters from `s`. Since `"apple"` is longer than `"ale"`, it is selected.

- **Input**: `s = "abpcplea"`, `dictionary = ["a","b","c"]`
  - **Output**: `"a"`
  - **Explanation**: `"a"`, `"b"`, and `"c"` can all be formed. `"a"` is chosen because it is lexicographically smallest.

8.1.2 Constraints

- $1 \leq$ s.length ≤ 1000
- $1 \leq$ dictionary.length ≤ 1000
- $1 \leq$ dictionary$[i]$.length ≤ 1000
- `s` and `dictionary[i]` consist of lowercase English letters.

8.2 Detailed Solution (C++)

A direct linear scan through the dictionary is highly efficient if we apply **early pruning**:

1. Initialize `best` to an empty string `""`.
2. For each `word` in the `dictionary`:
   - **Pruning Check**: Before checking if `word` is a subsequence of `s`, compare its length and lexicographical order with `best`. If `word` is shorter than `best`, or if it is of equal length but lexicographically greater than or equal to `best`, then even if it is a subsequence, it cannot become the new `best`. We can immediately `continue` to the next word.
   - **Subsequence Check**: If the word passes the pruning check, use the two-pointer technique to check if it is a subsequence of `s`.
   - If it is a subsequence, update `best = word`.
3. Return `best`.

8.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <string>
#include <algorithm>
#include <cstddef>

class Solution {
private:
    // Helper function to check if 'word' is a subsequence of 's'
    bool isSubsequence(const std::string& word, const std::string& s) noexcept {
        const size_t w_len = word.length();
        const size_t s_len = s.length();

        // If the word is longer than the source string, it cannot be a subsequence
        if (w_len > s_len) {
            return false;
        }
```

```
        if (w_len == 0) {
            return true;
        }

        size_t w_idx = 0;
        for (const char c : s) {
            if (word[w_idx] == c) {
                w_idx++;
                if (w_idx == w_len) {
                    return true;
                }
            }
        }
        return false;
    }

public:
    std::string findLongestWord(const std::string& s, const std::vector<std::string>& dictionary) {
        std::string best = "";

        for (const std::string& word : dictionary) {
            // Pruning: Skip checking if the word cannot beat the current 'best'
            if (word.length() < best.length() ||
                (word.length() == best.length() && word >= best)) {
                continue;
            }

            if (isSubsequence(word, s)) {
                best = word;
            }
        }

        return best;
    }
};
```

8.3 Detailed Solution (Python)

8.3.1 Standard Python Production Code

Below is the type-hinted Python implementation, leveraging short-circuit logic to minimize subsequence checks.

```python
from typing import List

class Solution:
    def findLongestWord(self, s: str, dictionary: List[str]) -> str:
        """
        Finds the longest word in dictionary that is a subsequence of s.

        Time Complexity: O(N * L) where N = len(dictionary), L = len(s)
        Space Complexity: O(1) auxiliary space
        """
        def isSubsequence(word: str, target: str) -> bool:
            w_len = len(word)
            t_len = len(target)
            if w_len > t_len:
                return False
            if w_len == 0:
                return True

            w_idx = 0
            for char in target:
                if word[w_idx] == char:
                    w_idx += 1
                    if w_idx == w_len:
                        return True
            return False

        best = ""

        for word in dictionary:
            # Prune: Skip checks for words that cannot improve our result
            if len(word) < len(best) or (len(word) == len(best) and word >= best):
                continue

            if isSubsequence(word, s):
                best = word

        return best
```

8.4 Python-Specific Complexities & Implementation Considerations

8.4.1 1. Sorting the Dictionary (Trade-off)

- An alternative implementation is to sort the dictionary beforehand:

```
dictionary.sort(key=lambda x: (-len(x), x))
```

This sorts the dictionary by length descending, and then lexicographically ascending. The first word in the sorted dictionary that is a subsequence of `s` is the correct answer.

- **Trade-off**: Sorting takes $\mathcal{O}(n \log n \cdot W)$ time (where W is the maximum word length). Our linear scan approach takes $\mathcal{O}(n \cdot L_s)$ time. Since $n \leq 1000$ and $L_s \leq 1000$, the linear scan with pruning is much faster in practice and avoids memory allocations for sorting.

8.4.2 2. Lexicographical Comparison

- In Python, string comparisons like `word >= best` are implemented at the C level in CPython and run very fast. Since it checks character-by-character, it will return as soon as a mismatch is found, saving execution cycles.

8.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \cdot L_s)$	We perform at most n subsequence checks. Each check scans string `s` of length L_s. Pruning makes the average case much faster.
Space Complexity	$\mathcal{O}(1)$ auxiliary	The algorithm uses only index pointers and variables to track the best word, requiring constant space.

8.6 Common Follow-Up Questions & How to Answer

8.6.1 Q1: What if we have many queries of string `s`? How can we preprocess the dictionary?

- **Answer**: If the dictionary is static and we receive multiple source strings `s`, we can build a **Trie** from the dictionary.
- For each query string `s`, we perform a DFS search over the Trie, matching characters of `s`. This allows us to search multiple dictionary words simultaneously, pruning entire branches of words that do not match the character stream.

8.6.2 Q2: What if the dictionary contains billions of words?

- **Answer**: If the dictionary is too large to fit in memory, we can stream the dictionary from external storage.
- Because our linear scan maintains only a single `best` string of size up to 1000 bytes, the space complexity remains $\mathcal{O}(1)$ auxiliary, making it highly suitable for streaming/out-of-core computation.

8.7 Pro-Tip: How to Impress the Interviewer

- **Proactively Highlight Pruning Logic**:
  - Point out that checking `word.length() < best.length()` first saves substantial CPU cycles because it bypasses the linear character-matching loop for any word that is too short.
- **Explain Small String Optimization (SSO)**:
  - In C++, explain that if dictionary strings are small (less than 15 characters on most 64-bit systems), they are stored directly inside the string control block rather than on the heap. This prevents heap allocations during dictionary iteration and yields excellent L1/L2 cache hit ratios.
- **State Space Bounds of the Return String**:
  - If the interviewer asks about return-value copy overhead, explain that returning `std::string` from a function in C++11 and newer leverages **Move Semantics** or **Return Value Optimization (RVO)**, making the return operation $\mathcal{O}(1)$ instead of copying the string bytes.

9 K-diff Pairs in an Array

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Algorithm Engineer
- **Source**: LeetCode 532, Glassdoor
- **Flashcards**: Two Pointers deck

9.1 Question Description

Given an array of integers `nums` and an integer `k`, return *the number of **unique** k-diff pairs in the array.*

A **k-diff pair** is an integer pair `(nums[i], nums[j])`, where the following are true: * $0 \leq i, j <$ nums.length * $i \neq j$ * $|\text{nums}[i] - \text{nums}[j]| == k$

Notice that $|val|$ denotes the absolute value of val.

9.1.1 Examples

- **Input**: `nums = [3,1,4,1,5]`, `k = 2`
  - **Output**: `2`
  - **Explanation**: There are two 2-diff pairs in the array, `(1, 3)` and `(3, 5)`. Although we have two `1`s in the input, we should only return the number of unique pairs.
- **Input**: `nums = [1,2,3,4,5]`, `k = 1`
  - **Output**: `4`
  - **Explanation**: There are four 1-diff pairs in the array: `(1, 2)`, `(2, 3)`, `(3, 4)`, and `(4, 5)`.
- **Input**: `nums = [1,3,1,5,4]`, `k = 0`
  - **Output**: `1`
  - **Explanation**: There is one 0-diff pair in the array, `(1, 1)`.

9.1.2 Constraints

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^7 \leq \text{nums}[i] \leq 10^7$
- $0 \leq k \leq 10^7$

9.2 Detailed Solution (C++)

To solve this problem with $\mathcal{O}(1)$ auxiliary space, we sort the array and apply the **Two-Pointer technique**:

1. **Edge Case**: If $k < 0$, it is impossible for absolute differences to be negative. Return `0`.
2. **Sort the Array**: Sorting makes it easy to find target differences and skip duplicates.
3. **Initialize Pointers**: Place `left = 0` and `right = 1`. Both pointers must be distinct (`left < right`).
4. **Iterate**:
   - Calculate `diff = nums[right] - nums[left]`.
   - If `diff < k`, we need a larger difference, so we increment `right`.
   - If `diff > k`, we need a smaller difference, so we increment `left`.
   - If `diff == k`, we have found a valid pair. We increment `count` and skip duplicate elements:
     - Save the values of `nums[left]` and `nums[right]`, then advance both `left` and `right` past all occurrences of these values.
   - **Collision Safety**: If `left == right`, we must increment `right` to maintain the constraint $i \neq j$.

9.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    int findPairs(std::vector<int>& nums, int k) {
        // Absolute differences cannot be negative
        if (k < 0) {
            return 0;
        }

        // Sort the array in-place to use two pointers
        std::sort(nums.begin(), nums.end());

        int count = 0;
        int left = 0;
        int right = 1;
        const int n = static_cast<int>(nums.size());

        while (right < n) {
            int diff = nums[right] - nums[left];
```

```
            if (diff < k) {
                right++;
            } else if (diff > k) {
                left++;
            } else {
                // Found a unique k-diff pair
                count++;

                // Skip all duplicates for the current pair to avoid double counting
                int left_val = nums[left];
                int right_val = nums[right];

                while (left < n && nums[left] == left_val) {
                    left++;
                }
                while (right < n && nums[right] == right_val) {
                    right++;
                }
            }

            // Ensure pointers don't overlap (since we need distinct indices i != j)
            if (left == right) {
                right++;
            }
        }

        return count;
    }
};
```

9.3 Detailed Solution (Python)

9.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using sorting and two pointers.

```
from typing import List

class Solution:
    def findPairs(self, nums: List[int], k: int) -> int:
        """
        Returns the number of unique k-diff pairs in nums.
```

```
    Time Complexity: O(N log N)
    Space Complexity: O(1) auxiliary (excluding sorting space)
    """
    if k < 0:
        # Absolute difference cannot be negative
        return 0

    # Sort in-place to avoid copying overhead
    nums.sort()

    count = 0
    left, right = 0, 1
    n = len(nums)

    while right < n:
        diff = nums[right] - nums[left]

        if diff < k:
            right += 1
        elif diff > k:
            left += 1
        else:
            count += 1
            left_val, right_val = nums[left], nums[right]

            # Advance pointers past duplicates
            while left < n and nums[left] == left_val:
                left += 1
            while right < n and nums[right] == right_val:
                right += 1

        # Pointers cannot point to the same index
        if left == right:
            right += 1

    return count
```

9.4 Python-Specific Complexities & Implementation Considerations

9.4.1 1. Hash Map Alternative (Pythonic Solution)

- In Python, hash map lookups are extremely fast. We can trade space for time by using a frequency dictionary (`Counter`):

```python
from collections import Counter

class Solution:
    def findPairs(self, nums: List[int], k: int) -> int:
        if k < 0:
            return 0
        counts = Counter(nums)
        if k == 0:
            return sum(1 for x in counts if counts[x] > 1)
        return sum(1 for x in counts if (x + k) in counts)
```

- **Pros**: This reduces time complexity to $\mathcal{O}(n)$ and is very clean and readable.
- **Cons**: It increases auxiliary space complexity to $\mathcal{O}(n)$ to store the frequencies.

9.4.2 2. Large List Memory Footprint

- Python's `Counter` allocates hash tables with bucket overheads. For 10^5 elements, the two-pointer approach on a sorted list uses significantly less memory than allocating hash maps, making sorting preferable when memory is constrained.

9.5 Complexity Analysis

Metric	Two-Pointer	Hash Map / Set	Description
Time Complexity	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Sorting dominates two-pointer time, whereas hash map lookups take average $\mathcal{O}(1)$ time per element.
Space Complexity	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Two-pointer uses only loop counters. Hash Map stores frequencies for all unique numbers.

9.6 Common Follow-Up Questions & How to Answer

9.6.1 Q1: What if we have a stream of incoming numbers and need to query the count of unique pairs dynamically?

- **Answer**: We should maintain a hash map of counts of the elements we have seen so far, along with a `pair_count` counter.
- When a new number `x` is added:
  1. If `k == 0`:

- Increment `counts[x]`. If `counts[x] == 2`, increment `pair_count` by 1.

2. If `k > 0`:
   - If `x` is seen for the first time (`counts[x] == 0`):
     * If `x - k` is in `counts`, increment `pair_count`.
     * If `x + k` is in `counts`, increment `pair_count`.
   - Increment `counts[x]`.

- This allows updates and queries to be performed in $\mathcal{O}(1)$ average time.

9.6.2 Q2: What if we want to return the actual pairs rather than just their count?

- **Answer**:
  - In the two-pointer approach, we can simply store `(nums[left_val], nums[right_val])` in a list before skipping duplicates.
  - In the Hash Map approach, we collect `(x, x + k)` for all valid keys.

9.7 Pro-Tip: How to Impress the Interviewer

- **Propose the Space-Time Trade-off Early**:
  - Don't just implement one solution. State both the $\mathcal{O}(n)$ time / $\mathcal{O}(n)$ space Hash Map solution and the $\mathcal{O}(n \log n)$ time / $\mathcal{O}(1)$ space Two-Pointer solution. Let the interviewer choose which resource constraint (CPU vs. RAM) to prioritize.
- **Explain Integer Safety Range**:
  - In C++, point out that the subtraction `nums[right] - nums[left]` is safe from integer overflow because values are between -10^7 and 10^7. The maximum difference is 2×10^7, which fits comfortably in a standard 32-bit signed integer.
- **Address Cache Line Coherence**:
  - Explain that sorting the array and running the two-pointer scan has excellent spatial locality. The CPU can fetch sequential memory addresses into cache line buffers, whereas Hash Map bucket pointers cause random memory jumps and potential cache thrashing.

10 Number of Segments in a String

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 434, Glassdoor
- **Flashcards**: String deck

10.1 Question Description

Given a string `s`, return the number of segments in the string.

A **segment** is defined to be a contiguous sequence of non-space characters.

10.1.1 Examples

- **Input**: `s = "Hello, my name is John"`
  - **Output**: `5`
  - **Explanation**: The five segments are `["Hello,", "my", "name", "is", "John"]`.
- **Input**: `s = "Hello"`
  - **Output**: `1`

10.2 Detailed Solution (C++)

To solve this in **one pass** with $\mathcal{O}(1)$ auxiliary space, we can model this as a simple two-state machine: * State 1: We are outside a segment (encountering spaces). * State 2: We are inside a segment (encountering non-space characters).

When we iterate through the string: * If we find a non-space character while currently *not* in a segment, it indicates the start of a new segment. We increment our counter and set `in_segment` to `true`. * If we find a space character, we set `in_segment` to `false`.

Alternatively, we can count the number of characters that are not spaces and are either at index `0` or preceded by a space.

10.2.1 Standard C++ Production Code

```cpp
#include <string>

class Solution {
public:
    int countSegments(const std::string& s) noexcept {
        int count = 0;
        bool in_segment = false;

        for (const char c : s) {
            if (c != ' ') {
                if (!in_segment) {
                    count++;
                    in_segment = true;
                }
            } else {
                in_segment = false;
            }
        }

        return count;
    }
};
```

10.3 Detailed Solution (Python)

10.3.1 Standard Python Production Code

Below is the optimized single-pass state-machine approach in Python, followed by a discussion on Python's built-in split approach.

```python
class Solution:
    def countSegments(self, s: str) -> int:
        """
        Counts segments in a string without creating temporary string objects.

        Time Complexity: O(N)
        Space Complexity: O(1)
        """
        count = 0
        in_segment = False

        for ch in s:
            if ch != " ":
                if not in_segment:
                    count += 1
                    in_segment = True
            else:
                in_segment = False

        return count
```

10.4 Python-Specific Complexities & Implementation Considerations

10.4.1 1. Built-in `split()` vs State Machine

- A naive Python solution is `len(s.split())`. By default, `s.split()` splits by any whitespace run.
- **Performance Tradeoff**: While `len(s.split())` is very concise and written in highly optimized C under the hood, it creates a list containing the split substrings. If the input string is very large (e.g. several megabytes), this creates an auxiliary array of strings, leading to $\mathcal{O}(n)$ memory allocation and high garbage collection overhead.
- Our state machine approach maintains $\mathcal{O}(1)$ auxiliary space and avoids creating any new string objects, which is critical for low-level performance.

10.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We scan the string of length n exactly once.
Space Complexity	$\mathcal{O}(1)$	Only a few primitive variables (`count`, `in_segment`) are allocated.

10.6 Common Follow-Up Questions & How to Answer

10.6.1 Q1: What if segments are separated by arbitrary whitespace (tabs, newlines, carriage returns)?

- **Answer**: In C++, we should replace the check `c != ' '` with `!std::isspace(static_cast<unsigned char>(c))`. In Python, we can check `not ch.isspace()`.

10.6.2 Q2: How can we implement this if the input string is a stream (e.g., reading from a network socket or file) that doesn't fit in memory?

- **Answer**: Because the algorithm only looks at the current character and a single boolean state flag, it is perfectly suited for streaming. We can read the stream character-by-character (or in chunks) and update the state.
- **C++ Stream Example**:

```cpp
int countSegments(std::istream& stream) {
    int count = 0;
    bool in_segment = false;
    char c;
    while (stream.get(c)) {
        if (!std::isspace(static_cast<unsigned char>(c))) {
            if (!in_segment) {
                count++;
                in_segment = true;
            }
        } else {
            in_segment = false;
        }
    }
    return count;
}
```

10.7 Pro-Tip: How to Impress the Interviewer

- **Critique Built-in Split Libraries**: Proactively mention that while using library functions like `std::stringstream` or Python's `split()` is easy to write, they copy parts of the string and allocate memory. Highlighting the memory allocation downside shows you prioritize efficiency.
- **Character Classification Safety**: In C++, when using `std::isspace`, always mention that you must cast the character to `unsigned char` first. Standard C library functions like `isspace` exhibit undefined behavior if passed a negative value, which can happen with signed characters representing non-ASCII or extended characters (e.g., ISO-8859-1). Showing this level of safety is a massive differentiator.

11 License Key Formatting

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 482, Glassdoor
- **Flashcards**: String deck

11.1 Question Description

You are given a license key represented as a string `s` that consists of only alphanumeric characters and dashes. The string is separated into $n + 1$ groups by n dashes. You are also given an integer `k`.

We want to reformat the string `s` such that each group contains exactly `k` characters, except for the first group, which could be shorter than `k` but still must contain at least one character. Furthermore, there must be a dash inserted between two groups, and you should convert all lowercase letters to uppercase.

Return the reformatted license key.

11.1.1 Examples

- **Input**: `s = "5F3Z-2e-9-w"`, `k = 4`
  - **Output**: `"5F3Z-2E9W"`
  - **Explanation**: The string `s` has been split into two parts, each part has 4 characters. Note that the two extra dashes in the original string are not needed and can be removed.
- **Input**: `s = "2-5g-3-J"`, `k = 2`
  - **Output**: `"2-5G-3J"`
  - **Explanation**: The string `s` has been split into three parts, each part has 2 characters except the first part as it could be shorter as mentioned above.

11.2 Detailed Solution (C++)

A naive approach involves removing all dashes, converting letters to uppercase, and then slicing/grouping. However, this requires multiple allocations and string copies.

An optimized approach is to traverse the string **backwards**: 1. Since the first group is allowed to be shorter than k and all subsequent groups must be exactly k characters, we can build the groups from the end. 2. We loop through `s` from the last index down to `0`. 3. If the character is not a dash: * If we have already gathered `k` characters for the current group, we append a dash `-` to our result and reset the counter. * We convert the character to uppercase and append it to our result. 4. Once the loop completes, the generated string is in reverse order. We reverse it once before returning.

This algorithm achieves $\mathcal{O}(n)$ time complexity and uses only one allocation for the output string.

11.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <cctype>
#include <algorithm>

class Solution {
public:
    std::string licenseKeyFormatting(const std::string& s, int k) noexcept {
        std::string result;
        result.reserve(s.length());
        int char_count = 0;

        // Traverse the string in reverse to easily handle the shorter first group
        for (int i = static_cast<int>(s.length()) - 1; i >= 0; --i) {
            if (s[i] != '-') {
                // If we have completed a group of size k, insert a dash first
                if (char_count == k) {
                    result += '-';
                    char_count = 0;
                }
                // Append the uppercase character
                result += static_cast<char>(std::toupper(static_cast<unsigned char>(s[i])));
                char_count++;
            }
        }

        // The result is currently built backwards, so reverse it
        std::reverse(result.begin(), result.end());
        return result;
    }
};
```

11.3 Detailed Solution (Python)

11.3.1 Standard Python Production Code

Below is the Python implementation. We can use a simple backward traversal with a list of characters, or we can use Python's built-in string manipulation which runs very fast due to C-level optimizations.

```python
class Solution:
    def licenseKeyFormatting(self, s: str, k: int) -> str:
        """
        Formats a license key using backward traversal.

        Time Complexity: O(N)
        Space Complexity: O(N) to store the result list.
        """
        result = []
        char_count = 0

        # Traverse backwards
        for i in range(len(s) - 1, -1, -1):
            char = s[i]
            if char != '-':
                if char_count == k:
                    result.append('-')
                    char_count = 0
                result.append(char.upper())
                char_count += 1

        # Reconstruct the string in correct order
        return "".join(reversed(result))
```

11.4 Python-Specific Complexities & Implementation Considerations

11.4.1 1. Direct String Manipulation (`replace` and Slicing)

- An alternative Python approach is:

  ```python
  cleaned = s.replace("-", "").upper()
  first_len = len(cleaned) % k
  # build groups...
  ```

- **Comparison**: `s.replace("-", "").upper()` runs highly optimized C-code inside Python's runtime. For typical LeetCode string lengths, it is often faster than a Python-level `for` loop because loops in Python have interpreter overhead.

- However, in interviews, explaining the backward traversal is preferred because it generalizes to low-level, memory-restricted systems where you cannot allocate multiple intermediate copies of a string.

11.4.2 2. Avoiding Prepending to String

- In Python, strings are immutable. Doing `result = char + result` inside a loop takes $\mathcal{O}(n^2)$ time because a new string is copied in memory each time.
- Always use a `list` to append elements, then call `join()` at the end, which is an $\mathcal{O}(n)$ operation.

11.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We scan the string of length n once. Reversing the output string also takes $\mathcal{O}(n)$ time.
Space Complexity	$\mathcal{O}(n)$	We allocate memory for the formatted result string, which can be at most $n + \lfloor n/k \rfloor$ in length. Auxiliary space is $\mathcal{O}(1)$ beyond the output buffer.

11.6 Common Follow-Up Questions & How to Answer

11.6.1 Q1: What if k is larger than the number of alphanumeric characters in the string?

- **Answer**: The modulo operation `len(cleaned) % k` or the backward traversal naturally handles this. If the count of characters is less than k, they will all form a single group without any dashes, which is correct (as the first group can be shorter than k).

11.6.2 Q2: How would you optimize this for a system with strict memory limits (e.g. formatting a string in-place)?

- **Answer**: If we are allowed to modify the input string `s` in-place, and we have enough space allocated at the end of `s`, we can use two pointers starting from the end of `s` and the end of the target size, moving characters backwards to avoid overwriting elements. Since `s` has dashes, the final string is usually shorter or equal in length to the original string (unless the original string has very few dashes, in which case we might need to expand it).

11.7 Pro-Tip: How to Impress the Interviewer

- **Highlight String Immutability Costs**: Discuss how string concatenation (`s = s + c`) in languages with immutable strings (like Python or Java) is an anti-pattern. Show your understanding of memory

overhead by always using list appending or pre-allocating buffers.

- **Identify the Backward Traversal Trick**: Point out that formatting from the front is complex because the size of the first group is variable (`n % k`). Traversing backwards simplifies the logic significantly because all groups built from the end are guaranteed to be of size `k`, leaving the remainder for the first group automatically.

12 Detect Capital

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 520, Glassdoor
- **Flashcards**: String deck

12.1 Question Description

We define the usage of capitals in a word to be right when one of the following cases holds: 1. All letters in this word are capitals, like `"USA"`. 2. All letters in this word are not capitals, like `"leetcode"`. 3. Only the first letter in this word is capital, like `"Google"`.

Given a string `word`, return `true` if the usage of capitals in it is right.

12.1.1 Examples

- **Input**: `word = "USA"`
  - **Output**: `true`
- **Input**: `word = "FlaG"`
  - **Output**: `false`

12.2 Detailed Solution (C++)

The rules can be consolidated by counting the total number of uppercase letters in the word: * If **all** letters are uppercase, it is valid (`upper == len`). * If **zero** letters are uppercase, it is valid (`upper == 0`). * If **exactly one** letter is uppercase and it is the **first** letter, it is valid (`upper == 1 && isupper(word[0])`). * In all other cases, the capital usage is invalid.

We can solve this in a single pass over the string in $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ auxiliary space.

12.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <cctype>

class Solution {
```

```
public:
    bool detectCapitalUse(const std::string& word) noexcept {
        const int len = static_cast<int>(word.size());
        if (len <= 1) {
            return true;
        }

        int upper_count = 0;
        for (const char c : word) {
            if (std::isupper(static_cast<unsigned char>(c))) {
                upper_count++;
            }
        }

        // Case 1: All letters are capitals
        // Case 2: No letters are capitals
        // Case 3: Only the first letter is capital
        return (upper_count == len) ||
               (upper_count == 0) ||
               (upper_count == 1 && std::isupper(static_cast<unsigned char>(word[0])));
    }
};
```

12.3 Detailed Solution (Python)

12.3.1 Standard Python Production Code

Below are two approaches to solve the problem in Python. The first approach is an elegant pythonic one-liner, and the second is an optimized single-pass loop that avoids string slicing.

```
class Solution:
    def detectCapitalUse(self, word: str) -> bool:
        """
        Pythonic approach using built-in methods.

        Time Complexity: O(N)
        Space Complexity: O(N) due to string slicing.
        """
        # word.isupper() -> all uppercase
        # word.islower() -> all lowercase
        # word[1:].islower() -> handles "Google" and all lowercase
        return word.isupper() or word.islower() or word[1:].islower()

    def detectCapitalUseOptimized(self, word: str) -> bool:
```

```
    """
    Optimized approach avoiding slicing to achieve O(1) auxiliary space.

    Time Complexity: O(N)
    Space Complexity: O(1)
    """
    n = len(word)
    if n <= 1:
        return True

    upper_count = 0
    for ch in word:
        if ch.isupper():
            upper_count += 1

    return (upper_count == n) or \
           (upper_count == 0) or \
           (upper_count == 1 and word[0].isupper())
```

12.4 Python-Specific Complexities & Implementation Considerations

12.4.1 1. Slicing Memory Overhead

- The expression `word[1:]` creates a **shallow copy** of the substring. In Python, this requires memory allocation proportional to the length of the string $(n-1)$.
- For constraints where $n \leq 100$, this overhead is negligible. However, for huge strings, slicing should be avoided to prevent garbage collection load.

12.4.2 2. Early-Exit Optimization

- We can optimize the search by exiting early as soon as we detect a violation.
- For example, if the first letter is lowercase, all subsequent letters *must* be lowercase. The moment we find an uppercase letter after the first character, we can return `False` immediately.
- Similarly, if the first two letters are uppercase, all subsequent letters must be uppercase. The moment we find a lowercase letter, we can return `False`.

12.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We scan the string of length n at most once.

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(1)$	Only a count accumulator is maintained, requiring constant memory. (Or $\mathcal{O}(n)$ for the pythonic slicing approach).

12.6 Common Follow-Up Questions & How to Answer

12.6.1 Q1: Can you write this using a regular expression?

- **Answer**: Yes, the rules map directly to a simple regex pattern.
- **Regex Pattern**: `^[A-Z]+$|^[a-z]+$|^[A-Z][a-z]*$`
- **Explanation**: This matches either a string of all capitals, a string of all lowercase, or a string starting with a capital followed by zero or more lowercase letters. Note that regex engines typically run slower than manual loops because of state machine overhead.

12.6.2 Q2: How can we implement this with an early-exit check to minimize character scans?

- **Answer**: We can inspect the first two characters to set our expectation:
  - If the first character is lowercase, all remaining characters must be lowercase.
  - If the first two characters are uppercase, all remaining characters must be uppercase.
  - Otherwise (first is uppercase, second is lowercase), all remaining characters must be lowercase.
- **C++ Early-Exit Code**:

```cpp
bool detectCapitalUse(const std::string& word) {
    const int n = static_cast<int>(word.length());
    if (n <= 1) return true;

    const bool expect_upper = std::isupper(static_cast<unsigned char>(word[0])) &&
                              std::isupper(static_cast<unsigned char>(word[1]));

    for (int i = 1; i < n; ++i) {
        if (expect_upper && std::islower(static_cast<unsigned char>(word[i]))) return false;
        if (!expect_upper && std::isupper(static_cast<unsigned char>(word[i]))) return false;
    }
    return true;
}
```

12.7 Pro-Tip: How to Impress the Interviewer

- **Explain Python's String Copying Behavior**: Point out that slicing strings in Python (`word[1:]`) creates a copy, and explain how an interviewer's question on memory constraints might make a slicing approach unacceptable despite its simplicity.
- **Highlight Early Exit / Branching**: Discussing the early-exit optimization shows you care about average-case performance. In typical english text, most words are lowercase or capitalized ("Google"), so setting expectations early allows the processor to skip scanning the whole string for the majority of inputs.

13 Longest Uncommon Subsequence I

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 521, Glassdoor
- **Flashcards**: String deck

13.1 Question Description

Given two strings `a` and `b`, return the length of the **longest uncommon subsequence** between `a` and `b`. If no such uncommon subsequence exists, return `-1`.

An **uncommon subsequence** between two strings is a string that is a subsequence of **exactly one** of them.

A **subsequence** of a string is a new string generated from the original string by deleting some (or no) characters without changing the relative order of the remaining characters. For example, `"ace"` is a subsequence of `"abcde"` while `"aec"` is not.

13.1.1 Examples

- **Input**: `a = "aba"`, `b = "cdc"`
  - **Output**: `3`
  - **Explanation**: One longest uncommon subsequence is `"aba"` because `"aba"` is a subsequence of `"aba"` but not `"cdc"`. Note that `"cdc"` is also a longest uncommon subsequence.
- **Input**: `a = "aaa"`, `b = "bbb"`
  - **Output**: `3`
  - **Explanation**: The longest uncommon subsequences are `"aaa"` and `"bbb"`.
- **Input**: `a = "aaa"`, `b = "aaa"`
  - **Output**: `-1`
  - **Explanation**: Every subsequence of string `a` is also a subsequence of string `b`. Similarly, every subsequence of string `b` is also a subsequence of string `a`. So the answer is `-1`.

13.2 Detailed Solution (C++)

This is a **logical riddle** rather than a complex algorithm problem. Let's break down the logic:

1. **If the strings are identical (`a == b`)**:
   - Any subsequence of `a` is also a subsequence of `b`.
   - Therefore, there is no subsequence that belongs to exactly one of them. The answer is `-1`.
2. **If the strings are different (`a != b`)**:
   - If `a.length() != b.length()`, let's assume `a` is the longer string. The entire string `a` is a subsequence of itself, but it **cannot** be a subsequence of `b` because a shorter string (`b`) cannot contain a longer string (`a`) as a subsequence. Thus, the longest uncommon subsequence is `a` itself, and its length is `a.length()`.
   - If `a.length() == b.length()` but they are not equal, then `a` cannot be a subsequence of `b`. (If it were, they would have to be identical, which contradicts `a != b`). Thus, `a` is an uncommon subsequence of length `a.length()`.

In summary: * If `a == b`, return `-1`. * Otherwise, return `std::max(a.length(), b.length())`.

13.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <algorithm>

class Solution {
public:
    int findLUSlength(const std::string& a, const std::string& b) noexcept {
        // If both strings are equal, no uncommon subsequence can exist
        if (a == b) {
            return -1;
        }

        // Otherwise, the longer string itself is the longest uncommon subsequence
        return static_cast<int>(std::max(a.length(), b.length()));
    }
};
```

13.3 Detailed Solution (Python)

13.3.1 Standard Python Production Code

Below is the Python implementation matching the mathematical logic.

```python
class Solution:
    def findLUSlength(self, a: str, b: str) -> int:
        """
        Finds the length of the longest uncommon subsequence.
```

```
    Time Complexity: O(min(N, M)) where N and M are the lengths of a and b.
    Space Complexity: O(1)
    """
    if a == b:
        return -1
    return max(len(a), len(b))
```

13.4 Python-Specific Complexities & Implementation Considerations

13.4.1 1. String Equality Complexity

- The comparison `a == b` first checks if the string lengths are equal. If the lengths differ, it returns `False` in $\mathcal{O}(1)$ time.
- If the lengths are equal, it compares character by character in $\mathcal{O}(d)$ time (where d is the length of the strings). This is highly efficient.

13.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\min(n, m))$	The equality comparison `a == b` takes at most $\mathcal{O}(\min(n, m))$ operations (as it exits immediately if lengths differ, and compares characters up to length n otherwise).
Space Complexity	$\mathcal{O}(1)$	No additional memory is allocated.

13.6 Common Follow-Up Questions & How to Answer

13.6.1 Q1: What is the Longest Uncommon Subsequence II? (LeetCode 522)

- **The Problem**: Given an array of strings, return the length of the longest uncommon subsequence among them. An uncommon subsequence is a string that is a subsequence of exactly one of the given strings.
- **The Solution**: We can sort the strings by length in descending order. For each string, we check if it is a subsequence of any other string in the list. The first string (from longest to shortest) that is not a subsequence of any other string is our answer. If we check all and find none, we return `-1`.
- **Subsequence Check function (C++)**:

```cpp
bool isSubsequence(const std::string& s1, const std::string& s2) {
    const int n1 = static_cast<int>(s1.length());
    const int n2 = static_cast<int>(s2.length());
    int i = 0, j = 0;
    while (i < n1 && j < n2) {
        if (s1[i] == s2[j]) i++;
        j++;
    }
    return i == n1;
}
```

13.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Suboptimal DP**: Many candidates immediately jump to Dynamic Programming (similar to Longest Common Subsequence - LCS) which is $\mathcal{O}(n \cdot m)$ time and space. Pointing out the logical simplification *before* writing any code shows you analyze the mathematical properties of a problem instead of blindly applying templates.
- **Verify Length Checks**: Mention that checking string equality in C++ (`std::string::operator==`) first checks the sizes of the strings. If they differ, the check is an $\mathcal{O}(1)$ operation. This shows you understand library implementation details.

14 Linked List Cycle

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 141, Glassdoor
- **Flashcards**: Fast & Slow Pointers deck

14.1 Question Description

Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that the tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter.**

Return `true` if there is a cycle in the linked list. Otherwise, return `false`.

14.1.1 Examples

- **Input**: `head = [3,2,0,-4]`, `pos = 1`
  - **Output**: `true`

  - **Explanation**: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).
- **Input**: `head = [1,2]`, `pos = 0`
  - **Output**: `true`
  - **Explanation**: There is a cycle in the linked list, where the tail connects to the 0th node.
- **Input**: `head = [1]`, `pos = -1`
  - **Output**: `false`
  - **Explanation**: There is no cycle in the linked list.

14.2 Detailed Solution (C++)

The standard approach to detect cycles in a singly-linked list without using extra memory is **Floyd's Cycle-Finding Algorithm** (also known as the "Tortoise and the Hare" algorithm).

We maintain two pointers traversing the list at different speeds: 1. **Slow Pointer (`slow`)**: Moves forward by 1 node at each step. 2. **Fast Pointer (`fast`)**: Moves forward by 2 nodes at each step.

If there is no cycle, the fast pointer will eventually reach the end of the list (`nullptr`). If there is a cycle, the fast pointer will enter the cycle first and eventually catch up to the slow pointer, resulting in `slow == fast`.

14.2.1 Standard C++ Production Code

```cpp
#include <cstddef>

// Definition for singly-linked list node.
struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

class Solution {
public:
    bool hasCycle(ListNode *head) noexcept {
        // Edge Case: An empty list or a list with only one node and no loop cannot have a cycle
        if (!head || !head->next) {
            return false;
        }

        ListNode *slow = head;
        ListNode *fast = head;

        while (fast && fast->next) {
            slow = slow->next;
```

```
            fast = fast->next->next;

            // If the fast pointer catches up to the slow pointer, a cycle exists
            if (slow == fast) {
                return true;
            }
        }

        return false;
    }
};
```

14.3 Detailed Solution (Python)

14.3.1 Standard Python Production Code

Below is the iterative, type-hinted Python implementation using Floyd's Cycle-Finding Algorithm.

```
from typing import Optional

# Definition for singly-linked list.
class ListNode:
    def __init__(self, x: int):
        self.val = x
        self.next = None

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        """
        Determines if a linked list has a cycle using Floyd's Cycle-Finding Algorithm.

        Time Complexity: O(N)
        Space Complexity: O(1)
        """
        # Edge Case: empty list or list with a single node
        if not head or not head.next:
            return False

        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
```

```
        # Identical object comparison using 'is' for speed and safety
        if slow is fast:
            return True

    return False
```

14.4 Python-Specific Complexities & Implementation Considerations

14.4.1 1. Object Identity Comparison (`is` vs `==`)

- In Python, comparing nodes using `==` will invoke the default class representation or custom equivalence operators if defined (checking value equivalence).
- For cycle detection, we want to know if two pointer variables point to the exact same memory address (object identity).
- Always use the `is` operator (`slow is fast`) instead of `==`. The `is` operator compares memory addresses directly (like comparing raw pointers in C++), which is faster and prevents unintended side effects from overridden value-comparison methods.

14.4.2 2. Guarding against AttributeError

- When navigating nested properties (`fast.next.next`), we must guarantee that `fast` and `fast.next` are both non-null before dereferencing.
- Checking `while fast and fast.next:` in Python prevents `AttributeError: 'NoneType' object has no attribute 'next'`.

14.4.3 3. Stack Space & Python Optimization

- While cycles could conceptually be checked recursively, recursive traversal in Python uses $\mathcal{O}(N)$ memory on the execution stack.
- Given Python's default stack recursion depth limit of 1000, a long list would trigger a `RecursionError`. The iterative approach guarantees $\mathcal{O}(1)$ auxiliary space.

14.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	In the worst case (with a cycle), the fast pointer will traverse the cycle at most twice before meeting the slow pointer.
Space Complexity	$\mathcal{O}(1)$	Only two pointers are tracked, using constant auxiliary space.

14.6 Common Follow-Up Questions & How to Answer

14.6.1 Q1: How do you find the exact node where the cycle begins? (LeetCode 142)

- **The Answer**: Once the meeting point is found, reset the `slow` pointer to `head`. Keep the `fast` pointer at the meeting point. Then, move both pointers one step at a time. The node where they meet again is the start of the cycle.
- **Mathematical Proof**: Let the distance from the head to the cycle start node be A. Let the distance from the cycle start to the meeting point be B. Let the length of the cycle be C.
  - Distance traveled by `slow` pointer: $A + B + k_1 \cdot C$
  - Distance traveled by `fast` pointer: $A + B + k_2 \cdot C$
  - Since the fast pointer runs twice as fast:

$$2(A + B + k_1 \cdot C) = A + B + k_2 \cdot C$$

$$A + B = (k_2 - 2k_1) \cdot C = M \cdot C$$

$$A = M \cdot C - B$$

  - This implies that moving a pointer A steps from the head reaches the same node as moving another pointer A steps (which is $M \cdot C - B$ steps) from the meeting point. They will meet exactly at the cycle entrance.

14.6.2 Q2: What if we cannot modify pointers? How would you solve this with a Hash Table?

- **The Answer**: We can store the memory addresses of the visited nodes in a hash set. At each node, we check if it is already present in the set. If it is, a cycle is detected.
- **Trade-off**:
  - **Time Complexity**: $\mathcal{O}(n)$
  - **Space Complexity**: $\mathcal{O}(n)$ to store list nodes.
  - *System constraint*: A hash set requires dynamic memory allocation on the heap for each unique node. In low-level or embedded systems, heap allocations are expensive and can fail due to fragmentation. The two-pointer solution is highly preferred.

14.7 Pro-Tip: How to Impress the Interviewer

- **Mention Cache Locality**: Point out that linked lists inherently have poor cache locality compared to contiguous arrays. When traversing the linked list, dereferencing `next` pointers causes frequent CPU cache misses. In a performance-critical system, data structures like **Arena-allocated linked lists** (where nodes are stored in contiguous memory pages) or flat arrays are preferred to maximize L1/L2 cache hit rates.
- **Identify the Cycle in Array Representation (LeetCode 287)**: Show that Floyd's Cycle-Finding Algorithm is not restricted to linked lists; it can be used to find duplicate values in arrays

where each index points to another index, representing a functional graph.

15 Happy Number

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 202, Glassdoor
- **Flashcards**: Fast & Slow Pointers deck

15.1 Question Description

Write an algorithm to determine if a number `n` is "happy".

A happy number is a number defined by the following process: 1. Starting with any positive integer, replace the number by the sum of the squares of its digits. 2. Repeat the process until the number equals 1 (where it will stay), or it loops endlessly in a cycle which does not include 1. 3. Those numbers for which this process ends in 1 are happy.

Return `true` if `n` is a happy number, and `false` if not.

15.1.1 Examples

- **Input**: `n = 19`
  - **Output**: `true`
  - **Explanation**:
    - $1^2 + 9^2 = 82$
    - $8^2 + 2^2 = 68$
    - $6^2 + 8^2 = 100$
    - $1^2 + 0^2 + 0^2 = 1$
- **Input**: `n = 2`
  - **Output**: `false`

15.2 Detailed Solution (C++)

A naive approach uses a hash set to keep track of visited numbers. However, we can detect cycles with zero auxiliary space using **Floyd's Cycle-Finding Algorithm** (Slow and Fast pointers).

We define a helper function `getNext(n)` to calculate the sum of the squares of the digits of `n`. * We initialize `slow` to `n` and `fast` to `getNext(n)`. * In each step, `slow` transitions to `getNext(slow)`, while `fast` transitions to `getNext(getNext(fast))`. * If `fast` reaches 1, the number is happy. If `slow` and `fast` meet at any value other than 1, we have detected a cycle, meaning the number is not happy.

15.2.1 Standard C++ Production Code

```cpp
#include <cstddef>

class Solution {
private:
    // Helper function to calculate sum of squared digits
    int getNext(int num) const noexcept {
        int total = 0;
        while (num > 0) {
            int digit = num % 10;
            total += digit * digit;
            num /= 10;
        }
        return total;
    }

public:
    bool isHappy(int n) const noexcept {
        int slow = n;
        int fast = getNext(n);

        while (fast != 1 && slow != fast) {
            slow = getNext(slow);
            fast = getNext(getNext(fast));
        }

        return fast == 1;
    }
};
```

15.3 Detailed Solution (Python)

15.3.1 Standard Python Production Code

Below is the type-hinted Python implementation. It uses `divmod` to optimize the digit extraction loop.

```python
class Solution:
    def isHappy(self, n: int) -> bool:
        """
        Determines if a number is happy using Floyd's Cycle-Finding Algorithm.

        Time Complexity: O(log N)
        Space Complexity: O(1)
        """
```

```
    def get_next(num: int) -> int:
        total = 0
        while num > 0:
            # divmod returns (num // 10, num % 10) in a single optimized operation
            num, digit = divmod(num, 10)
            total += digit * digit
        return total

    slow = n
    fast = get_next(n)

    while fast != 1 and slow != fast:
        slow = get_next(slow)
        fast = get_next(get_next(fast))

    return fast == 1
```

15.4 Python-Specific Complexities & Implementation Considerations

15.4.1 1. Optimizing Digit Extraction with `divmod`

- In Python, performing division (`//`) and modulo (`%`) separately results in two bytecode lookups and operations.
- The built-in function `divmod(x, y)` returns both the quotient and remainder as a tuple `(x // y, x % y)`. It is implemented in C and runs faster than performing the operations individually, which is a great trick for python performance tuning.

15.4.2 2. Space Benefits of Pointer Approach

- A hash-set implementation in Python requires dynamic allocation of hash-table slots. In Python, each integer object occupies 28 bytes of memory, and set entry overhead is high.
- By using slow and fast pointers, we only store two integers, avoiding garbage collection overhead and heap allocation pressure.

15.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(\log n)$	The number of digits in n is $\log_{10} n$. The time taken to calculate the next value is proportional to the number of digits. The sequence either reaches 1 or enters a small, bounded cycle extremely quickly.
Space Complexity	$\mathcal{O}(1)$	No extra memory or hash sets are used; only pointers/integers are stored.

15.6 Common Follow-Up Questions & How to Answer

15.6.1 Q1: Why is it guaranteed that the sequence of numbers will not grow infinitely?

- **The Answer**: For any number n, let's analyze how the sum of the squares of its digits behaves.
  - A d-digit number has a value of at least 10^{d-1}.
  - The maximum possible sum of squares of digits for a d-digit number is $d \times 9^2 = 81d$.
  - For $d \geq 4$, $10^{d-1} > 81d$ (for example, if $d = 4$, the minimum value is 1000 and the maximum sum of squares is $4 \times 81 = 324$).
  - This means any number $n \geq 1000$ will strictly shrink at each step until it is less than 1000.
  - For numbers below 1000, the maximum value is 999, which transitions to $3 \times 81 = 243$.
  - Thus, the sequence is always trapped in a finite state space $[1, 243]$. Since the state space is finite, the sequence must eventually repeat a value, either entering a cycle or landing on 1.

15.6.2 Q2: What is the cycle that non-happy numbers fall into?

- **The Answer**: All non-happy numbers eventually cycle through the same loop:

$$4 \to 16 \to 37 \to 58 \to 89 \to 145 \to 42 \to 20 \to 4$$

- Knowing this, we could optimize the solution for systems with extremely tight timing budgets by checking if the sequence ever hits `4` (or any other number in that cycle). If it does, we can return `false` immediately.

15.7 Pro-Tip: How to Impress the Interviewer

- **Explain Mathematical Convergence**: Proactively explain the convergence proof shown in Q1. Showing that you can mathematically prove why an algorithm terminates (and does not stack overflow or run forever) demonstrates computer science maturity.

- **Mention Lookup Tables**: Mention that for a high-throughput system processing millions of numbers, we can precompute the happiness of all integers up to 1000 in a static bitset/lookup array. Any input number can then be reduced to a number < 1000 in a single digit-square step, followed by an $\mathcal{O}(1)$ lookup.

16 Middle of the Linked List

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer
- **Source**: LeetCode 876, Glassdoor
- **Flashcards**: Fast & Slow Pointers deck

16.1 Question Description

Given the `head` of a singly linked list, return *the middle node of the linked list.*

If there are two middle nodes, return **the second middle node**.

16.1.1 Examples

- **Input**: `head = [1,2,3,4,5]`
  - **Output**: `[3,4,5]` (returns the node with value 3 and the remaining list)
  - **Explanation**: The middle node of the list is node 3.
- **Input**: `head = [1,2,3,4,5,6]`
  - **Output**: `[4,5,6]`
  - **Explanation**: Since the list has two middle nodes with values 3 and 4, we return the second one.

16.2 Detailed Solution (C++)

Using a two-pointer approach (often called the **Tortoise and the Hare**), we can find the middle in a single pass: 1. Initialize both `slow` and `fast` pointers at the `head`. 2. Traverse the list: * Move `slow` forward by 1 step: `slow = slow->next`. * Move `fast` forward by 2 steps: `fast = fast->next->next`. 3. When `fast` reaches the end of the list (meaning `fast` or `fast->next` becomes `nullptr`), the `slow` pointer will be exactly at the middle node of the list.

16.2.1 Standard C++ Production Code

```cpp
#include <cstddef>

// Definition for singly-linked list.
struct ListNode {
    int val;
```

```
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class Solution {
public:
    ListNode* middleNode(ListNode* head) noexcept {
        // Edge Case: If the list is empty or has only one element, return head
        if (!head || !head->next) {
            return head;
        }

        ListNode *slow = head;
        ListNode *fast = head;

        // Traverse until fast reach the end of the list
        while (fast && fast->next) {
            slow = slow->next;
            fast = fast->next->next;
        }

        return slow;
    }
};
```

16.3 Detailed Solution (Python)

16.3.1 Standard Python Production Code

Below is the type-hinted, iterative Python implementation.

```
from typing import Optional

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def middleNode(self, head: Optional[ListNode]) -> Optional[ListNode]:
        """
```

```
    Finds the middle of a singly-linked list.
    If there are two middle nodes, returns the second middle node.

    Time Complexity: O(N)
    Space Complexity: O(1)
    """
    slow = head
    fast = head

    # Loop terminates when fast reaches the end
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow
```

16.4 Python-Specific Complexities & Implementation Considerations

16.4.1 1. Pointer Dereferencing Safety

- Just like in C++, checking `while fast and fast.next:` in Python is crucial to avoid a runtime `AttributeError` when accessing `fast.next.next`.
- If `fast` is `None` or `fast.next` is `None`, the loop ends safely.

16.4.2 2. Slicing/Iterative Alternative Risks

- A common anti-pattern in Python is converting the linked list into a Python `list` (array) to find the middle via index arithmetic (`arr[len(arr) // 2]`).
- While this achieves $\mathcal{O}(n)$ time complexity, it requires $\mathcal{O}(n)$ auxiliary space to store the node references. Under tight system constraints or memory-bound tasks, the two-pointer solution is highly preferred as it maintains $\mathcal{O}(1)$ space.

16.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	The fast pointer reaches the end in $n/2$ steps, making a single traversal of the list.
Space Complexity	$\mathcal{O}(1)$	Only two pointer variables are allocated.

16.6 Common Follow-Up Questions & How to Answer

16.6.1 Q1: What if we want the FIRST middle node in an even-length list?

- **The Answer**: For a list of length 4 (e.g., `[1, 2, 3, 4]`), the default code returns `3` (the second middle node). If we want to return `2` (the first middle node), we can modify the loop condition to stop one step earlier.
- **Modification**:

```
ListNode* slow = head;
ListNode* fast = head;
// Look ahead 2 nodes: stops fast before it reaches the final node
while (fast && fast->next && fast->next->next) {
    slow = slow->next;
    fast = fast->next->next;
}
return slow;
```

  - *Dry Run on `[1, 2, 3, 4]`*:
    - Start: `slow = 1`, `fast = 1`.
    - `fast->next` is 2, `fast->next->next` is 3 (not null). Advance: `slow = 2`, `fast = 3`.
    - Next check: `fast = 3`, `fast->next` is 4, but `fast->next->next` is `nullptr`. Loop terminates.
    - Returns `slow` (which is `2`).

16.6.2 Q2: How would you delete the middle node of the list? (LeetCode 2095)

- **The Answer**: To delete a node from a singly linked list, we must know its predecessor.
  - Maintain a `prev` pointer initialized to `nullptr`.
  - In the loop, assign `prev = slow` right before updating `slow`.
  - After the loop, delete the middle node by linking `prev->next = slow->next`, and then freeing the memory of `slow` (in C++).

16.6.3 Q3: Why is this method critical for sorting a linked list? (LeetCode 148)

- **The Answer**: This two-pointer split method is the foundation of **Merge Sort** on a linked list. In Merge Sort, we divide the list into two halves, recursively sort them, and merge them. Finding the middle node allows us to disconnect the two halves (`slow->next = nullptr`) in $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ space.

16.7 Pro-Tip: How to Impress the Interviewer

- **Highlight Cache-Friendliness**: Discuss how traversing a linked list causes CPU cache misses because the nodes are scattered across heap memory. Mention that if this list was stored in a contiguous block (like in an custom-allocated pool/arena allocator), the traversal would benefit from CPU prefetching.

- **Address Multi-threaded Access Safety**: Point out that if the linked list is mutable and shared across multiple threads, read-only traversals like `middleNode` require synchronization or must operate on a local snapshot to avoid race conditions (e.g., another thread deleting a node currently being accessed by `fast->next`).

17 Longest Substring Without Repeating Characters

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 3, Glassdoor
- **Flashcards**: Sliding Window deck

17.1 Question Description

Given a string `s`, find the length of the **longest substring** without duplicate characters.

A **substring** is a contiguous, non-empty sequence of characters within a string.

17.1.1 Examples

- **Input**: `s = "abcabcbb"`
  - **Output**: `3`
  - **Explanation**: The answer is `"abc"`, with the length of 3.
- **Input**: `s = "bbbbb"`
  - **Output**: `1`
  - **Explanation**: The answer is `"b"`, with the length of 1.
- **Input**: `s = "pwwkew"`
  - **Output**: `3`
  - **Explanation**: The answer is `"wke"`, with the length of 3. Note that the answer must be a substring, `"pwke"` is a subsequence and not a substring.

17.2 Detailed Solution (C++)

The core algorithm uses a **Sliding Window** optimized with a direct lookup structure. Rather than maintaining a hash set of characters in the current window and shrinking the left pointer incrementally (which can lead to multiple operations per character), we store the last seen 0-based index of each character.

As we iterate with the right pointer `i`: 1. If the character `c = s[i]` was already seen and its last recorded index is within the current window (i.e. $\geq$ left), we jump the `left` boundary of our window directly to `last_index + 1`. This safely excludes the duplicate. 2. We update the last seen index of `c` to `i`. 3. The size of the current window is `i - left + 1`. We update the maximum length if the current size is larger.

Using a fixed-size array or `std::vector<int>` instead of `std::unordered_map` prevents hash collisions and dynamic memory allocation overhead, yielding optimal cache locality and performance.

17.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <vector>
#include <algorithm>

class Solution {
public:
    int lengthOfLongestSubstring(const std::string& s) noexcept {
        // Optimize lookup by using a fixed-size vector for all 256 possible byte values
        // Initializing with -1 to indicate that no character has been seen yet
        std::vector<int> last_seen(256, -1);

        int left = 0;
        int max_len = 0;
        const int n = static_cast<int>(s.length());

        for (int i = 0; i < n; ++i) {
            // Get ASCII value of the character safely
            unsigned char c = s[i];

            // If the character is inside the current window, move the left pointer past it
            if (last_seen[c] >= left) {
                left = last_seen[c] + 1;
            }

            // Record/update the index of the current character
            last_seen[c] = i;

            // Update the maximum substring length found so far
            max_len = std::max(max_len, i - left + 1);
        }

        return max_len;
    }
};
```

17.3 Detailed Solution (Python)

17.3.1 Standard Python Production Code

Below is the optimized sliding window Python implementation. It uses a dictionary to store the last seen index of each character, allowing constant-time jumps for the left pointer.

```python
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        """
        Finds the length of the longest substring without repeating characters.

        Time Complexity: O(N)
        Space Complexity: O(min(N, M)) where M is the character set size.
        """
        char_index: dict[str, int] = {}
        left = 0
        max_len = 0

        for right, ch in enumerate(s):
            # If the character is duplicate and within the current active window
            if ch in char_index and char_index[ch] >= left:
                left = char_index[ch] + 1

            # Update the character's last seen position
            char_index[ch] = right

            # Calculate window size and update max_len
            max_len = max(max_len, right - left + 1)

        return max_len
```

17.4 Python-Specific Complexities & Implementation Considerations

17.4.1 1. Dictionary Lookup and Memory Overhead

- Python's built-in `dict` uses a hash table implementation that has a relatively high memory overhead per key-value pair compared to a primitive array in C++.
- However, since the character set is bounded (e.g., standard ASCII has 128 characters, unicode character space is also bounded in typical tests), the dictionary size will not grow indefinitely.

17.4.2 2. Space Optimization using Lists

- If we know the string contains only standard ASCII characters, we can replace the dictionary with a list of size 128 initialized to `-1` (e.g. `char_index = [-1] * 128` and using `ord(ch)` to index).

- In Python, direct list indexing (`char_index[ord(ch)]`) is often slightly faster than dictionary lookups due to the lack of hashing operations, though Python's dynamic string hashing is heavily optimized in CPython.

17.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We scan the string of length n exactly once using the right pointer. Each character update is $\mathcal{O}(1)$.
Space Complexity	$\mathcal{O}(\min(n, m))$	Auxiliary space is bounded by the size of the alphabet m or the length of the string n. For ASCII, $m \leq 128$, making it $\mathcal{O}(1)$ auxiliary space.

17.6 Common Follow-Up Questions & How to Answer

17.6.1 Q1: What if the character set is extended to Unicode?

- **Answer**: The C++ implementation using `std::vector<int> last_seen(256, -1)` will handle all 256 single-byte (Extended ASCII/UTF-8 byte) values, but will not correctly track multi-byte UTF-8 sequences as individual characters or handle wider Unicode code points.
- **Modification**: In C++, we should use a `std::unordered_map<wchar_t, int>` or `std::unordered_map<char32_t, int>` to handle Unicode characters dynamically, or decode the UTF-8 input string to UTF-32 first. In Python, strings are natively Unicode, so the `dict` implementation works seamlessly without changes.

17.6.2 Q2: Can you return the actual longest substring instead of just the length?

- **Answer**: Yes, we can track the starting index and length of the longest substring.
- **Code Addition (C++)**:

```cpp
int max_len = 0, start_idx = 0;
// Inside the loop:
if (i - left + 1 > max_len) {
    max_len = i - left + 1;
    start_idx = left;
}
// Return:
return s.substr(start_idx, max_len);
```

17.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Suboptimal Set Shrinking**: Many candidates write a sliding window using a `std::unordered_set` or `std::set`. When a duplicate is encountered, they increment `left` one-by-one and erase characters from the set until the duplicate is removed. Explain to the interviewer that while this is technically $\mathcal{O}(n)$, it does unnecessary operations. Storing the indices lets us perform an **instant** $\mathcal{O}(1)$ **jump**.
- **C++ Vector vs Unordered Map**: Highlight that `std::unordered_map` is implemented as a closed-addressing hash table (linked lists on bucket collisions). This causes heap allocations and bad cache-locality. Using a contiguous `std::vector<int>` (or a fixed-size stack array `int last_seen[256]`) eliminates allocation overhead and guarantees $\mathcal{O}(1)$ lookup times.
- **Unicode/UTF-8 Awareness**: Demonstrate real-world software engineering depth by asking the interviewer about the encoding of `s`. If the string is UTF-8 encoded, characters can occupy multiple bytes. Treating a single byte as a character is incorrect. Showing you are aware of localization and encoding is a massive green flag.

18 Longest Substring with At Least K Repeating Characters

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 395, Glassdoor
- **Flashcards**: Sliding Window deck

18.1 Question Description

Given a string `s` and an integer `k`, return the length of the longest substring of `s` such that the frequency of each character in this substring is greater than or equal to `k`. If no such substring exists, return `0`.

18.1.1 Examples

- **Input**: `s = "aaabb"`, `k = 3`
  - **Output**: `3`
  - **Explanation**: The longest substring is `"aaa"`, as `'a'` is repeated 3 times.
- **Input**: `s = "ababbc"`, `k = 2`
  - **Output**: `5`
  - **Explanation**: The longest substring is `"ababb"`, as `'a'` is repeated 2 times and `'b'` is repeated 3 times.

18.2 Detailed Solution (C++)

There are two primary paradigms to solve this problem: **Sliding Window** and **Divide and Conquer**.

18.2.1 1. Sliding Window (Iterative Fixed-Alphabet Approach)

A traditional sliding window doesn't work directly because the validity condition is non-monotonic (expanding the window might make a invalid substring valid, and shrinking it could also make a valid substring invalid). To make it monotonic, we constrain the number of unique characters allowed in the sliding window. Since the string consists only of lowercase English letters, the number of unique characters T can range from 1 to 26. For each target $T \in [1, 26]$: * We slide a window `[left, right]`. * We count the number of `unique` characters in the window, and the number of characters that have a frequency of at least `k` (`at_least_k`). * If `unique > T`, we shrink the window from the left until `unique <= T`. * If `unique == T` and `at_least_k == T`, the window is valid, and we update the maximum length.

18.2.2 2. Divide and Conquer

If a character `c` in `s` has a total frequency in the current substring that is less than `k`, then `c` cannot be part of any valid substring. Therefore, any valid substring must lie either completely to the left of the first occurrence of `c`, or completely to the right. We can split the string using `c` as a delimiter and recursively solve for the resulting segments, returning the maximum length obtained.

18.2.3 Standard C++ Production Code

```cpp
#include <string>
#include <vector>
#include <algorithm>

class Solution {
public:
    // Sliding Window Approach: O(26 * N) time, O(1) space
    int longestSubstring(const std::string& s, int k) noexcept {
        int max_len = 0;
        const int n = static_cast<int>(s.length());

        // Iterate through all possible number of unique characters in the window
        for (int t = 1; t <= 26; ++t) {
            int freq[26] = {0};
            int left = 0;
            int unique = 0;
            int at_least_k = 0;

            for (int right = 0; right < n; ++right) {
                int idx = s[right] - 'a';
                if (freq[idx] == 0) {
                    unique++;
```

```
                }
                freq[idx]++;
                if (freq[idx] == k) {
                    at_least_k++;
                }

                // Shrink the window if the unique character count exceeds t
                while (unique > t) {
                    int lidx = s[left] - 'a';
                    if (freq[lidx] == k) {
                        at_least_k--;
                    }
                    freq[lidx]--;
                    if (freq[lidx] == 0) {
                        unique--;
                    }
                    left++;
                }

                // If current state matches the target uniqueness and frequency criteria
                if (unique == t && at_least_k == t) {
                    max_len = std::max(max_len, right - left + 1);
                }
            }
        }
        return max_len;
    }

    // Divide and Conquer Approach: O(N^2) worst case, O(N log N) average, O(N) space
    int longestSubstringDC(const std::string& s, int k) {
        return longestSubstringHelperDC(s, k, 0, static_cast<int>(s.length()));
    }

private:
    int longestSubstringHelperDC(const std::string& s, int k, int start, int end) {
        if (end - start < k) {
            return 0;
        }

        int freq[26] = {0};
        for (int i = start; i < end; ++i) {
            freq[s[i] - 'a']++;
        }
```

```
        // Find the first character that occurs more than 0 but fewer than k times
        int bad_char = -1;
        for (int i = 0; i < 26; ++i) {
            if (freq[i] > 0 && freq[i] < k) {
                bad_char = 'a' + i;
                break;
            }
        }

        // If no bad character exists, the whole substring is valid
        if (bad_char == -1) {
            return end - start;
        }

        int max_len = 0;
        int seg_start = start;

        for (int i = start; i <= end; ++i) {
            if (i == end || s[i] == bad_char) {
                max_len = std::max(max_len, longestSubstringHelperDC(s, k, seg_start, i));
                seg_start = i + 1;
            }
        }

        return max_len;
    }
};
```

18.3 Detailed Solution (Python)

18.3.1 Standard Python Production Code

Below are the Python solutions representing both the sliding window approach and the divide-and-conquer strategy.

```
from collections import Counter

class Solution:
    def longestSubstring(self, s: str, k: int) -> int:
        """
        Sliding window implementation with a fixed target of unique characters.

        Time Complexity: O(26 * N) = O(N)
        Space Complexity: O(1) as we use fixed-size tables
```

```
    """
    max_len = 0
    n = len(s)

    for t in range(1, 27):
        freq: dict[str, int] = {}
        left = 0
        unique = 0
        at_least_k = 0

        for right in range(n):
            ch = s[right]
            freq[ch] = freq.get(ch, 0) + 1
            if freq[ch] == 1:
                unique += 1
            if freq[ch] == k:
                at_least_k += 1

            # Shrink left pointer if we have too many unique characters
            while unique > t:
                lc = s[left]
                if freq[lc] == k:
                    at_least_k -= 1
                freq[lc] -= 1
                if freq[lc] == 0:
                    unique -= 1
                left += 1

            # Check validity
            if unique == t and at_least_k == t:
                max_len = max(max_len, right - left + 1)

    return max_len

def longestSubstringDC(self, s: str, k: int) -> int:
    """
    Divide and Conquer implementation.

    Time Complexity: O(N log N) average, O(N^2) worst case
    Space Complexity: O(N) due to recursion stack and split strings
    """
    if len(s) < k:
        return 0
```

```
        freq = Counter(s)
        for char, count in freq.items():
            if count < k:
                # Split s on the bad character and recursively solve for all parts
                return max(self.longestSubstringDC(sub, k) for sub in s.split(char))

        return len(s)
```

18.4 Python-Specific Complexities & Implementation Considerations

18.4.1 1. Recursion Overhead and `split()` in Python

- The divide-and-conquer code in Python is highly elegant because of `s.split(char)`. However, `split()` creates new strings on the heap, which leads to $\mathcal{O}(n)$ dynamic allocation per split.
- For deep recursion, this creates high GC (Garbage Collection) pressure. In memory-constrained or high-throughput systems, the iterative sliding window or a custom index-based split is preferred.

18.4.2 2. Time-Complexity Overhead of Python's Dictionary

- In the sliding window approach, using `dict` instead of a list of size 26 creates minor runtime overhead due to dictionary hashing. Replacing the lookup table with `freq = [0] * 26` and mapping characters via `ord(ch) - 97` can yield a significant constant-factor speedup in Python.

18.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity (Sliding Window)	$\mathcal{O}(n)$	We repeat a standard sliding window 26 times (once for each possible count of unique characters). Each pass takes $\mathcal{O}(n)$ time.
Space Complexity (Sliding Window)	$\mathcal{O}(1)$	We only store frequency counts for the English alphabet (size 26), which requires constant space.
Time Complexity (Divide & Conquer)	$\mathcal{O}(n \log n)$ average	On average, splits happen near the center, leading to a recurrence of $T(n) = 2T(n/2) + \mathcal{O}(n)$. In the worst-case, it can degrade to $\mathcal{O}(n^2)$ if splits occur at the ends.

Metric	Complexity	Description
Space Complexity (Divide & Conquer)	$\mathcal{O}(n)$	Due to recursion stack depth and substring storage during splitting.

18.6 Common Follow-Up Questions & How to Answer

18.6.1 Q1: What is the optimal approach when k is very large?

- **Answer**: If k is greater than the length of s, we can return `0` immediately. If k is large, the Divide and Conquer approach tends to find "bad" characters very quickly near the root of the recursion, causing it to prune early. Thus, for large k, Divide and Conquer typically runs faster than the sliding window.

18.6.2 Q2: How can we optimize the Divide and Conquer code to use $\mathcal{O}(1)$ auxiliary space?

- **Answer**: Instead of copying substrings using slicing or `.split()`, we can pass `start` and `end` index boundaries to the recursive helper function (similar to our C++ implementation). This avoids creating copies of string slices in memory, reducing space complexity from $\mathcal{O}(n)$ to $\mathcal{O}(d)$ where d is the recursion depth (worst-case $\mathcal{O}(n)$, average $\mathcal{O}(\log n)$ stack frames).

18.7 Pro-Tip: How to Impress the Interviewer

- **Explain Why Simple Sliding Window Fails**: Start your solution by explaining *why* a naive sliding window doesn't work. The fact that the validity condition is not monotonic is a key observation that most candidates miss. Pointing this out shows you understand the theoretical limits of greedy sliding windows.
- **Pruning in Divide and Conquer**: Mention that you can optimize the divide-and-conquer approach by returning early if the length of the string segment is less than `k`, or if the segment does not contain enough characters to satisfy the condition.
- **Alphabet Size Invariance**: Discuss how the sliding window complexity $\mathcal{O}(\Sigma \cdot n)$ (where Σ is the alphabet size) scales. If the alphabet is generic Unicode rather than lowercase English characters, $\Sigma \approx 2^{16}$ or 2^{21}, making the sliding window approach impractical compared to Divide and Conquer. This shows deep system-design awareness.

19 Longest Repeating Character Replacement

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 424, Glassdoor
- **Flashcards**: Sliding Window deck

19.1 Question Description

You are given a string `s` and an integer `k`. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most `k` times.

Return the length of the longest substring containing the same letter you can get after performing the above operations.

19.1.1 Examples

- **Input**: `s = "ABAB"`, `k = 2`
  - **Output**: `4`
  - **Explanation**: Replace the two 'A's with two 'B's or vice versa.
- **Input**: `s = "AABABBA"`, `k = 1`
  - **Output**: `4`
  - **Explanation**: Replace the one 'A' in the middle with 'B' and form `"AABBBBA"`. The substring `"BBBB"` has the longest repeating letters, which is 4.

19.2 Detailed Solution (C++)

This problem is solved using the **Sliding Window** technique. We maintain a window `[left, right]` and a frequency table of characters inside the window.

The key observation is: * The total number of characters in the window is `right - left + 1`. * Let `max_freq` be the frequency of the most frequent character in the current window. * To make all characters in the window the same, we must replace the remaining `(right - left + 1) - max_freq` characters. * If this count is $\leq k$, the window is valid, and we can potentially update our maximum length. * If this count $> k$, the window is invalid, so we increment `left` and decrement the frequency of the character leaving the window.

19.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <vector>
#include <algorithm>

class Solution {
public:
    int characterReplacement(const std::string& s, int k) noexcept {
        int freq[26] = {0};
        int max_freq = 0;
        int left = 0;
        int max_len = 0;
        const int n = static_cast<int>(s.length());
```

```
        for (int right = 0; right < n; ++right) {
            int idx = s[right] - 'A';
            freq[idx]++;

            // Track the frequency of the most common character in the current window
            max_freq = std::max(max_freq, freq[idx]);

            // If the number of characters to replace is greater than k, shrink the window
            while ((right - left + 1) - max_freq > k) {
                freq[s[left] - 'A']--;
                left++;
            }

            // Update the maximum length of a valid window
            max_len = std::max(max_len, right - left + 1);
        }

        return max_len;
    }
};
```

19.3 Detailed Solution (Python)

19.3.1 Standard Python Production Code

Below is the optimized sliding window implementation in Python using standard collections and dictionary lookups.

```
class Solution:
    def characterReplacement(self, s: str, k: int) -> int:
        """
        Finds the length of the longest substring containing the same letter
        after replacing at most k characters.

        Time Complexity: O(N)
        Space Complexity: O(1) as the frequency dictionary holds at most 26 keys.
        """
        freq: dict[str, int] = {}
        max_freq = 0
        left = 0
        max_len = 0

        for right, ch in enumerate(s):
            freq[ch] = freq.get(ch, 0) + 1
```

```
        max_freq = max(max_freq, freq[ch])

        # If replacement budget is exceeded, shrink the window from left
        while (right - left + 1) - max_freq > k:
            freq[s[left]] -= 1
            left += 1

        max_len = max(max_len, right - left + 1)

    return max_len
```

19.4 Python-Specific Complexities & Implementation Considerations

19.4.1 1. Optimization of `max_freq` Recomputation

A common point of confusion is whether `max_freq` needs to be decremented when a character leaves the window (i.e. when `left` is incremented). * **No recomputation is required**: We are only looking for a window strictly *larger* than the current `max_len`. A larger window is only possible if we find a new character with a frequency higher than our historical `max_freq`. * Therefore, `max_freq` only needs to track the historical maximum frequency inside any window we've seen. Not needing to scan the frequency dictionary for the new max value on every shrink keeps our inner loop extremely fast.

19.4.2 2. Contiguous Allocations vs Hash Map

- In C++, `int freq[26]` is stack-allocated, which is extremely fast.
- In Python, using `ord(ch) - 65` with a list `freq = [0] * 26` can run up to 2× faster than a dictionary because it bypasses hashing, hash collision resolution, and dictionary resizing.

19.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We scan the string of length n with a right pointer. The left pointer only moves forward. Each step does $\mathcal{O}(1)$ frequency map operations.
Space Complexity	$\mathcal{O}(1)$	The character set is restricted to 26 uppercase English letters, requiring constant auxiliary space.

19.6 Common Follow-Up Questions & How to Answer

19.6.1 Q1: Can this be solved with a non-shrinking window?

- **Answer**: Yes. Instead of shrinking the window with a `while` loop, we can keep the window size constant or growing. If the window becomes invalid, we simply shift the whole window (both `left` and `right` advance by 1).
- **C++ Code (Non-shrinking)**:

```
int freq[26] = {0};
int max_freq = 0, left = 0;
for (int right = 0; right < s.length(); ++right) {
    max_freq = std::max(max_freq, ++freq[s[right] - 'A']);
    if ((right - left + 1) - max_freq > k) {
        freq[s[left] - 'A']--;
        left++; // Shift left by 1 without shrinking
    }
}
return s.length() - left;
```

- **Benefit**: This eliminates the inner loop entirely, leading to cleaner branch prediction.

19.6.2 Q2: What if the string can contain any ASCII character?

- **Answer**: The logic remains exactly the same, but the frequency array size must be expanded from 26 to 128 (or 256 for extended ASCII) to prevent out-of-bounds indexing.

19.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Historical Max Frequency Intuition**: Proactively explain why `max_freq` does not need to be recomputed when the window shrinks. The fact that `max_freq` can represent a *historical* maximum rather than the *current* maximum is a brilliant mathematical optimization. Mentioning this demonstrates outstanding analytical depth.
- **Branch Prediction Efficiency**: Explain how the non-shrinking window approach is more friendly to the CPU pipeline. Because the window only shifts or grows (the branching is highly predictable), there are fewer CPU pipeline stalls compared to the shrinking window approach where the inner `while` loop condition can be highly dynamic.

20 Find All Anagrams in a String

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 438, Glassdoor
- **Flashcards**: Sliding Window deck

20.1 Question Description

Given two strings `s` and `p`, return an array of all the start indices of `p`'s anagrams in `s`. You may return the answer in any order.

An **Anagram** is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

20.1.1 Examples

- **Input**: `s = "cbaebabacd"`, `p = "abc"`
  - **Output**: `[0, 6]`
  - **Explanation**:
    - The substring with start index = `0` is `"cba"`, which is an anagram of `"abc"`.
    - The substring with start index = `6` is `"bac"`, which is an anagram of `"abc"`.
- **Input**: `s = "abab"`, `p = "ab"`
  - **Output**: `[0, 1, 2]`
  - **Explanation**:
    - The substring with start index = `0` is `"ab"`, which is an anagram of `"ab"`.
    - The substring with start index = `1` is `"ba"`, which is an anagram of `"ab"`.
    - The substring with start index = `2` is `"ab"`, which is an anagram of `"ab"`.

20.2 Detailed Solution (C++)

The core approach is a **Sliding Window of Fixed Size** matching the length of `p`. We maintain the character frequency counts of `p` and the sliding window of `s`. 1. Initialize frequency count arrays (or `std::array<int, 26>`) for `p` and the first `plen` characters of `s`. 2. Compare the two tables. If they are identical, index `0` is a start position of an anagram. 3. Slide the window from left to right: * Add the new character at the right of the window. * Remove the character at the left of the window. * Compare the tables. If they are identical, add the new start index to the result.

20.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <vector>
#include <array>

class Solution {
public:
    std::vector<int> findAnagrams(const std::string& s, const std::string& p) noexcept {
        std::vector<int> result;
        const int slen = static_cast<int>(s.size());
        const int plen = static_cast<int>(p.size());
```

```
        // Edge Case: If pattern is longer than the string, no anagram can exist
        if (plen > slen) {
            return result;
        }

        std::array<int, 26> p_freq{};
        std::array<int, 26> s_freq{};

        // Initialize the count tables for the first window
        for (int i = 0; i < plen; ++i) {
            p_freq[p[i] - 'a']++;
            s_freq[s[i] - 'a']++;
        }

        // Compare the first window
        if (p_freq == s_freq) {
            result.push_back(0);
        }

        // Slide the window across the rest of the string
        for (int i = plen; i < slen; ++i) {
            // Include the new character entering the window
            s_freq[s[i] - 'a']++;
            // Exclude the character leaving the window
            s_freq[s[i - plen] - 'a']--;

            // Fast array comparison (takes O(26) operations)
            if (p_freq == s_freq) {
                result.push_back(i - plen + 1);
            }
        }

        return result;
    }
};
```

20.3 Detailed Solution (Python)

20.3.1 Standard Python Production Code

Below is the Python sliding window implementation. It uses a character frequency map and manually slides the window to avoid recreating the frequency maps at each step.

```python
from collections import Counter
from typing import List

class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        """
        Finds all start indices of p's anagrams in s.

        Time Complexity: O(N) where N is the length of s.
        Space Complexity: O(1) as the frequency counters have at most 26 keys.
        """
        slen, plen = len(s), len(p)
        if plen > slen:
            return []

        p_count = Counter(p)
        s_count = Counter(s[:plen])
        result = []

        if s_count == p_count:
            result.append(0)

        for i in range(plen, slen):
            # Slide the window right: add current char, remove oldest char
            entering_char = s[i]
            leaving_char = s[i - plen]

            s_count[entering_char] = s_count.get(entering_char, 0) + 1
            s_count[leaving_char] -= 1

            # Clean up the key from dictionary if count drops to 0 to make dict comparisons accurate
            if s_count[leaving_char] == 0:
                del s_count[leaving_char]

            if s_count == p_count:
                result.append(i - plen + 1)

        return result
```

20.4 Python-Specific Complexities & Implementation Considerations

20.4.1 1. Dictionary Comparison Overhead

- In Python, comparing two `Counter` dictionaries (e.g. `s_count == p_count`) checks if both have identical keys and values.
- To ensure accuracy, it is critical to **delete** keys when their count reaches zero (`del s_count[leaving_char]`). Otherwise, a key with count `0` in `s_count` will cause `s_count == p_count` to return `False` even if all other frequencies match.

20.4.2 2. Alternative: Using `list` with size 26

- To optimize Python performance, you can use two lists of size 26 (`s_count = [0] * 26`) and compare them: `s_count == p_count`.
- Comparing lists of size 26 in Python is highly optimized in C and avoids the overhead of dictionary key deletions.

20.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We iterate through `s` once. In each iteration, comparing two frequency maps of fixed size 26 takes $\mathcal{O}(26) = \mathcal{O}(1)$ time.
Space Complexity	$\mathcal{O}(1)$	The frequency counters are bounded by the size of the English alphabet (26), which requires constant space.

20.6 Common Follow-Up Questions & How to Answer

20.6.1 Q1: Can we achieve true $\mathcal{O}(1)$ comparison time without comparing the full array/dict at each step?

- **Answer**: Yes, by maintaining a `matches` count representing the number of unique characters that have the exact same frequency in both the window and the pattern `p`.
- **Explanation**: When sliding the window, only two characters (entering and leaving) have their frequencies modified. We can check if their modified frequencies now match or no longer match the pattern's counts, updating the `matches` tracker accordingly. When `matches == 26`, we found an anagram.
- **C++ Code Snippet**:

```
int matches = 0;
for (int i = 0; i < 26; ++i) {
    if (s_freq[i] == p_freq[i]) matches++;
}
// During sliding window:
// Update matches when s_freq[idx] changes to match/mismatch p_freq[idx]
```

20.6.2 Q2: How does this generalize to find matches of any permutation of a pattern including wildcards?

- **Answer**: If a wildcard character (e.g., `?`) matches any character, we can adjust our comparison logic to treat wildcards as a credit. The size of the window remains fixed at `plen`, and we match the frequency table allowing up to the wildcard count of discrepancies.

20.7 Pro-Tip: How to Impress the Interviewer

- **Mention `del` for dict comparison**: In Python, point out that checking equality of two counters requires clearing keys with zero values. Explaining this details that you understand how Python's dictionary equality operator (`__eq__`) works under the hood (checking presence of all keys).
- **Highlight Cache Locality**: Using `std::array<int, 26>` instead of `std::unordered_map` ensures that the frequency counters are stored contiguously on the stack. The comparison `sf == pf` compiles to highly efficient vector instructions (SIMD) on modern compilers, making it faster than custom checks in practice.

21 Permutation in String

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 567, Glassdoor
- **Flashcards**: Sliding Window deck

21.1 Question Description

Given two strings `s1` and `s2`, return `true` if `s2` contains a permutation of `s1`, or `false` otherwise.

In other words, return `true` if one of `s1`'s permutations is the substring of `s2`.

21.1.1 Examples

- **Input**: `s1 = "ab"`, `s2 = "eidbaooo"`
  - **Output**: `true`
  - **Explanation**: `s2` contains one permutation of `s1` (`"ba"`).
- **Input**: `s1 = "ab"`, `s2 = "eidboaoo"`

– **Output**: `false`

21.2 Detailed Solution (C++)

This problem is isomorphic to **Find All Anagrams in a String**, except we return a boolean value immediately upon finding the first match instead of accumulating indices.

We use a **Sliding Window of Fixed Size** matching the length of `s1`. 1. We populate a frequency map `target` of size 26 for string `s1`. 2. As we iterate through `s2` using a sliding window pointer `right`, we add `s2[right]` to our `current` frequency map. 3. If the window length exceeds `s1.length()`, we remove the leftmost character `s2[right - s1.length()]` from `current`. 4. If `target` is equal to `current`, we return `true` immediately. 5. If the loop finishes without a match, we return `false`.

21.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <array>

class Solution {
public:
    bool checkInclusion(const std::string& s1, const std::string& s2) noexcept {
        const int n1 = static_cast<int>(s1.size());
        const int n2 = static_cast<int>(s2.size());

        // Edge Case: If s1 is longer than s2, s2 cannot contain a permutation of s1
        if (n1 > n2) {
            return false;
        }

        std::array<int, 26> target{};
        std::array<int, 26> current{};

        // Populate the frequency table for s1
        for (int i = 0; i < n1; ++i) {
            target[s1[i] - 'a']++;
        }

        // Slide the window across s2
        for (int right = 0; right < n2; ++right) {
            current[s2[right] - 'a']++;

            // Shrink/slide window by removing the element that is out of bounds
            if (right >= n1) {
                current[s2[right - n1] - 'a']--;
```

```
            }

            // Direct O(26) comparison of arrays
            if (target == current) {
                return true;
            }
        }

        return false;
    }
};
```

21.3 Detailed Solution (Python)

21.3.1 Standard Python Production Code

Below is the Python implementation. We use a simple integer list of size 26 for frequencies to avoid the overhead of dictionary operations, matching the logic of the C++ implementation.

```python
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        """
        Determines if s2 contains a permutation of s1.

        Time Complexity: O(N) where N is the length of s2.
        Space Complexity: O(1) as the list uses a fixed size of 26.
        """
        n1, n2 = len(s1), len(s2)
        if n1 > n2:
            return False

        target = [0] * 26
        current = [0] * 26

        # Populate target frequency table for s1
        for ch in s1:
            target[ord(ch) - 97] += 1  # 97 is ord('a')

        # Slide the window across s2
        for right in range(n2):
            current[ord(s2[right]) - 97] += 1

            # Remove character that fell outside of window
            if right >= n1:
```

```
            current[ord(s2[right - n1]) - 97] -= 1

        # Compare the frequency lists
        if current == target:
            return True

    return False
```

21.4 Python-Specific Complexities & Implementation Considerations

21.4.1 1. Choice of List vs Dictionary

- In Python, comparing lists (`list1 == list2`) checks elements by index in C and is extremely fast.
- Using dictionaries (`Counter`) would require deleting elements that reach `0` to keep the key counts aligned. List comparison avoids this logic and is much cleaner and faster in practice.

21.4.2 2. Micro-optimization: Avoiding `ord()` in Inner Loop

- In Python, calling `ord()` inside the loop introduces function call overhead.
- We can convert both strings into lists of integers using `[ord(c) - 97 for c in string]` beforehand. This pre-processing moves the function call overhead outside of the main loop, speeding up execution for very long strings.

21.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n_2)$	We scan string `s2` of length n_2 once. At each step, comparing two lists of size 26 is $\mathcal{O}(26) = \mathcal{O}(1)$.
Space Complexity	$\mathcal{O}(1)$	The alphabet is lowercase English letters, so the memory footprint is bounded at 26 integers.

21.6 Common Follow-Up Questions & How to Answer

21.6.1 Q1: Can we optimize the array comparison from $\mathcal{O}(26)$ to true $\mathcal{O}(1)$?

- **Answer**: Yes, by maintaining a `matches` counter initialized to the number of matches between the first window and target.

- **Explanation**: We count how many characters have the exact same frequency in both `target` and `current`. As the window slides, we only modify the frequencies of two characters (entering and leaving). We adjust the `matches` count accordingly:
  - If the new frequency of the character matches `target`, we increment `matches`.
  - If it used to match but now does not, we decrement `matches`.
  - If `matches == 26`, we return `true`.

21.6.2 Q2: What if we want to find the start index of the first occurrence of the permutation?

- **Answer**: We simply return `right - n1 + 1` instead of returning `true`. If the loop completes without a match, we return `-1`.

21.7 Pro-Tip: How to Impress the Interviewer

- **Mention SIMD and Array Layout**: Explain that in C++, `std::array<int, 26>` is stored as a contiguous stack block. Compilers are highly capable of auto-vectorizing array comparisons (using AVX/SIMD instructions) so that checking `target == current` executes in only a couple of CPU clock cycles.
- **Space-Time Tradeoff awareness**: Mention that pre-indexing the string into a numeric array (such as converting ASCII to integers `0-25`) reduces the runtime overhead of string parsing and character arithmetic inside the sliding window loop.

22 Merge Intervals

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 56, Glassdoor
- **Flashcards**: Merge Intervals deck

22.1 Question Description

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

22.1.1 Examples

- **Input**: `intervals = [[1,3],[2,6],[8,10],[15,18]]`
  - **Output**: `[[1,6],[8,10],[15,18]]`
  - **Explanation**: Since intervals `[1,3]` and `[2,6]` overlap, merge them into `[1,6]`.
- **Input**: `intervals = [[1,4],[4,5]]`
  - **Output**: `[[1,5]]`
  - **Explanation**: Intervals `[1,4]` and `[4,5]` are considered overlapping.

22.2 Detailed Solution (C++)

To merge intervals efficiently, we can use a greedy sorting approach: 1. **Sort the intervals** by their starting value. This ensures that any intervals that could potentially overlap are placed next to each other. 2. **Iterate through the sorted list** and maintain a list of merged intervals. * Initialize `merged` with the first interval. * For each subsequent interval `current`, compare its start with the end of the last interval in `merged`. * If `current[0] <= merged.back()[1]`, they overlap. Merge them by updating the end of the last interval to `max(merged.back()[1], current[1])`. * If they do not overlap, simply append `current` to `merged`.

22.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    std::vector<std::vector<int>> merge(std::vector<std::vector<int>>& intervals) {
        // Edge Case: Empty input
        if (intervals.empty()) {
            return {};
        }

        // Sort intervals by starting time. std::sort on std::vector<int> compares elements lexicographicall
        // which naturally sorts by start time (intervals[i][0]).
        std::sort(intervals.begin(), intervals.end());

        std::vector<std::vector<int>> merged;
        merged.push_back(intervals[0]);

        for (size_t i = 1; i < intervals.size(); ++i) {
            // Overlap check: if current start is <= previous end
            if (intervals[i][0] <= merged.back()[1]) {
                merged.back()[1] = std::max(merged.back()[1], intervals[i][1]);
            } else {
                // No overlap, push to merged result
                merged.push_back(intervals[i]);
            }
        }

        return merged;
    }
};
```

22.3 Detailed Solution (Python)

22.3.1 Standard Python Production Code

Below is the type-hinted Python implementation. It avoids copying list elements by modifying the last element of the result in-place.

```python
from typing import List

class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        """
        Merges overlapping intervals by sorting by start time.

        Time Complexity: O(N log N)
        Space Complexity: O(N) for the merged output
        """
        if not intervals:
            return []

        # Sort intervals in-place by starting point
        intervals.sort(key=lambda x: x[0])

        merged = [intervals[0]]

        for start, end in intervals[1:]:
            last_end = merged[-1][1]

            # If current interval overlaps with the last merged one, merge them
            if start <= last_end:
                merged[-1][1] = max(last_end, end)
            else:
                # Disjoint interval, append directly
                merged.append([start, end])

        return merged
```

22.4 Python-Specific Complexities & Implementation Considerations

22.4.1 1. TimSort and Custom Sorting Key

- Python's `list.sort()` uses **Timsort** under the hood, which runs in $\mathcal{O}(n \log n)$ time. Timsort is highly optimized for runs of already sorted data.

- Sorting lists of lists: using `intervals.sort(key=lambda x: x[0])` tells Python to sort solely based on the first element (start time). If we did not provide `key=lambda x: x[0]`, Python would fall back to comparing the second elements when the first elements are equal. Since we only need to order by the start time, specifying `key=lambda x: x[0]` is slightly more efficient.

22.4.2 2. In-Place List Operations

- Instead of copying subarrays (which incurs memory allocation overhead), the solution appends references to the sublists or unpacks them. Be aware that `merged = [intervals[0]]` adds a reference to `intervals[0]`. In a system environment where the original input must not be modified, you should create a copy, e.g., `merged = [list(intervals[0])]`.

22.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n)$	Sorting the array of n intervals takes $\mathcal{O}(n \log n)$ time. The linear traversal and merging phase take $\mathcal{O}(n)$ time.
Space Complexity	$\mathcal{O}(n)$	Standard implementations allocate memory for the merged list. (If sorted in-place, auxiliary space is $\mathcal{O}(\log n)$ or $\mathcal{O}(1)$ depending on the sort implementation).

22.6 Common Follow-Up Questions & How to Answer

22.6.1 Q1: Can we perform the merge in-place to achieve $\mathcal{O}(1)$ auxiliary space?

- **The Answer**: Yes. We can reuse the input vector as a buffer by keeping track of a `write_idx` pointer.
- **Implementation (C++)**:

```cpp
std::sort(intervals.begin(), intervals.end());
int write_idx = 0;
for (size_t i = 1; i < intervals.size(); ++i) {
    if (intervals[i][0] <= intervals[write_idx][1]) {
        intervals[write_idx][1] = std::max(intervals[write_idx][1], intervals[i][1]);
    } else {
        write_idx++;
        intervals[write_idx] = intervals[i];
    }
```

```
}
intervals.resize(write_idx + 1);
```

This avoids allocating a new vector and returns the filtered array in-place, which is critical for memory-constrained system programming.

22.6.2 Q2: What if the input intervals are already sorted by their start values?

- **The Answer**: If the input is pre-sorted, we can skip the sorting step entirely. The algorithm's time complexity becomes $\mathcal{O}(n)$ and we can perform the merge in a single linear scan.

22.6.3 Q3: How do you handle an online/streaming data version of this problem?

- **The Answer**: If intervals arrive one by one in real-time, we cannot sort the array every time.
  - Instead, we maintain a balanced BST of disjoint intervals (like `std::set` in C++).
  - When a new interval $[S, E]$ arrives, we query the BST to find all intervals overlapping with $[S, E]$ in $\mathcal{O}(\log n)$ time.
  - We merge $[S, E]$ with the overlapping intervals, remove the old intervals, and insert the new merged interval.

22.7 Pro-Tip: How to Impress the Interviewer

- **Design for Memory Safety**: In C++, explain that using `std::move` when inserting non-overlapping intervals (e.g. `merged.push_back(std::move(intervals[i]))`) can save heap allocations and copies if the sub-containers are large.
- **Mention Cache Coherency of 2D Vectors**: Note that `std::vector<std::vector<int>>` is a vector of vector structures, which stores pointers to heap-allocated sub-vectors. This structure has poor cache locality. For a high-performance engine, we would store intervals as a flat `std::vector<std::pair<int, int>>` or a flat `int*` array, where elements are stored contiguously in memory.

23 Insert Interval

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 57, Glassdoor
- **Flashcards**: Merge Intervals deck

23.1 Question Description

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [start_i, end_i]` represent the start and the end of the i-th interval and `intervals` is sorted in ascending order by `start_i`.

You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `start_i` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return `intervals` after the insertion.

Note: You do not need to modify `intervals` in-place. You can make a new array and return it.

23.1.1 Examples

- **Input**: `intervals = [[1,3],[6,9]]`, `newInterval = [2,5]`
  - **Output**: `[[1,5],[6,9]]`
  - **Explanation**: The new interval `[2,5]` overlaps with `[1,3]`, merging them into `[1,5]`.
- **Input**: `intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]]`, `newInterval = [4,8]`
  - **Output**: `[[1,2],[3,10],[12,16]]`
  - **Explanation**: Because the new interval `[4,8]` overlaps with `[3,5],[6,7],[8,10]`, they are merged into one single interval `[3,10]`.

23.2 Detailed Solution (C++)

Because the input `intervals` is already sorted, we can build the result in a single pass of three sequential steps: 1. **Add all intervals** that end before the `newInterval` starts (no overlap possible). 2. **Merge all intervals** that overlap with the `newInterval`. An interval overlaps with `newInterval` if its start point is less than or equal to `newInterval[1]`. For each overlapping interval, we update `newInterval`'s start to the minimum of both starts, and its end to the maximum of both ends. Once the overlap loop finishes, we push the merged `newInterval` to our result. 3. **Add all remaining intervals** that start after `newInterval` ends.

23.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    std::vector<std::vector<int>> insert(const std::vector<std::vector<int>>& intervals, std::vector<int>& n
        std::vector<std::vector<int>> result;
        const size_t n = intervals.size();
        result.reserve(n + 1);
        size_t i = 0;

        // Step 1: Add all intervals that end before newInterval starts
        while (i < n && intervals[i][1] < newInterval[0]) {
            result.push_back(intervals[i]);
```

```
            i++;
        }

        // Step 2: Merge overlapping intervals
        while (i < n && intervals[i][0] <= newInterval[1]) {
            newInterval[0] = std::min(newInterval[0], intervals[i][0]);
            newInterval[1] = std::max(newInterval[1], intervals[i][1]);
            i++;
        }
        result.push_back(newInterval);

        // Step 3: Add all remaining intervals
        while (i < n) {
            result.push_back(intervals[i]);
            i++;
        }

        return result;
    }
};
```

23.3 Detailed Solution (Python)

23.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the three-stage linear traversal.

```
from typing import List

class Solution:
    def insert(self, intervals: List[List[int]], newInterval: List[int]) -> List[List[int]]:
        """
        Inserts a new interval into a sorted list of non-overlapping intervals,
        merging overlapping intervals as needed.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        result: List[List[int]] = []
        i = 0
        n = len(intervals)

        # Step 1: Add all intervals ending before newInterval starts
        while i < n and intervals[i][1] < newInterval[0]:
```

```
        result.append(intervals[i])
        i += 1

    # Step 2: Merge overlapping intervals
    while i < n and intervals[i][0] <= newInterval[1]:
        newInterval[0] = min(newInterval[0], intervals[i][0])
        newInterval[1] = max(newInterval[1], intervals[i][1])
        i += 1
    result.append(newInterval)

    # Step 3: Add all remaining intervals
    while i < n:
        result.append(intervals[i])
        i += 1

    return result
```

23.4 Python-Specific Complexities & Implementation Considerations

23.4.1 1. Sublist Copying vs Reference Appends

- In Python, `result.append(intervals[i])` appends a reference to the existing list object, which is fast ($\mathcal{O}(1)$ time).
- However, `result.append(newInterval)` references the mutable `newInterval` list. Since `newInterval` was modified during the merging phase, appending it directly is fine as long as we do not modify `newInterval` afterwards. To be completely safe and avoid mutating shared objects, `result.append(list(newInterval))` or `result.append(newInterval[:])` can be used.

23.4.2 2. Guarding against Out-Of-Bounds

- The indexing checks `while i < n` must precede the sub-index accesses `intervals[i][1]` or `intervals[i][0]` to prevent `IndexError` exceptions. Python evaluates logical conditions lazily, so placing `i < n` first guarantees the index is valid before evaluation.

23.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We perform a single pass over the intervals array, processing each interval at most once.

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(n)$	We allocate a new list/vector to store the resulting list of intervals.

23.6 Common Follow-Up Questions & How to Answer

23.6.1 Q1: Can we use Binary Search to find the insertion and merging points?

- **The Answer**: Yes. Because the list is sorted, we can search for the first interval that might overlap.
  - Find the insertion index using binary search: the first interval whose end time is $\geq$ `newInterval[0]`.
  - Find the termination index: the first interval whose start time is $>$ `newInterval[1]`.
  - While binary search identifies these indices in $\mathcal{O}(\log n)$ time, constructing the merged array (or performing in-place insertion/deletion in C++) still takes $\mathcal{O}(n)$ due to shifting elements in contiguous memory. Thus, the asymptotic time complexity remains $\mathcal{O}(n)$, but it reduces the number of comparisons.

23.6.2 Q2: What if we are required to do it strictly in-place in C++?

- **The Answer**: We can do it by finding the overlap bounds, replacing the first overlapping element with the merged interval, and erasing the remaining overlapping elements using `std::vector::erase`. However, vector `erase` calls shift all subsequent elements, which runs in $\mathcal{O}(n)$ time anyway.

23.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Trade-offs of Vector Reservation**: In C++, we can optimize our memory footprint by reserving memory for the result vector: `result.reserve(intervals.size() + 1)`. This avoids multiple reallocations and copies as the vector grows dynamically, which is a major system-level optimization.
- **Point out cache locality advantages**: If intervals are structured as a custom struct rather than `std::vector<int>` (e.g. `struct Interval { int start; int end; };`), the compiler can pack them contiguously in memory without double-indirection, resulting in vastly superior cache performance.

24 Meeting Rooms II

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 253, Glassdoor
- **Flashcards**: Merge Intervals deck

24.1 Question Description

Given an array of meeting time intervals `intervals` where `intervals[i] = [start_i, end_i]`, return *the minimum number of conference rooms required.*

24.1.1 Examples

- **Input**: `intervals = [[0,30],[5,10],[15,20]]`
  - **Output**: `2`
  - **Explanation**:
    - Meeting 1 starts at 0 and ends at 30. We need Room 1.
    - Meeting 2 starts at 5 and ends at 10. Room 1 is occupied. We need Room 2.
    - Meeting 3 starts at 15 and ends at 20. Room 2 is free (since Meeting 2 finished at 10), so we reuse Room 2.
    - Maximum concurrent rooms needed is 2.
- **Input**: `intervals = [[7,10],[2,4]]`
  - **Output**: `1`
  - **Explanation**: The meetings do not overlap, so only 1 room is needed.

24.2 Detailed Solution (C++)

A standard chronological sweep-line approach operates by separating the start and end times: 1. **Extract and Sort**: Store all start times in one array `starts` and all end times in another array `ends`. Sort both arrays independently. 2. **Two Pointers**: Maintain a pointer for `starts` and a pointer for `ends`. * If a meeting starts at `starts[startPtr]` before the oldest meeting ends at `ends[endPtr]`, it means we must allocate a new room (`rooms++`). * If `starts[startPtr] >= ends[endPtr]`, the oldest meeting has finished. We can reuse its room. We increment `endPtr` to point to the next meeting ending time. 3. **Result**: The number of rooms counted at the end is the peak overlap of active meetings.

This approach is highly efficient as it eliminates the overhead of maintaining a heap/priority queue.

24.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    int minMeetingRooms(const std::vector<std::vector<int>>& intervals) {
        if (intervals.empty()) {
            return 0;
        }

        const size_t n = intervals.size();
        std::vector<int> starts;
```

```
        std::vector<int> ends;
        starts.reserve(n);
        ends.reserve(n);

        // Separate start and end times
        for (const auto& interval : intervals) {
            starts.push_back(interval[0]);
            ends.push_back(interval[1]);
        }

        // Sort both arrays independently
        std::sort(starts.begin(), starts.end());
        std::sort(ends.begin(), ends.end());

        int rooms = 0;
        size_t endPtr = 0;

        // Traverse all start times
        for (size_t startPtr = 0; startPtr < n; ++startPtr) {
            // If the meeting starts at or after the earliest ending meeting
            if (starts[startPtr] >= ends[endPtr]) {
                // Reuse the room, so advance the end pointer
                endPtr++;
            } else {
                // Overlap occurs, allocate a new room
                rooms++;
            }
        }

        return rooms;
    }
};
```

24.3 Detailed Solution (Python)

24.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the two-pointer separate-sorting algorithm.

```
from typing import List

class Solution:
    def minMeetingRooms(self, intervals: List[List[int]]) -> int:
        """
```

```
    Calculates the minimum conference rooms required using two-pointer sorting.

    Time Complexity: O(N log N)
    Space Complexity: O(N)
    """
    if not intervals:
        return 0

    # Extract and sort start/end times independently
    starts = sorted(interval[0] for interval in intervals)
    ends = sorted(interval[1] for interval in intervals)

    rooms = 0
    end_ptr = 0

    for start in starts:
        # If the next meeting starts after the earliest active meeting ends
        if start >= ends[end_ptr]:
            # Reuse the room (advance the end pointer)
            end_ptr += 1
        else:
            # No room is free, allocate a new room
            rooms += 1

    return rooms
```

24.4 Python-Specific Complexities & Implementation Considerations

24.4.1 1. Generator Expressions in `sorted`

- In Python, `sorted(interval[0] for interval in intervals)` uses a generator expression to feed elements into the sorting algorithm.
- This avoids creating an intermediate list of start times in memory before sorting, reducing memory allocation overhead.

24.4.2 2. Timsort Optimizations

- Because Timsort runs in $\mathcal{O}(n \log n)$ time, sorting `starts` and `ends` separately is highly efficient. In cases where the input meetings are already chronological, `starts` will sort in $\mathcal{O}(n)$ time due to Timsort detecting pre-sorted segments.

24.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n)$	Sorting the `starts` and `ends` arrays takes $\mathcal{O}(n \log n)$ time. The subsequent two-pointer scan takes linear $\mathcal{O}(n)$ time.
Space Complexity	$\mathcal{O}(n)$	We allocate two arrays of size n to hold the start and end times.

24.6 Common Follow-Up Questions & How to Answer

24.6.1 Q1: How does this Two-Pointer approach compare to a Heap-based solution?

- **The Answer**:
  - **Min-Heap Approach**: We sort the intervals by start time, and push the end times of active meetings into a min-heap. For each new meeting, we pop from the heap if its start time $\geq$ the top of the heap, then push its end time. The maximum size of the heap is the answer.
  - **Comparison**:
    - *Time Complexity*: Both are $\mathcal{O}(n \log n)$. However, the heap approach does $\mathcal{O}(n \log k)$ heap operations (where k is the room count).
    - *Memory/Cache*: The two-pointer approach sorted array scan has superior cache locality because it traverses contiguous arrays sequentially. Heap operations, on the other hand, require pointer hopping across nodes, resulting in cache misses.
    - *Streaming Advantage*: If intervals arrive as a stream in real-time, the Min-Heap is superior because we do not need to re-sort arrays; we just insert/extract from the heap in $\mathcal{O}(\log k)$ time per event.

24.6.2 Q2: How would you solve this using a Prefix Sum / Event-Based Sweep Line?

- **The Answer**: We can represent each start as a `+1` event and each end as a `-1` event.
  - Create a list of pairs `(time, event_type)` where `event_type = 1` for start and `-1` for end.
  - Sort the list. **Crucial rule**: If two times are equal, sort the end event (`-1`) *before* the start event (`+1`) so we don't count an extra room for contiguous meetings.
  - Traverse the sorted events and maintain a running prefix sum of active rooms. The maximum value of this sum is the answer.

24.7 Pro-Tip: How to Impress the Interviewer

- **Reserve Vector Capacity**: In C++, always write `starts.reserve(n)` and `ends.reserve(n)`. This signals to the interviewer that you understand vector growth mechanics and want to avoid the performance cost of multiple dynamic reallocations.
- **Cache Locality and Memory Alignment**: Highlight that separating intervals into two primitive `std::vector<int>` arrays (`starts` and `ends`) is far more cache-friendly than sorting

`std::vector<std::vector<int>>` or `std::vector<std::pair<int, int>>` because it keeps primitive integers aligned contiguously in the CPU cache lines.

25 Teemo Attacking

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 495, Glassdoor
- **Flashcards**: Merge Intervals deck

25.1 Question Description

Our hero Teemo is attacking an enemy Ashe with poison attacks! When Teemo attacks Ashe, Ashe gets poisoned for exactly `duration` seconds. More formally, an attack at second `t` will mean Ashe is poisoned during the inclusive time interval `[t, t + duration - 1]`. If Teemo attacks again before the poison effect ends, the timer for it is reset, and the poison effect will end `duration` seconds after the new attack.

You are given a non-decreasing integer array `timeSeries`, where `timeSeries[i]` denotes that Teemo attacks Ashe at second `timeSeries[i]`, and an integer `duration`.

Return *the total number of seconds that Ashe is poisoned*.

25.1.1 Examples

- **Input**: `timeSeries = [1,4]`, `duration = 2`
  - **Output**: `4`
  - **Explanation**: Teemo's attacks on Ashe go as follows:
    - At second 1, Teemo attacks, and Ashe is poisoned for seconds 1 and 2.
    - At second 4, Teemo attacks, and Ashe is poisoned for seconds 4 and 5.
    - Ashe is poisoned for seconds 1, 2, 4, and 5, which is 4 seconds in total.
- **Input**: `timeSeries = [1,2]`, `duration = 2`
  - **Output**: `3`
  - **Explanation**: Teemo's attacks on Ashe go as follows:
    - At second 1, Teemo attacks, and Ashe is poisoned for seconds 1 and 2.
    - At second 2, Teemo attacks again and resets the poison timer. Ashe is poisoned for seconds 2 and 3.
    - Ashe is poisoned for seconds 1, 2, and 3, which is 3 seconds in total.

25.2 Detailed Solution (C++)

This problem can be modeled as a simplified interval-merging simulation. 1. Because `timeSeries` is already sorted in non-decreasing order, we can compare adjacent elements. 2. For any attack i at `timeSeries[i]`, the poison lasts until `timeSeries[i] + duration`. 3. If the next attack at `timeSeries[i+1]` occurs before

the current poison expires, the actual poisoned duration for attack i is capped at `timeSeries[i+1] - timeSeries[i]`. 4. Otherwise, the poison runs its full course of `duration` seconds. 5. Thus, the poison duration contributed by attack i is:

$$\min(\text{duration}, \text{timeSeries}[i+1] - \text{timeSeries}[i])$$

6. The last attack (at index $n-1$) always runs for the full `duration` because there are no subsequent attacks.

25.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    int findPoisonedDuration(const std::vector<int>& timeSeries, int duration) noexcept {
        // Edge Case: No attacks
        if (timeSeries.empty()) {
            return 0;
        }

        int totalDuration = 0;
        const size_t n = timeSeries.size();

        // Calculate the poisoned time contributed by each attack up to the second-to-last
        for (size_t i = 0; i < n - 1; ++i) {
            totalDuration += std::min(duration, timeSeries[i + 1] - timeSeries[i]);
        }

        // The final attack is guaranteed to run for the full duration
        return totalDuration + duration;
    }
};
```

25.3 Detailed Solution (Python)

25.3.1 Standard Python Production Code

Below is the type-hinted Python implementation. It avoids indexing errors by iterating through the array up to the second-to-last element.

```python
from typing import List

class Solution:
```

```python
def findPoisonedDuration(self, timeSeries: List[int], duration: int) -> int:
    """
    Calculates the total duration of the poison effect.

    Time Complexity: O(N)
    Space Complexity: O(1)
    """
    if not timeSeries:
        return 0

    total_duration = 0
    n = len(timeSeries)

    # Iterate over consecutive attack timestamps
    for i in range(n - 1):
        total_duration += min(duration, timeSeries[i + 1] - timeSeries[i])

    # Add the full duration for the last attack
    return total_duration + duration
```

25.4 Python-Specific Complexities & Implementation Considerations

25.4.1 1. Large Integers and Precision

- The constraints mention that `timeSeries[i]` and `duration` can be up to 10^7, and the length of the array can be up to 10^4.
- The maximum potential answer is $10^4 \times 10^7 = 10^{11}$, which fits easily within a standard 64-bit integer. Python handles arbitrarily large integers automatically, so overflow is not a concern. In C++, a standard 32-bit signed `int` goes up to $\approx 2 \times 10^9$, so 10^{11} would overflow a 32-bit signed integer. However, since the LeetCode return type is `int` (implying tests fit in 32-bit), standard `int` is used, but pointing this out to the interviewer shows excellent low-level engineering foresight.

25.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We scan the `timeSeries` array of size n in a single pass.
Space Complexity	$\mathcal{O}(1)$	No extra memory structures are allocated; only a few scalar variables are maintained.

25.6 Common Follow-Up Questions & How to Answer

25.6.1 Q1: How does this problem relate to the Merge Intervals pattern?

- **The Answer**: Each attack represents an interval `[timeSeries[i], timeSeries[i] + duration - 1]`.
  - If we used the standard **Merge Intervals** algorithm, we would sort the intervals and merge overlaps, which takes $\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$ time and $\mathcal{O}(n)$ space.
  - However, since the intervals are already sorted by start time and have a fixed length, we can calculate the union size on the fly. This optimization reduces the auxiliary space complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$.

25.6.2 Q2: What if we have variable durations for each attack (e.g. `durationSeries[i]`)?

- **The Answer**: If durations vary, a simple pairwise comparison is no longer sufficient. An earlier attack with a very long duration could overlap and fully cover subsequent attacks.
  - To solve this, we can maintain the end time of the active poison: `current_poison_end`.
  - For each attack at `t = timeSeries[i]` with duration `d = durationSeries[i]`:
    - If `t > current_poison_end` (no overlap), we add `d` to our total, and update `current_poison_end = t + d`.
    - If `t <= current_poison_end` (overlap), we only add the incremental portion: if `t + d > current_poison_end`, we add `(t + d) - current_poison_end` to our total, and update `current_poison_end = t + d`.
  - This sweep-line logic still takes $\mathcal{O}(n)$ time and $\mathcal{O}(1)$ space.

25.7 Pro-Tip: How to Impress the Interviewer

- **Mention 32-bit Integer Overflow**: Point out that if the cumulative poison duration exceeds $2^{31}-1$, a standard 32-bit signed integer in C++ will overflow. Suggesting the use of `long long` for the accumulator variable (`totalDuration`) demonstrates a production-first mindset regarding mathematical overflow.
- **Discuss Compiler Vectorization**: Note that the loop only performs basic arithmetic (`min` and subtraction) on contiguous elements of `timeSeries`. Because there are no data dependencies between iterations (each step relies solely on i and $i+1$), modern compilers can auto-vectorize this loop using SIMD instructions (AVX/SSE) to process multiple attacks per CPU cycle.

26 Insert Delete GetRandom O(1)

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 380, Glassdoor
- **Flashcards**: Hashmap deck

26.1 Question Description

Implement the `RandomizedSet` class:

- `RandomizedSet()` Initializes the `RandomizedSet` object.
- `bool insert(int val)` Inserts an item `val` into the set if not present. Returns `true` if the item was not already present, `false` otherwise.
- `bool remove(int val)` Removes an item `val` from the set if present. Returns `true` if the item was present, `false` otherwise.
- `int getRandom()` Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the **same probability** of being returned.

You must implement the functions of the class such that each function works in **average** $\mathcal{O}(1)$ time complexity.

26.1.1 Examples

- **Input**:

```
["RandomizedSet", "insert", "remove", "insert", "getRandom", "remove", "insert", "getRandom"]
[[], [1], [2], [2], [], [1], [2], []]
```

- **Output**:

```
[null, true, false, true, 2, true, false, 2]
```

- **Explanation**:

```
RandomizedSet randomizedSet;
randomizedSet.insert(1);   // Inserts 1 to the set. Returns true (1 inserted successfully).
randomizedSet.remove(2);   // Returns false as 2 does not exist in the set.
randomizedSet.insert(2);   // Inserts 2 to the set, returns true. Set now contains [1,2].
randomizedSet.getRandom(); // getRandom() should return either 1 or 2 randomly with equal probability.
randomizedSet.remove(1);   // Removes 1 from the set, returns true. Set now contains [2].
randomizedSet.insert(2);   // 2 was already in the set, so return false.
randomizedSet.getRandom(); // Since 2 is the only number in the set, getRandom() always returns 2.
```

26.2 Detailed Solution (C++)

To implement all three operations in average $\mathcal{O}(1)$ time: 1. **getRandom() in** $\mathcal{O}(1)$: We must be able to choose a random index in $[0, N-1]$ and look up the element at that index in $\mathcal{O}(1)$ time. This requires a **dynamic array** (`std::vector` in C++). 2. **insert(val) in** $\mathcal{O}(1)$: We append `val` to the end of our dynamic array, and record its index in a **hash map** (`std::unordered_map` in C++) mapping `val -> index`. 3. **remove(val) in** $\mathcal{O}(1)$: We look up the index of `val` in the hash map. To avoid shifting elements (which would cost $\mathcal{O}(N)$), we swap the target element with the **last** element in our array. Then, we update the last element's index in the hash map, pop the last element from the array, and delete the target key from the hash map.

26.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <unordered_map>
#include <random>
#include <stdexcept>

class RandomizedSet {
private:
    std::vector<int> vals;
    std::unordered_map<int, int> idx_map;

    // Modern C++ Random Engine
    mutable std::mt19937 rng;

public:
    /** Initialize your data structure here. */
    RandomizedSet() : rng(std::random_device{}()) {}

    /** Inserts a value to the set. Returns true if the set did not already contain the specified element. */
    bool insert(int val) {
        if (idx_map.find(val) != idx_map.end()) {
            return false;
        }
        // Save the index where val will be placed in the vector
        idx_map[val] = static_cast<int>(vals.size());
        vals.push_back(val);
        return true;
    }

    /** Removes a value from the set. Returns true if the set contained the specified element. */
    bool remove(int val) {
        auto it = idx_map.find(val);
        if (it == idx_map.end()) {
            return false;
        }

        int target_idx = it->second;
        int last_val = vals.back();

        // Swap target element with the last element
        vals[target_idx] = last_val;
        idx_map[last_val] = target_idx; // Update last element's index
```

```
        // Remove the last element
        vals.pop_back();
        idx_map.erase(it);

        return true;
    }

    /** Get a random element from the set. */
    int getRandom() const {
        if (vals.empty()) {
            throw std::runtime_error("Set is empty");
        }
        std::uniform_int_distribution<size_t> dist(0, vals.size() - 1);
        return vals[dist(rng)];
    }
};
```

26.3 Detailed Solution (Python)

26.3.1 Standard Python Production Code

```
import random

class RandomizedSet:
    def __init__(self):
        """
        Initializes the RandomizedSet data structure.
        """
        self.vals: list[int] = []
        self.idx_map: dict[int, int] = {}

    def insert(self, val: int) -> bool:
        """
        Inserts a value to the set. Returns True if the set did not already contain the specified element.
        """
        if val in self.idx_map:
            return False

        self.idx_map[val] = len(self.vals)
        self.vals.append(val)
        return True

    def remove(self, val: int) -> bool:
```

```
        """
        Removes a value from the set. Returns True if the set contained the specified element.
        """
        if val not in self.idx_map:
            return False

        # Get target index and value of the last element
        target_idx = self.idx_map[val]
        last_val = self.vals[-1]

        # Swap target element with the last element
        self.vals[target_idx] = last_val
        self.idx_map[last_val] = target_idx

        # Pop the last element and remove from map
        self.vals.pop()
        del self.idx_map[val]
        return True

    def getRandom(self) -> int:
        """
        Get a random element from the set.
        """
        # random.choice uses constant time list index retrieval under the hood
        return random.choice(self.vals)
```

26.4 Python-Specific Complexities & Implementation Considerations

26.4.1 1. The Swap-and-Pop Performance Optimization

- Using `list.remove(val)` or `list.pop(idx)` in Python is a common pitfall. Both shift elements to fill the gap, incurring $\mathcal{O}(N)$ time complexity.
- By copying the last element into the target index (`self.vals[target_idx] = last_val`) and running `self.vals.pop()`, we achieve a true $\mathcal{O}(1)$ operation because popping the last element of a Python list does not require element shifting.

26.4.2 2. Time Complexity of `in` Operator on List vs Dict

- Checking `val in self.vals` takes $\mathcal{O}(N)$ time.
- Checking `val in self.idx_map` utilizes the dictionary's underlying hash table, taking average $\mathcal{O}(1)$ time. Always check existence using the dictionary.

26.4.3 3. Memory Allocation Overhead of Dicts

- Python dictionaries have significant memory overhead due to hash table allocation, collision resolution structures, and object wrapping.
- If memory footprint is critical, pre-sizing or utilizing a custom hash map could be considered, though standard python `dict` is highly optimized in CPython 3.6+.

26.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(1)$ average	Hash map lookups/deletions and dynamic array appends/pops execute in average constant time.
Space Complexity	$\mathcal{O}(n)$	We store n elements in both the dynamic array and the hash map.

26.6 Common Follow-Up Questions & How to Answer

26.6.1 Q1: What if duplicates are allowed? (LeetCode 381)

- **Answer**: When duplicates are allowed, a value can map to multiple indices.
  1. Change the map schema to `std::unordered_map<int, std::unordered_set<int>>` mapping each value to a set of indices.
  2. For `remove(val)`, retrieve the set of indices for `val`, grab any index from it (e.g. the first one), swap it with the last element in the array, and update the index set of the swapped element.

26.6.2 Q2: How does the load factor of the hash map affect the average-case $\mathcal{O}(1)$ time complexity?

- **Answer**: As elements are added, if the load factor (ratio of elements to buckets) exceeds a threshold, the hash map reallocates and rehashes all keys, costing $\mathcal{O}(N)$ in the worst case.
- To achieve stable performance in latency-sensitive systems, we can call `reserve()` or `rehash()` in C++ beforehand if the maximum size is known, avoiding runtime rehashing.

26.7 Pro-Tip: How to Impress the Interviewer

- **Mention Iterator Invalidation**: In C++, pushing elements into `std::vector` can cause reallocation, which invalidates all iterators, pointers, and references to elements. Although we only store raw integers (where invalidation is less dangerous than object references), pointing this out shows strong systems-level C++ hygiene.

- **Randomness Determinism**: Highlight that in testing and production systems, it is vital to seed the random engine properly (using `std::random_device` for non-deterministic behavior in production, and deterministic seeds for reproducible test cases).

27 Linked List Random Node

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 382, Glassdoor
- **Flashcards**: Hashmap deck

27.1 Question Description

Given a singly linked list, return a random node's value from the linked list. Each node must have the **same probability** of being chosen.

Implement the `Solution` class:

- `Solution(ListNode head)` Initializes the object with the head of the singly-linked list.
- `int getRandom()` Chooses a node randomly from the list and returns its value. All the nodes of the list should be equally likely to be chosen.

27.1.1 Follow up:

- What if the linked list is extremely large and its length is unknown to you?
- Could you solve this efficiently without using extra space?

27.1.2 Examples

- **Input**:

```
["Solution", "getRandom", "getRandom", "getRandom", "getRandom", "getRandom"]
[[[1, 2, 3]], [], [], [], [], []]
```

- **Output**:

```
[null, 1, 3, 2, 2, 3]
```

- **Explanation**:

```
ListNode* head = new ListNode(1);
head->next = new ListNode(2);
head->next->next = new ListNode(3);
Solution solution(head);

// getRandom() should return either 1, 2, or 3 randomly.
// Each element should have equal probability of returning.
```

```
solution.getRandom(); // returns 1
solution.getRandom(); // returns 3
solution.getRandom(); // returns 2
```

27.2 Detailed Solution (C++)

When the list size is unknown or extremely large (streaming data), storing all values in memory is not feasible. We use **Reservoir Sampling**: 1. Initialize `res = head->val`. 2. Traverse the list starting from the second node (`head->next`). Let the current index be i (2-indexed). 3. For each node, generate a random number between 1 and i. 4. If the random number is 1, we replace the reservoir value `res` with the current node's value. 5. Move to the next node and repeat.

27.2.1 Standard C++ Production Code

```cpp
#include <random>

// Definition for singly-linked list.
struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};

class Solution {
private:
    ListNode* head;
    mutable std::mt19937 rng;

public:
    Solution(ListNode* h) : head(h), rng(std::random_device{}()) {}

    int getRandom() const {
        int res = head->val;
        ListNode* cur = head->next;
        int i = 2;

        while (cur) {
            // Pick a random number in [1, i]
            std::uniform_int_distribution<int> dist(1, i);
            if (dist(rng) == 1) {
                res = cur->val;
```

```
            }
            cur = cur->next;
            i++;
        }

        return res;
    }
};
```

27.3 Detailed Solution (Python)

27.3.1 Standard Python Production Code

```python
import random
from typing import Optional

# Definition for singly-linked list.
class ListNode:
    def __init__(self, val: int = 0, next: Optional['ListNode'] = None):
        self.val = val
        self.next = next

class Solution:
    def __init__(self, head: Optional[ListNode]):
        """
        Initializes the object with the head of the singly-linked list.
        """
        self.head = head

    def getRandom(self) -> int:
        """
        Chooses a node randomly from the list and returns its value.
        Uses Reservoir Sampling to handle unknown length in O(1) auxiliary space.
        """
        if not self.head:
            raise ValueError("The list is empty.")

        res = self.head.val
        cur = self.head.next
        i = 2

        while cur:
            # Generate a random integer in the range [1, i] (inclusive)
```

```
            if random.randint(1, i) == 1:
                res = cur.val
            cur = cur.next
            i += 1

        return res
```

27.4 Python-Specific Complexities & Implementation Considerations

27.4.1 1. Inclusive Range in `random.randint`

- In Python, `random.randint(a, b)` generates an integer inclusive of both bounds $[a, b]$.
- This is different from standard range-based functions like `range()` or `random.randrange(a, b)` which exclude the upper bound. Using `random.randint(1, i)` correctly gives a $1/i$ chance of selecting 1.

27.4.2 2. Stream Data vs Precomputation Trade-off

- If `getRandom()` is called many times, traversing the linked list of length N on each call takes $\mathcal{O}(N)$ time.
- In standard Python environments where memory is not extremely tight, pre-loading all list values into a Python `list` during initialization allows $\mathcal{O}(1)$ random lookup via `random.choice(self.vals)`, but requires $\mathcal{O}(N)$ memory.

27.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N)$	We traverse the entire linked list of length N to sample the stream.
Space Complexity	$\mathcal{O}(1)$	We only maintain a few pointer variables and a counter, regardless of the list size.

27.6 Common Follow-Up Questions & How to Answer

27.6.1 Q1: What if we need to select k random nodes instead of 1? (Algorithm R)

- **Answer**: To sample k items, we maintain a reservoir array of size k containing the first k nodes.
  - For the i-th node (where $i > k$), we generate a random index j in $[0, i-1]$.
  - If $j < k$, we replace `reservoir[j]` with the i-th node.
  - This ensures all nodes have a uniform k/N chance of being selected.

27.6.2 Q2: What is the math proof of Reservoir Sampling?

- **Answer**: By induction, the probability of selecting the i-th item at step i is $1/i$.
  - For the last item N, its final probability of survival is $1/N$.
  - For any previous item k, the probability of survival is the probability it was chosen at step k, multiplied by the probability it was not replaced at any subsequent step:

$$\mathbb{P}(\text{survives}) = \frac{1}{k} \times \left(1 - \frac{1}{k+1}\right) \times \left(1 - \frac{1}{k+2}\right) \times \cdots \times \left(1 - \frac{1}{N}\right)$$

$$\mathbb{P}(\text{survives}) = \frac{1}{k} \times \frac{k}{k+1} \times \frac{k+1}{k+2} \times \cdots \times \frac{N-1}{N} = \frac{1}{N}$$

27.7 Pro-Tip: How to Impress the Interviewer

- **Mention Modulo Bias**: Point out that doing `rand() % i` in C/C++ introduces **modulo bias** if the range of the random engine is not a multiple of i. Explain that modern standard libraries provide `std::uniform_int_distribution` to avoid this and ensure a mathematically uniform distribution.
- **Explain Hardware Cache Inefficiency**: Explain that traversing a linked list for random selection is highly cache-inefficient because nodes are scattered in heap memory, causing frequent CPU cache line misses. In a production system, converting the linked list to a contiguous array (`std::vector`) is significantly faster unless memory constraints prevent it.

28 Perfect Rectangle

- **Category**: Coding
- **Difficulty**: Hard
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 391, Glassdoor
- **Flashcards**: Hashmap deck

28.1 Question Description

Given an array `rectangles` where `rectangles[i] = [xi, yi, ai, bi]` represents an axis-aligned rectangle. The bottom-left point of the rectangle is `(xi, yi)` and the top-right point of it is `(ai, bi)`.

Return `true` if all the rectangles together form an **exact cover** of a rectangular region (i.e., no gaps, no overlaps).

28.1.1 Examples

- **Input**: `rectangles = [[1,1,3,3],[3,1,4,2],[3,2,4,4],[1,3,2,4],[2,3,3,4]]`
  - **Output**: `true`
  - **Explanation**: All 5 rectangles together form an exact cover of a rectangular region.
- **Input**: `rectangles = [[1,1,2,3],[1,3,2,4],[3,1,4,2],[3,2,4,4]]`

  - **Output**: `false`
  - **Explanation**: Because there is a gap between the two rectangular regions.
- **Input**: `rectangles = [[1,1,3,3],[3,1,4,2],[1,3,2,4],[2,2,4,4]]`
  - **Output**: `false`
  - **Explanation**: Because two of the rectangles overlap with each other.

28.2 Detailed Solution (C++)

To verify if the set of sub-rectangles forms a perfect cover: 1. **Area Invariant**: The sum of the areas of all sub-rectangles must equal the area of the overall bounding box.

$$\text{Total Area} = (x_{\max} - x_{\min}) \times (y_{\max} - y_{\min})$$

2. **Corner Invariant**: In an exact cover, any internal corner must be shared by exactly 2 or 4 sub-rectangles. Any boundary corner (excluding the 4 main bounding box corners) must be shared by exactly 2 sub-rectangles. * If we track the corners and toggle their presence in a set (insert if absent, remove if present), all even-occurring corners will cancel out. * At the end of processing all rectangles, the set must contain **exactly** the four corners of the overall bounding box:

$$\{(x_{\min}, y_{\min}), (x_{\min}, y_{\max}), (x_{\max}, y_{\min}), (x_{\max}, y_{\max})\}$$

28.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <set>
#include <algorithm>
#include <utility>
#include <limits>

class Solution {
public:
    bool isRectangleCover(const std::vector<std::vector<int>>& rectangles) {
        std::set<std::pair<long long, long long>> corners;
        long long total_area = 0;

        long long min_x = std::numeric_limits<long long>::max();
        long long min_y = std::numeric_limits<long long>::max();
        long long max_x = std::numeric_limits<long long>::min();
        long long max_y = std::numeric_limits<long long>::min();

        for (const auto& rect : rectangles) {
            long long x1 = rect[0], y1 = rect[1];
            long long x2 = rect[2], y2 = rect[3];
```

```
            // Update bounding box boundaries
            min_x = std::min(min_x, x1);
            min_y = std::min(min_y, y1);
            max_x = std::max(max_x, x2);
            max_y = std::max(max_y, y2);

            // Accumulate areas safely using 64-bit integers to prevent overflow
            total_area += (x2 - x1) * (y2 - y1);

            // The 4 corners of the current rectangle
            std::pair<long long, long long> rect_corners[] = {
                {x1, y1}, {x1, y2}, {x2, y1}, {x2, y2}
            };

            // Toggle corner presence in the set
            for (const auto& point : rect_corners) {
                auto it = corners.find(point);
                if (it != corners.end()) {
                    corners.erase(it);
                } else {
                    corners.insert(point);
                }
            }
        }

        // Exact cover requires exactly the 4 outermost corners remaining
        if (corners.size() != 4) {
            return false;
        }

        std::set<std::pair<long long, long long>> expected_corners = {
            {min_x, min_y}, {min_x, max_y}, {max_x, min_y}, {max_x, max_y}
        };

        if (corners != expected_corners) {
            return false;
        }

        // Verify the sum of parts matches the whole area
        return total_area == (max_x - min_x) * (max_y - min_y);
    }
};
```

28.3 Detailed Solution (Python)

28.3.1 Standard Python Production Code

```python
from typing import List

class Solution:
    def isRectangleCover(self, rectangles: List[List[int]]) -> bool:
        """
        Determines if all rectangles together form an exact cover of a rectangular region.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        corners: set[tuple[int, int]] = set()
        total_area = 0

        min_x = min_y = float('inf')
        max_x = max_y = float('-inf')

        for x1, y1, x2, y2 in rectangles:
            min_x = min(min_x, x1)
            min_y = min(min_y, y1)
            max_x = max(max_x, x2)
            max_y = max(max_y, y2)

            # Accumulate sub-rectangle area
            total_area += (x2 - x1) * (y2 - y1)

            # Toggle corners
            for corner in [(x1, y1), (x1, y2), (x2, y1), (x2, y2)]:
                if corner in corners:
                    corners.remove(corner)
                else:
                    corners.add(corner)

        # There must be exactly 4 odd-count corners in the set
        if len(corners) != 4:
            return False

        # They must match the bounding box corners
        expected_corners = {
            (min_x, min_y),
            (min_x, max_y),
            (max_x, min_y),
```

```
    (max_x, max_y)
}

if corners != expected_corners:
    return False

# Area validation
return total_area == (max_x - min_x) * (max_y - min_y)
```

28.4 Python-Specific Complexities & Implementation Considerations

28.4.1 1. Hashability of Tuples vs Lists

- In Python, lists are mutable and therefore **unhashable**, so they cannot be stored in a `set`.
- We represent each corner coordinate as a 2-tuple (e.g., `(x1, y1)`), which is immutable and hashable, allowing $\mathcal{O}(1)$ average-time checks and insertions into the `corners` set.

28.4.2 2. Floating-Point Infinities for Min/Max Initialization

- Python's `float('inf')` and `float('-inf')` provide a clean way to initialize boundaries without relying on hardcoded constants like $2^{31} - 1$.
- Since comparisons between floats and integers work seamlessly in Python (`min(float('inf'), integer)` correctly returns the integer), this is a safe, readable pattern.

28.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Standard set operations (addition/deletion) run in $\mathcal{O}(1)$ average time per corner. (In C++, `std::set` takes $\mathcal{O}(n \log n)$; `std::unordered_set` can achieve $\mathcal{O}(n)$).
Space Complexity	$\mathcal{O}(n)$	In the worst case, the set stores all $4n$ corners of the input rectangles.

28.6 Common Follow-Up Questions & How to Answer

28.6.1 Q1: Can you solve this using a Sweep-Line algorithm?

- **Answer**: Yes. Sweep-line is the standard geometric way to solve overlaps.

1. Create events for the vertical boundaries of each rectangle (e.g., entering at x_1, leaving at x_2).
2. Sort these events by x.
3. Iterate through x and maintain the active y-intervals in a Segment Tree or interval tree.
4. Verify that entering intervals do not overlap with existing active intervals, and that the union of intervals is contiguous.

– **Pros**: Generalizes to finding the union of overlapping shapes.
– **Cons**: Higher complexity ($\mathcal{O}(n \log n)$ time and complex implementation).

28.6.2 Q2: How does this generalize to 3D box covers?

- **Answer**: In 3D, the area constraint becomes a volume constraint: Total Volume $= \sum$ Volumes.
  – The corners constraint extends to the vertices of 3D boxes. A perfect cover in 3D would result in exactly 8 outermost corners remaining in our vertex set after parity cancellation.

28.7 Pro-Tip: How to Impress the Interviewer

- **Proactively Avoid Integer Overflow**: Explicitly mention that multiplying coordinates like $(x_2 - x_1) \times (y_2 - y_1)$ can easily overflow a 32-bit integer in C++ if coordinates are large (e.g. 10^5). Show production hygiene by casting to `long long` before computing area.
- **Explain the Topological Parity Invariant**: Rather than just reciting the code, explain *why* the corner parity cancellation works: internal corners are shared by an even number of rectangles (2 or 4), which cancel out, while outermost corners are shared by exactly 1 rectangle, surviving the parity check.

29 Random Pick Index

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 398, Glassdoor
- **Flashcards**: Hashmap deck

29.1 Question Description

Given an integer array `nums` with possible duplicates, randomly output the index of a given `target` number. You can assume that the given `target` number must exist in the array.

Implement the `Solution` class:

- `Solution(int[] nums)` Initializes the object with the array `nums`.
- `int pick(int target)` Picks a random index `i` where `nums[i] == target`. Each index where `nums[i] == target` must have the **same probability** of being returned.

29.1.1 Examples

- **Input**:

```
["Solution", "pick", "pick", "pick"]
[[[1, 2, 3, 3, 3]], [3], [1], [3]]
```

- **Output**:

```
[null, 4, 0, 2]
```

- **Explanation**:

```
Solution solution({1, 2, 3, 3, 3});
solution.pick(3); // returns 2, 3, or 4 randomly. Each index should have equal probability (33.3%).
solution.pick(1); // returns 0 since nums[0] is the only element equal to 1.
solution.pick(3); // returns 2, 3, or 4 randomly.
```

29.2 Detailed Solution (C++)

There are two main approaches to this problem, presenting a classic memory-versus-time trade-off:

1. **Hash Map Precomputation (Time-Optimal for Queries)**:
   - Store indices of all values in a hash map: `std::unordered_map<int, std::vector<int>>`.
   - Querying takes $\mathcal{O}(1)$ average time by selecting a random index from the mapped vector.
   - Requires $\mathcal{O}(N)$ additional space.
2. **Reservoir Sampling (Space-Optimal)**:
   - Only store a reference/pointer to `nums` during initialization ($\mathcal{O}(1)$ space).
   - In `pick(target)`, iterate through `nums` and count matching elements. For the k-th matching element, select its index with probability $1/k$.
   - Querying takes $\mathcal{O}(N)$ time but uses $\mathcal{O}(1)$ auxiliary space.

Below is the **Hash Map Precomputation** approach, which is ideal when multiple queries are expected.

29.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <unordered_map>
#include <random>

class Solution {
private:
    std::unordered_map<int, std::vector<int>> idx_map;
    mutable std::mt19937 rng;

public:
    Solution(const std::vector<int>& nums) : rng(std::random_device{}()) {
        for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
```

```
            idx_map[nums[i]].push_back(i);
        }
    }

    int pick(int target) const {
        const auto& indices = idx_map.at(target);
        std::uniform_int_distribution<size_t> dist(0, indices.size() - 1);
        return indices[dist(rng)];
    }
};
```

29.3 Detailed Solution (Python)

29.3.1 Standard Python Production Code

```python
import random
from collections import defaultdict
from typing import List

class Solution:
    def __init__(self, nums: List[int]):
        """
        Initializes the object with the array nums.
        Precomputes indices for O(1) query time.
        """
        self.idx_map = defaultdict(list)
        for i, val in enumerate(nums):
            self.idx_map[val].append(i)

    def pick(self, target: int) -> int:
        """
        Picks a random index where nums[i] == target.
        Time Complexity: O(1)
        """
        # random.choice returns an element from a non-empty sequence in O(1)
        return random.choice(self.idx_map[target])
```

29.4 Python-Specific Complexities & Implementation Considerations

29.4.1 1. Default Dictionary (`defaultdict`)

- Using `collections.defaultdict(list)` simplifies construction by avoiding boilerplate check-and-initialize operations (`if target not in map: map[target] = []`).

- This pattern executes faster than standard `dict.setdefault()` in CPython because it relies on C-level lookup optimizations.

29.4.2 2. Large Memory Footprint of `defaultdict`

- Storing a list of indices for every number in the input array can be memory-heavy in Python. Each integer object has a 28-byte overhead, and each list has dynamic resizing overallocation.
- If memory footprint is constrained, we should implement the **Reservoir Sampling** option.

29.5 Complexity Analysis

29.5.1 Hash Map Precomputation Approach (Shown Above)

Metric	Complexity	Description
Time Complexity (Init)	$\mathcal{O}(N)$	We iterate through the array of length N exactly once.
Time Complexity (Pick)	$\mathcal{O}(1)$ average	$\mathcal{O}(1)$ lookup in the hash map and $\mathcal{O}(1)$ random element selection.
Space Complexity	$\mathcal{O}(N)$	We store all N indices of the original array in the hash map.

29.6 Common Follow-Up Questions & How to Answer

29.6.1 Q1: What if the memory is extremely constrained? (Reservoir Sampling Approach)

- **Answer**: We can implement `pick` in $\mathcal{O}(1)$ auxiliary space and $\mathcal{O}(N)$ time:

```cpp
class Solution {
private:
    const std::vector<int>& nums_ref;
    mutable std::mt19937 rng;
public:
    Solution(const std::vector<int>& nums) : nums_ref(nums), rng(std::random_device{}()) {}

    int pick(int target) const {
        int result = -1;
        int count = 0;
        for (int i = 0; i < nums_ref.size(); ++i) {
            if (nums_ref[i] == target) {
                count++;
                std::uniform_int_distribution<int> dist(1, count);
                if (dist(rng) == 1) {
                    result = i;
```

```
                }
            }
        }
        return result;
    }
};
```

29.6.2 Q2: How do we choose between these two approaches in a real system?

- **Answer**: Compare the Query-to-Init Ratio.
  - If the number of queries $Q \ll N$, Reservoir Sampling is superior since precomputing indices is wasteful.
  - If $Q \approx N$ or $Q \gg N$, precomputation is far superior since it amortizes the $\mathcal{O}(N)$ construction time across all queries, resulting in $\mathcal{O}(1)$ per pick.

29.7 Pro-Tip: How to Impress the Interviewer

- **Mention Thread-Safety of RNGs**: Point out that standard random number engines (`std::mt19937` or Python's `random`) are **not thread-safe**. If `pick` is called by multiple threads concurrently, it can lead to race conditions. In C++, marking the random engine `thread_local` resolves this.
- **Prevent Reference Lifetime Issues**: When storing a reference to `nums` in the Reservoir Sampling constructor, warn the interviewer that the input vector must outlive the `Solution` class instance. If the caller deletes the array, the reference becomes a dangling pointer, leading to undefined behavior.

30 Number of Boomerangs

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 447, Glassdoor
- **Flashcards**: Hashmap deck

30.1 Question Description

You are given n points in the plane that are all **distinct**, where `points[i] = [xi, yi]`. A **boomerang** is a tuple of points `(i, j, k)` such that the distance between `i` and `j` equals the distance between `i` and `k` (the **order of the tuple matters**).

Return the number of boomerangs.

30.1.1 Examples

- **Input**: `points = [[0,0],[1,0],[2,0]]`

  - **Output**: `2`
  - **Explanation**: The two boomerangs are `[[1,0],[0,0],[2,0]]` and `[[1,0],[2,0],[0,0]]`. (Point `[1,0]` is the pivot).
- **Input**: `points = [[1,1],[2,2],[3,3]]`
  - **Output**: `2`
- **Input**: `points = [[1,1]]`
  - **Output**: `0`

30.1.2 Constraints

- $n ==$ points.length
- $1 \leq n \leq 500$
- points$[i].length == 2$
- $-10^4 \leq x_i, y_i \leq 10^4$
- All the points are **unique**.

30.2 Detailed Solution (C++)

A boomerang is determined by a **pivot** point `i` and two other points `j` and `k` that are equidistant from `i`. 1. For each point `i` in the dataset, treat it as the pivot. 2. Calculate the squared distance from `i` to all other points `j`. 3. Store the frequency of each squared distance in a hash map: `std::unordered_map<int, int>`. 4. For any squared distance d that appears c times, the number of ways to pick two distinct points j and k is the permutation of 2 items chosen from c, which is $c \times (c-1)$. 5. Sum these values across all distances and all pivots.

30.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <unordered_map>

class Solution {
public:
    int numberOfBoomerangs(const std::vector<std::vector<int>>& points) {
        int total_boomerangs = 0;
        int n = points.size();

        // We reuse the hash map to avoid repeated allocations in the outer loop
        std::unordered_map<int, int> dist_count;

        for (int i = 0; i < n; ++i) {
            dist_count.clear();
            // Pre-reserve capacity to minimize rehashing overhead
            dist_count.reserve(n);
```

```
            for (int j = 0; j < n; ++j) {
                if (i == j) continue;

                int dx = points[i][0] - points[j][0];
                int dy = points[i][1] - points[j][1];
                int dist_sq = dx * dx + dy * dy;

                dist_count[dist_sq]++;
            }

            for (const auto& pair : dist_count) {
                int count = pair.second;
                if (count > 1) {
                    total_boomerangs += count * (count - 1);
                }
            }
        }

        return total_boomerangs;
    }
};
```

30.3 Detailed Solution (Python)

30.3.1 Standard Python Production Code

```
from collections import defaultdict
from typing import List

class Solution:
    def numberOfBoomerangs(self, points: List[List[int]]) -> int:
        """
        Calculates the number of boomerangs in O(N^2) time.
        """
        total_boomerangs = 0

        for p1 in points:
            # Hash map to count occurrences of each squared distance from pivot p1
            dist_count = defaultdict(int)

            for p2 in points:
                # Square distance formula (no square root to preserve integer precision)
                dx = p1[0] - p2[0]
```

```
            dy = p1[1] - p2[1]
            dist_sq = dx * dx + dy * dy
            dist_count[dist_sq] += 1

        for count in dist_count.values():
            total_boomerangs += count * (count - 1)

    return total_boomerangs
```

30.4 Python-Specific Complexities & Implementation Considerations

30.4.1 1. Hash Map vs. Sorting-based Grouping

- **Hash Map approach (as shown above)**: Runs in $\mathcal{O}(N^2)$ time and uses $\mathcal{O}(N)$ memory. Since $N \leq 500$, this is highly efficient.
- **Sorting-based approach**: We can compute all squared distances for a pivot, sort them, and count consecutive equal values. This takes $\mathcal{O}(N^2 \log N)$ time and $\mathcal{O}(N)$ memory. While mathematically slower, sorting is implemented in C (Timsort) and can sometimes outperform hash map insertion overhead in Python for very small inputs.

30.4.2 2. Hash Collision Overhead

- Python's dictionary uses open addressing. When collision rates are high, lookup performance degrades. However, because our keys are unique squared distances, hash collision rates remain low.

30.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N^2)$	We iterate through all N points, and for each point, we compute the distance to the other $N-1$ points.
Space Complexity	$\mathcal{O}(N)$	The hash map stores at most N distinct distances for each pivot.

30.6 Common Follow-Up Questions & How to Answer

30.6.1 Q1: Why do we use squared Euclidean distance instead of the actual Euclidean distance?

- **Answer**: Using actual Euclidean distance requires calculating the square root ($\sqrt{dx^2 + dy^2}$).

1. **Precision**: Floating-point numbers have limited precision, which can cause points at mathematically equal distances to have slightly different floating-point values (e.g. `2.0000000000000004` vs `1.9999999999999998`), leading to hash map key mismatches.
2. **Performance**: Square root (`std::sqrt` or `math.sqrt`) is a computationally expensive CPU operation. Skipping it yields a significant speedup.

30.6.2 Q2: What if the order of the points in the boomerang did not matter?

- **Answer**: A boomerang is defined as an ordered tuple `(i, j, k)`. If the order of the outer endpoints `j` and `k` did not matter (i.e. `(i, j, k)` is equivalent to `(i, k, j)`), we would choose 2 elements from c combinations without order.
  - Instead of $c \times (c-1)$, the contribution of each distance cluster would be $\frac{c \times (c-1)}{2}$.

30.7 Pro-Tip: How to Impress the Interviewer

- **Demonstrate Integer Overflow Safety**: Walk the interviewer through the maximum possible value of `dx * dx + dy * dy`. Given that coordinates are between -10^4 and 10^4, the maximum delta is $2 \cdot 10^4$. The maximum squared distance is:

$$(2 \cdot 10^4)^2 + (2 \cdot 10^4)^2 = 8 \cdot 10^8$$

  Since standard 32-bit signed integers in C++ go up to 2.14×10^9, this calculation is guaranteed to be safe from integer overflow. Highlighting this analysis proves that you write system-safe code.
- **Optimize Map Re-allocations**: Mention that clearing and reserving capacity in the C++ `std::unordered_map` (e.g., `dist_count.clear()` and `dist_count.reserve(n)`) avoids allocating new nodes on every iteration of the outer loop, which saves critical CPU cycles in hot loops.

31 Keyboard Row

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 500, Glassdoor
- **Flashcards**: Hashmap deck

31.1 Question Description

Given an array of strings `words`, return the words that can be typed using letters of the alphabet on only **one row** of the American keyboard (QWERTY layout).

Note that the strings are case-insensitive, so both lowercase and uppercase versions of the same letter are treated as if they are in the same row.

The rows on the QWERTY keyboard are defined as: * **Row 1**: `"qwertyuiop"` * **Row 2**: `"asdfghjkl"` * **Row 3**: `"zxcvbnm"`

31.1.1 Examples

- **Input**: `words = ["Hello","Alaska","Dad","Peace"]`
  - **Output**: `["Alaska","Dad"]`
  - **Explanation**:
    - `"Alaska"` can be typed using only characters in Row 2.
    - `"Dad"` can be typed using only characters in Row 2.
    - `"Hello"` contains `'H'`, `'e'`, `'l'`, `'o'`, which span multiple rows.
- **Input**: `words = ["omk"]`
  - **Output**: `[]`
- **Input**: `words = ["adsdf","sfd"]`
  - **Output**: `["adsdf","sfd"]`

31.1.2 Constraints

- $1 \leq \text{words.length} \leq 20$
- $1 \leq \text{words}[i].length \leq 100$
- `words[i]` consists of English letters (both lowercase and uppercase).

31.2 Detailed Solution (C++)

To solve this problem efficiently: 1. Define a static lookup table mapping each of the 26 lowercase English letters directly to its row index (0, 1, or 2). 2. For each word, determine the target row based on its first character. 3. Check all subsequent characters in the word. If any character maps to a different row, discard the word immediately (**early exit**). 4. If all characters match the target row, add the word to the result vector.

31.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <string>
#include <cctype>

class Solution {
public:
    std::vector<std::string> findWords(const std::vector<std::string>& words) {
        // Map from lowercase letter ('a' - 'z') to its row number (0, 1, or 2)
        // Row 0: q,w,e,r,t,y,u,i,o,p
        // Row 1: a,s,d,f,g,h,j,k,l
        // Row 2: z,x,c,v,b,n,m
        static const int char_to_row[26] = {
            1, 2, 2, 1, 0, 1, 1, 1, 0, 1, 1, 1, 2, 2, 0, 0, 0, 0, 1, 0, 0, 2, 0, 2, 0, 2
```

```
        };

        std::vector<std::string> result;

        for (const std::string& word : words) {
            if (word.empty()) continue;

            // Get the row of the first character (using static_cast to prevent UB in std::tolower)
            int first_char = std::tolower(static_cast<unsigned char>(word[0])) - 'a';
            int target_row = char_to_row[first_char];
            bool same_row = true;

            for (size_t i = 1; i < word.length(); ++i) {
                int curr_char = std::tolower(static_cast<unsigned char>(word[i])) - 'a';
                if (char_to_row[curr_char] != target_row) {
                    same_row = false;
                    break; // Early exit
                }
            }

            if (same_row) {
                result.push_back(word);
            }
        }

        return result;
    }
};
```

31.3 Detailed Solution (Python)

31.3.1 Standard Python Production Code

```
from typing import List

class Solution:
    def findWords(self, words: List[str]) -> List[str]:
        """
        Filters words that can be typed using letters of only one QWERTY row.

        Time Complexity: O(L) where L is the total number of characters across all words.
        Space Complexity: O(1) auxiliary space.
        """
```

```
        rows = [
            set("qwertyuiop"),
            set("asdfghjkl"),
            set("zxcvbnm")
        ]

        result = []
        for word in words:
            # Convert word characters to a lowercase set
            word_set = set(word.lower())

            # Check if word_set is a subset of any row
            if any(word_set.issubset(row) for row in rows):
                result.append(word)

        return result
```

31.4 Python-Specific Complexities & Implementation Considerations

31.4.1 1. Set Operations and `issubset` Efficiency

- Python's `issubset` (or the `<=` operator) is written in C and is highly optimized.
- However, constructing a new set for every word (via `set(word.lower())`) allocates memory on the heap and creates garbage collector overhead. For performance-critical code, using a static mapping dictionary or list lookup is faster.

31.4.2 2. Case Folding

- We use `.lower()` to normalize the word. In internationalized environments, `.casefold()` is generally preferred over `.lower()` for case-insensitive comparisons, though for standard ASCII English keys, `.lower()` is perfectly sufficient.

31.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(L)$	We process each character of each word at most once, where L is the sum of lengths of all words.
Space Complexity	$\mathcal{O}(1)$	The alphabet mappings or row sets are constant size (26 characters).

31.6 Common Follow-Up Questions & How to Answer

31.6.1 Q1: How would you modify the solution to support a custom or international layout (like Dvorak or Colemak)?

- **Answer**: The structure of the code remains identical. The only change needed is updating the mapping table (`char_to_row` in C++ or the `rows` sets in Python) to reflect the key positions on the target keyboard.

31.6.2 Q2: If the input consists of millions of extremely long words, how would you optimize the performance?

- **Answer**:
  1. **Early Exit**: Terminate the check on a word immediately on the first character mismatch (already implemented in the C++ version).
  2. **Concurrency**: Since each word's check is independent, we can divide the word list among multiple threads using OpenMP in C++ or standard thread pools.

31.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Heap Allocations**: Highlight that by using a static lookup table (`static const int char_to_row[26]`) instead of dynamically creating sets or maps at runtime, the C++ solution operates with **zero heap allocations** for validation. This is a critical design pattern in low-latency systems.
- **Prevent Undefined Behavior in `tolower`**: In C++, passing a `char` directly to `std::tolower` can trigger undefined behavior if the value is negative (due to signed char expansion to EOF). Point out that casting the character to `unsigned char` first (`static_cast<unsigned char>(c)`) is the correct, standard-compliant way to prevent this crash.

32 Encode and Decode TinyURL

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 535, Glassdoor
- **Flashcards**: Hashmap deck

32.1 Question Description

TinyURL is a URL shortening service where you enter a URL such as `https://leetcode.com/problems/design-tinyurl` and it returns a short URL such as `http://tinyurl.com/4e9iAk`. Design a class to encode a URL and decode a tiny URL.

There is no restriction on how your encode/decode algorithm should work. You just need to ensure that a URL can be encoded to a tiny URL and the tiny URL can be decoded to the original URL.

Implement the `Solution` class:

- `Solution()` Initializes the object.
- `string encode(string longUrl)` Returns a tiny URL for the given `longUrl`.
- `string decode(string shortUrl)` Returns the original long URL for the given `shortUrl`.

32.1.1 Examples

- **Input**: `url = "https://leetcode.com/problems/design-tinyurl"`
  - **Output**: `"https://leetcode.com/problems/design-tinyurl"`
  - **Explanation**:

```
Solution solution;
string tiny = solution.encode("https://leetcode.com/problems/design-tinyurl");
// returns "http://tinyurl.com/000000" (or similar short code)
string original = solution.decode(tiny);
// returns "https://leetcode.com/problems/design-tinyurl"
```

32.2 Detailed Solution (C++)

For the single-machine coding version, we can map URLs using an incremental counter: 1. Use a counter that increments for each unique long URL encoded. 2. Convert the counter value to a **Base-62** string (characters `0-9`, `a-z`, `A-Z`) to represent the short code. A 6-character Base-62 string supports $62^6 \approx 56.8$ billion unique URLs. 3. Store the mapping `shortCode -> longUrl` and `longUrl -> shortCode` in two hash maps (`std::unordered_map`) to ensure both encoding and decoding run in $\mathcal{O}(1)$ average time.

32.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <unordered_map>
#include <algorithm>

class Solution {
private:
    std::unordered_map<std::string, std::string> code_map;
    std::unordered_map<std::string, std::string> url_map;
    int counter = 0;

    // Self-contained character set for Base-62 conversion
    const std::string chars = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

    std::string get_base62(int num) {
        if (num == 0) {
```

```
            return std::string(6, chars[0]);
        }
        std::string code = "";
        while (num > 0) {
            code += chars[num % 62];
            num /= 62;
        }
        std::reverse(code.begin(), code.end());
        // Left-pad with '0' to ensure fixed 6-character length
        if (code.size() < 6) {
            code = std::string(6 - code.size(), chars[0]) + code;
        }
        return code;
    }

public:
    // Encodes a URL to a shortened URL.
    std::string encode(const std::string& longUrl) {
        if (url_map.find(longUrl) != url_map.end()) {
            return "http://tinyurl.com/" + url_map[longUrl];
        }

        std::string code = get_base62(counter++);
        code_map[code] = longUrl;
        url_map[longUrl] = code;

        return "http://tinyurl.com/" + code;
    }

    // Decodes a shortened URL to its original URL.
    std::string decode(const std::string& shortUrl) {
        size_t last_slash = shortUrl.rfind('/');
        if (last_slash == std::string::npos) {
            return "";
        }
        std::string code = shortUrl.substr(last_slash + 1);
        return code_map.count(code) ? code_map[code] : "";
    }
};
```

32.3 Detailed Solution (Python)

32.3.1 Standard Python Production Code

```python
class Codec:
    def __init__(self):
        """
        Initializes the maps and character library for Base-62.
        """
        self.code_map: dict[str, str] = {}
        self.url_map: dict[str, str] = {}
        self.chars = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
        self.counter = 0

    def _get_base62(self, num: int) -> str:
        """
        Helper method to encode an integer into a 6-character Base-62 string.
        """
        if num == 0:
            return self.chars[0].zfill(6)

        code = []
        while num > 0:
            code.append(self.chars[num % 62])
            num //= 62

        # Join reversed characters and left-pad with '0' to length 6
        return "".join(reversed(code)).zfill(6)

    def encode(self, longUrl: str) -> str:
        """
        Encodes a URL to a shortened URL.
        """
        if longUrl in self.url_map:
            return f"http://tinyurl.com/{self.url_map[longUrl]}"

        short_code = self._get_base62(self.counter)
        self.counter += 1

        self.code_map[short_code] = longUrl
        self.url_map[longUrl] = short_code

        return f"http://tinyurl.com/{short_code}"

    def decode(self, shortUrl: str) -> str:
```

```
        """
        Decodes a shortened URL to its original URL.
        """
        # Split by '/' and take the last part representing the unique code
        short_code = shortUrl.split("/")[-1]
        return self.code_map.get(short_code, "")
```

32.4 Python-Specific Complexities & Implementation Considerations

32.4.1 1. Integer Precision and zfill

- Python handles arbitrarily large integers natively, meaning our `counter` will never overflow a standard data type, unlike standard fixed-width 32-bit integers in C++.
- `str.zfill(width)` is a convenient built-in Python string method that left-pads a numeric string with ASCII `'0'` characters to reach the target width.

32.4.2 2. URL Parsing Security

- Using `shortUrl.split("/")[-1]` is quick but vulnerable if the short URL contains trailing query parameters or fragments (e.g., `http://tinyurl.com/000000?ref=newsletter`).
- In production Python code, use `urllib.parse.urlparse` to safely extract only the path segment of the URL.

32.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(1)$	Dictionary lookups and insertions run in constant time. Base-62 conversion takes constant time since key length is bounded.
Space Complexity	$\mathcal{O}(N)$	We store the mappings for all N unique long URLs encoded.

32.6 Common Follow-Up Questions & How to Answer

32.6.1 Q1: How do you scale this service to handle billions of requests across a distributed system?

- **Answer**: In a distributed system:
  1. **Centralized ID Generation**: A single counter would cause thread contention and single-point-of-failure bottlenecks. Instead, use a distributed ID generator (e.g., **Twitter Snowflake** or a **ZooKeeper** coordinate server) to allocate ranges of IDs to each application server.

2. **Consistent Hashing**: Distribute the URL database across multiple database shards using consistent hashing.

32.6.2 Q2: How can you optimize this service for high read throughput?

- **Answer**: URL redirection is highly read-heavy.
  - Deploy a **Redis** or **Memcached** caching layer. Since most redirects follow a power-law distribution (a few URLs get 90% of clicks), storing active URL mapping keys in cache yields sub-millisecond response times.

32.7 Pro-Tip: How to Impress the Interviewer

- **Highlight URL Harvesting Vulnerability**: Warn the interviewer that using sequential counter-based codes (e.g. `000000`, `000001`) makes the service vulnerable to **URL harvesting**—adversaries can easily iterate through all sequence numbers and scrape the private target URLs. Suggest encrypting/obfuscating the counter (e.g. using a Feistel Cipher or a small encryption tool like Skip32) to generate pseudorandom looking short codes from sequential IDs.
- **Estimate Database Storage Size**: Calculate storage requirements to show real-world system architectural thinking:
  - Assume 100 million new URLs/day.
  - If each record (ID, long URL, short URL, metadata) takes 500 bytes:

$$\text{Storage/day} = 10^8 \times 500 \text{ bytes} \approx 50 \text{ GB/day}$$

$$\text{Storage/year} \approx 18 \text{ TB/year}$$

  - This shows you think like a Systems Architect, not just a programmer.

33 Distribute Candies

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer
- **Source**: LeetCode 575, Glassdoor
- **Flashcards**: Hashmap deck

33.1 Question Description

Alice has n candies, where the i-th candy is of type `candyType[i]`. Alice noticed that she started to gain weight, so she visited a doctor.

The doctor advised Alice to only eat $n/2$ of the candies she has (n is always even). Alice likes her candies very much, and she wants to eat the maximum number of different types of candies while still following the doctor's advice.

Given the integer array `candyType` of length n, return the **maximum number of different types** of candies she can eat if she only eats $n/2$ of them.

33.1.1 Examples

- **Input**: `candyType = [1,1,2,2,3,3]`
  - **Output**: `3`
  - **Explanation**: Alice can only eat $6/2 = 3$ candies. Since there are only 3 types (1, 2, and 3), she can eat one of each type.
- **Input**: `candyType = [1,1,2,3]`
  - **Output**: `2`
  - **Explanation**: Alice can only eat $4/2 = 2$ candies. Whether she eats types `[1,2]`, `[1,3]`, or `[2,3]`, the maximum number of different types she can eat is 2.
- **Input**: `candyType = [6,6,6,6]`
  - **Output**: `1`
  - **Explanation**: Alice can only eat $4/2 = 2$ candies. Even though she can eat 2 candies, there is only 1 type of candy available.

33.1.2 Constraints

- $n ==$ candyType.length
- $2 \leq n \leq 10^4$
- n is even.
- $-10^5 \leq \text{candyType}[i] \leq 10^5$

33.2 Detailed Solution (C++)

To maximize the number of unique candy types Alice can eat: 1. Find the number of unique candy types available, U. We can count this by inserting all elements of `candyType` into a hash set (`std::unordered_set`). 2. Alice can eat at most $n/2$ candies in total. 3. By the **Pigeonhole Principle**, the maximum number of unique types she can eat is bounded by both the number of unique types available (U) and the total candy allowance ($n/2$). 4. Therefore, the answer is $\min(U, n/2)$.

33.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <unordered_set>
#include <algorithm>

class Solution {
public:
    int distributeCandies(const std::vector<int>& candyType) {
        // Construct unordered_set to find unique candy types
        std::unordered_set<int> unique_types(candyType.begin(), candyType.end());
```

```
        int total_allowed = static_cast<int>(candyType.size() / 2);
        int unique_count = static_cast<int>(unique_types.size());

        return std::min(unique_count, total_allowed);
    }
};
```

33.3 Detailed Solution (Python)

33.3.1 Standard Python Production Code

```python
from typing import List

class Solution:
    def distributeCandies(self, candyType: List[int]) -> int:
        """
        Determines the maximum number of unique candy types Alice can eat.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        # Convert list to set to extract unique types
        unique_count = len(set(candyType))
        total_allowed = len(candyType) // 2

        return min(unique_count, total_allowed)
```

33.4 Python-Specific Complexities & Implementation Considerations

33.4.1 1. Hash Set vs. In-Place Sorting Trade-Off

- **Hash Set approach (as shown above)**: Runs in $\mathcal{O}(N)$ time and $\mathcal{O}(N)$ space.
- **In-Place Sorting approach**: We can sort the array `candyType.sort()`, and then scan the array to count unique adjacent elements. This runs in $\mathcal{O}(N \log N)$ time and $\mathcal{O}(1)$ auxiliary space.
- In memory-constrained environments, the sorting approach is preferred because it avoids heap allocations for set nodes.

33.4.2 2. CPython Optimization

- The Python function `set(candyType)` is implemented at the C level in CPython. It is highly optimized and constructs the set much faster than executing an explicit loop in Python bytecode.

33.5 Complexity Analysis

33.5.1 Hash Set Approach (Primary)

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N)$	We iterate through the array of size N once to build the hash set.
Space Complexity	$\mathcal{O}(N)$	In the worst case (all candies are unique), the hash set stores N elements.

33.6 Common Follow-Up Questions & How to Answer

33.6.1 Q1: What if the memory is extremely constrained? How do you solve this in $\mathcal{O}(1)$ auxiliary space?

- **Answer**: Sort the array in-place, then count the transitions where `candyType[i] != candyType[i-1]`.

```python
class Solution:
    def distributeCandies(self, candyType: List[int]) -> int:
        candyType.sort()
        unique_count = 1
        limit = len(candyType) // 2

        for i in range(1, len(candyType)):
            if unique_count >= limit:
                break
            if candyType[i] != candyType[i-1]:
                unique_count += 1

        return min(unique_count, limit)
```

33.6.2 Q2: How does this problem relate to the greedy LeetCode 135: Candy problem?

- **Answer**:
  - **LeetCode 575**: A simple counting/classification problem bounded by set theory.
  - **LeetCode 135 (Candy)**: A much harder greedy optimization problem. You must distribute candies to children in a row based on neighboring ratings. It requires a two-pass linear scan (left-to-right and right-to-left) to compute the minimum total candies needed.

33.7 Pro-Tip: How to Impress the Interviewer

- **Pigeonhole Principle Formulation**: Proactively explain the mathematical ceiling of the problem:

$$\text{Result} = \min(\text{Cardinality of Candy Types}, \text{Half of Candy Count})$$

  Formulating the problem using set cardinalities and bounds shows structured mathematical thinking.
- **Discuss Cache Inefficiency of Sets**: Mention that in C++, `std::unordered_set` is a bucket-based hash table that incurs significant pointer chasing and memory allocation overhead. For small datasets (e.g. $N \leq 64$), sorting the vector using `std::sort` is often faster due to hardware cache prefetching and contiguous memory layouts.

34 Binary Tree Level Order Traversal

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 102, Glassdoor
- **Flashcards**: BFS deck

34.1 Question Description

Given the `root` of a binary tree, return the level order traversal of its nodes' values (i.e., from left to right, level by level).

34.1.1 Examples

- **Input**: `root = [3,9,20,null,null,15,7]`
  - **Output**: `[[3],[9,20],[15,7]]`
- **Input**: `root = [1]`
  - **Output**: `[[1]]`
- **Input**: `root = []`
  - **Output**: `[]`

34.2 Detailed Solution (C++)

The core algorithm uses **Breadth-First Search (BFS)** with a queue. To separate the tree nodes level-by-level, we capture the queue's size at the beginning of each level iteration (`level_size`). This allows us to process exactly that number of nodes for the current level before moving to the next.

34.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <queue>
```

```cpp
#include <cstddef>
#include <utility>

// Definition for a binary tree node.
#ifdef LOCAL_TESTING
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};
#endif

class Solution {
public:
    std::vector<std::vector<int>> levelOrder(TreeNode* root) {
        std::vector<std::vector<int>> result;
        if (!root) {
            return result;
        }

        std::queue<TreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int level_size = static_cast<int>(q.size());
            std::vector<int> level;
            level.reserve(level_size); // Pre-allocate memory to avoid reallocations

            for (int i = 0; i < level_size; ++i) {
                TreeNode* node = q.front();
                q.pop();

                level.push_back(node->val);

                if (node->left) {
                    q.push(node->left);
                }
                if (node->right) {
                    q.push(node->right);
                }
            }
```

```
            result.push_back(std::move(level)); // Optimize via move semantics
        }

        return result;
    }
};
```

34.3 Detailed Solution (Python)

34.3.1 Standard Python Production Code

Below is the iterative BFS implementation utilizing `collections.deque` for efficient queue operations.

```python
from typing import List, Optional
from collections import deque

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        """
        Performs a level-order traversal (BFS) of a binary tree.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        if not root:
            return []

        result: List[List[int]] = []
        queue = deque([root])

        while queue:
            level_size = len(queue)
            level = []

            for _ in range(level_size):
                node = queue.popleft()
                level.append(node.val)
```

```
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(level)

    return result
```

34.4 Python-Specific Complexities & Implementation Considerations

34.4.1 1. Deque vs. List for Queue Operations

- In Python, using a standard `list` as a queue (e.g., `list.pop(0)`) is an $\mathcal{O}(n)$ operation because all subsequent elements must be shifted left in contiguous memory.
- Always use `collections.deque` (double-ended queue) which provides $\mathcal{O}(1)$ time complexity for both `append()` and `popleft()`. It is implemented as a doubly-linked list of fixed-size blocks, maximizing memory locality while avoiding shifts.

34.4.2 2. Generator/Yield Pattern

- For massive tree structures, instead of accumulating the full list in memory, we can yield levels lazily using Python generators:

```
def levelOrderGenerator(self, root: Optional[TreeNode]):
    if not root:
        return
    queue = deque([root])
    while queue:
        level_size = len(queue)
        yield [queue[i].val for i in range(level_size)]
        for _ in range(level_size):
            node = queue.popleft()
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
```

This reduces the active memory footprint if the consumer only processes one level at a time.

34.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Every node is visited and queued exactly once.
Space Complexity	$\mathcal{O}(n)$	The maximum space is occupied by the queue at the deepest level (leaf level), which can contain up to $\lceil n/2 \rceil$ nodes for a full binary tree.

34.6 Common Follow-Up Questions & How to Answer

34.6.1 Q1: How can you implement level-order traversal using DFS?

- **The Approach**: We can perform standard pre-order DFS and pass a depth parameter. We maintain a list of lists. If the current `depth` equals the size of the result list, we create a new sublist. We then append the node's value to the sublist corresponding to the current depth.
- **Python Code**:

```python
def levelOrderDFS(root: Optional[TreeNode]) -> List[List[int]]:
    res = []
    def dfs(node: Optional[TreeNode], depth: int):
        if not node:
            return
        if depth == len(res):
            res.append([])
        res[depth].append(node.val)
        dfs(node.left, depth + 1)
        dfs(node.right, depth + 1)
    dfs(root, 0)
    return res
```

- **Trade-off**: Time complexity remains $\mathcal{O}(n)$. Auxiliary space complexity drops to $\mathcal{O}(h)$ where h is the tree height (call stack size), which is better than BFS for balanced trees ($\mathcal{O}(\log n)$ vs $\mathcal{O}(n)$), but worse for extremely skewed trees ($\mathcal{O}(n)$).

34.6.2 Q2: How can we implement BFS without the memory allocation overhead of `std::queue` in C++?

- **The Approach**: We can use two flat vectors representing the `current_level` and the `next_level`.
- **Code Example**:

```cpp
std::vector<std::vector<int>> levelOrderFlat(TreeNode* root) {
    std::vector<std::vector<int>> res;
    if (!root) return res;
```

```
        std::vector<TreeNode*> cur = {root};
        while (!cur.empty()) {
            std::vector<int> level_vals;
            std::vector<TreeNode*> next_level;
            level_vals.reserve(cur.size());
            for (TreeNode* node : cur) {
                level_vals.push_back(node->val);
                if (node->left) next_level.push_back(node->left);
                if (node->right) next_level.push_back(node->right);
            }
            res.push_back(std::move(level_vals));
            cur = std::move(next_level);
        }
        return res;
    }
```

- **Benefits**: Avoids the dynamic node allocation overhead of standard queue/deque, resulting in contiguity and superior cache prefetching.

34.7 Pro-Tip: How to Impress the Interviewer

- **Move Semantics**: In C++, explain how `std::move(level)` transfers ownership of the vector's internal memory buffer to `result` instead of performing a deep copy. This ensures $\mathcal{O}(1)$ transfer complexity.
- **Contiguous Double-Buffering**: Mention that you are aware of hardware prefetching and CPU L1/L2 cache lines. A standard `std::queue` (which relies on `std::deque` allocations in chunks) has worse cache hit rates than a flat double-buffer vector approach because vectors guarantee contiguous memory layout.
- **Pre-sizing Vectors**: Explicitly calling `level.reserve(level_size)` prevents the dynamic array reallocation pattern (typically doubling capacity: 1, 2, 4, 8, 16...) which can trigger multiple memory copy passes.

35 Shortest Path in Binary Matrix

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 1091, Glassdoor
- **Flashcards**: BFS deck

35.1 Question Description

Given an $n \times n$ binary matrix `grid`, return the length of the shortest clear path in the matrix. If there is no clear path, return `-1`.

A **clear path** in a binary matrix is a path from the top-left cell (i.e., `(0, 0)`) to the bottom-right cell (i.e., `(n - 1, n - 1)`) such that: * All the visited cells of the path are `0` (walkable). * All the adjacent cells of the path are **8-directionally** connected (i.e., they share an edge or a corner). * The length of a clear path is the number of visited cells of this path.

35.1.1 Examples

- **Input**: `grid = [[0,1],[1,0]]`
  - **Output**: `2`
- **Input**: `grid = [[0,0,0],[1,1,0],[1,1,0]]`
  - **Output**: `4`
- **Input**: `grid = [[1,0,0],[1,1,0],[1,1,0]]`
  - **Output**: `-1`

35.2 Detailed Solution (C++)

The problem asks for the shortest path in an unweighted grid with 8-directional movement. The optimal algorithm for finding the shortest path in an unweighted graph is **Breadth-First Search (BFS)**.

To avoid the memory and time overhead of maintaining a separate `visited` matrix, we can mark cells as visited **in-place** by overwriting `grid[r][c] = 1`.

35.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <queue>
#include <tuple>
#include <cstddef>

class Solution {
public:
    int shortestPathBinaryMatrix(std::vector<std::vector<int>>& grid) {
        // Edge Case: Handle empty grid
        if (grid.empty() || grid[0].empty()) {
            return -1;
        }

        int n = static_cast<int>(grid.size());

        // Edge Case: If start or end cell is blocked
        if (grid[0][0] == 1 || grid[n - 1][n - 1] == 1) {
```

```
            return -1;
        }

        // Edge Case: 1x1 matrix
        if (n == 1) {
            return 1;
        }

        // Queue storing: {row, col, distance}
        std::queue<std::tuple<int, int, int>> q;
        q.push({0, 0, 1});
        grid[0][0] = 1; // Mark start as visited in-place

        // 8-directional arrays
        const int dr[] = {-1, -1, -1, 0, 0, 1, 1, 1};
        const int dc[] = {-1, 0, 1, -1, 1, -1, 0, 1};

        while (!q.empty()) {
            auto [r, c, dist] = q.front();
            q.pop();

            for (int d = 0; d < 8; ++d) {
                int nr = r + dr[d];
                int nc = c + dc[d];

                // Boundary and validity check
                if (nr >= 0 && nr < n && nc >= 0 && nc < n && grid[nr][nc] == 0) {
                    // Check if we reached the bottom-right corner
                    if (nr == n - 1 && nc == n - 1) {
                        return dist + 1;
                    }

                    // Mark visited immediately when pushing to prevent redundant queue entries
                    grid[nr][nc] = 1;
                    q.push({nr, nc, dist + 1});
                }
            }
        }

        return -1;
    }
};
```

35.3 Detailed Solution (Python)

35.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using `collections.deque` for the BFS queue.

```python
from typing import List
from collections import deque

class Solution:
    def shortestPathBinaryMatrix(self, grid: List[List[int]]) -> int:
        """
        Finds the shortest clear path in an N x N binary matrix.
        Uses BFS for shortest path in an unweighted grid with 8-directional movements.

        Time Complexity: O(N^2)
        Space Complexity: O(N^2) (or O(1) auxiliary if in-place modification is permitted)
        """
        if not grid or not grid[0]:
            return -1

        n = len(grid)

        # Edge Case: Start or destination cell is blocked
        if grid[0][0] == 1 or grid[n - 1][n - 1] == 1:
            return -1

        # Edge Case: 1x1 grid
        if n == 1:
            return 1

        # 8-directional offsets
        directions = [
            (-1, -1), (-1, 0), (-1, 1),
            (0, -1),           (0, 1),
            (1, -1),  (1, 0),  (1, 1)
        ]

        # Queue stores: (row, col, distance)
        queue = deque([(0, 0, 1)])
        grid[0][0] = 1  # Mark as visited in-place

        while queue:
            r, c, dist = queue.popleft()
```

```
        for dr, dc in directions:
            nr, nc = r + dr, c + dc

            if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0:
                if nr == n - 1 and nc == n - 1:
                    return dist + 1

                # Mark visited immediately before pushing to queue
                grid[nr][nc] = 1
                queue.append((nr, nc, dist + 1))

    return -1
```

35.4 Python-Specific Complexities & Implementation Considerations

35.4.1 1. BFS Queue Performance Gotcha

- Do not use Python's built-in `queue.Queue` from the standard library for coding interviews. `queue.Queue` is designed for multi-threaded systems and uses locking primitives (`threading.Lock`), which adds significant CPU thread overhead.
- Always use `collections.deque` for single-threaded queue needs.

35.4.2 2. Mutating Inputs vs. Functional Purity

- Overwriting `grid[r][c] = 1` avoids the $\mathcal{O}(n^2)$ space cost of a separate `visited` set or matrix.
- However, in production environments, mutating input arguments is often considered a side-effect that can lead to concurrency bugs (e.g., if other threads are reading the grid). If input mutation is not allowed, instantiate a `visited` matrix/set at the cost of $\mathcal{O}(n^2)$ space:

  ```
  visited = [[False] * n for _ in range(n)]
  ```

35.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n^2)$	In the worst-case, we check all $n \times n$ cells of the grid.
Space Complexity	$\mathcal{O}(n^2)$	The queue size can grow up to $\mathcal{O}(n^2)$ elements in worst-case scenarios. Auxiliary space is $\mathcal{O}(1)$ since we modify the grid in-place.

35.6 Common Follow-Up Questions & How to Answer

35.6.1 Q1: What if the grid is extremely large (e.g., $10^5 \times 10^5$) and cannot fit in memory?

- **The Problem**: A standard BFS will run out of memory because the size of the BFS queue (frontier) grows linearly with the perimeter of the search wavefront.
- **The Solution**: Use **A* search** with a heuristic function like **Chebyshev distance** (since 8-directional movement is permitted).

$$\text{Chebyshev}(A, B) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

A* will prune the search space by steering the search towards the target. If memory is still a constraint, **IDA* (Iterative Deepening A*)** or **Bidirectional BFS** can be used.

35.6.2 Q2: Why must we mark cells as visited *when pushing* rather than *when popping*?

- **The Core Bug**: If we only mark cells as visited when they are popped, multiple nodes in the current BFS layer can look at the same neighbor and push it into the queue. This leads to duplicate processing, exponential queue growth, and results in Time/Memory Limit Exceeded errors.
- **Correction**: Always mark the cell visited *immediately* before or during queue pushing.

35.6.3 Q3: How does Bidirectional BFS optimize search time?

- **The Concept**: Run two simultaneous BFS traversals—one starting at the source node, and the other starting at the target node. We alternate expanding layers between both queues.
- **Benefit**: If the shortest path is of length d, a single BFS search tree expands to $O(b^d)$ nodes (where b is the average branching factor). Bidirectional BFS will meet in the middle, expanding only $O(b^{d/2} + b^{d/2}) = O(b^{d/2})$ nodes. This yields an exponential reduction in state space exploration.

35.7 Pro-Tip: How to Impress the Interviewer

- **Mention CPU Branch Predictor and 8-Direction Iteration**: In C++, iterate through directions using static arrays (`dr` and `dc`) rather than conditional `if` statements. This reduces branching and lets the compiler optimize the loop using vectorization (SIMD) or loop unrolling.
- **Chebyshev vs. Manhattan Distance**: Point out that since diagonal moves are allowed, the appropriate heuristic is Chebyshev distance. If diagonal moves were forbidden, Manhattan distance ($|x_1-x_2|+|y_1-y_2|$) would be correct. Showing this theoretical understanding proves your pathfinding expertise.

36 Serialize and Deserialize BST

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 449, Glassdoor

- **Flashcards**: BFS deck

36.1 Question Description

Serialization is converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a **binary search tree (BST)**. There is no restriction on how your serialization/deserialization algorithm should work. You need to ensure that a binary search tree can be serialized to a string, and this string can be deserialized to the original tree structure.

The encoded string should be **as compact as possible**.

36.1.1 Examples

- **Input**: `root = [2,1,3]`
  - **Output**: `[2,1,3]`
- **Input**: `root = []`
  - **Output**: `[]`

36.2 Detailed Solution (C++)

A key property of a **Binary Search Tree (BST)** is that its preorder traversal carries enough structural information to uniquely reconstruct the tree *without* storing `null` pointers. This is because we can deduce where children belong using mathematical BST boundaries (lower and upper bounds).

This results in the most compact representation possible: a simple, space-separated string of numbers.

36.2.1 Standard C++ Production Code

```cpp
#include <string>
#include <sstream>
#include <vector>
#include <climits>

// Definition for a binary tree node.
#ifdef LOCAL_TESTING
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};
#endif
```

```
class Codec {
public:
    // Encodes a tree to a single string.
    std::string serialize(TreeNode* root) {
        std::string result;
        serializeHelper(root, result);
        if (!result.empty()) {
            result.pop_back(); // Remove trailing space
        }
        return result;
    }

    // Decodes your encoded data to tree.
    TreeNode* deserialize(std::string data) {
        if (data.empty()) {
            return nullptr;
        }

        std::vector<int> vals;
        std::stringstream ss(data);
        std::string token;
        while (ss >> token) {
            vals.push_back(std::stoi(token));
        }

        int index = 0;
        // Use LLONG_MIN/LLONG_MAX to prevent overflow boundaries
        return deserializeHelper(vals, index, LLONG_MIN, LLONG_MAX);
    }

private:
    // Helper to perform pre-order traversal
    void serializeHelper(TreeNode* root, std::string& result) {
        if (!root) {
            return;
        }
        result += std::to_string(root->val) + " ";
        serializeHelper(root->left, result);
        serializeHelper(root->right, result);
    }

    // Helper to reconstruct BST using range boundaries
    TreeNode* deserializeHelper(const std::vector<int>& vals, int& index, long long lower, long long upper)
```

```
        if (index >= static_cast<int>(vals.size())) {
            return nullptr;
        }

        long long val = vals[index];
        // If the current value violates the BST boundaries for this subtree,
        // it belongs to a different subtree. Return nullptr.
        if (val < lower || val > upper) {
            return nullptr;
        }

        TreeNode* root = new TreeNode(static_cast<int>(val));
        index++; // Increment pointer to consume the value

        root->left = deserializeHelper(vals, index, lower, val);
        root->right = deserializeHelper(vals, index, val, upper);

        return root;
    }
};

// Your Codec object will be instantiated and called as such:
// Codec* ser = new Codec();
// Codec* deser = new Codec();
// std::string tree = ser->serialize(root);
// TreeNode* ans = deser->deserialize(tree);
// return ans;
```

36.3 Detailed Solution (Python)

36.3.1 Standard Python Production Code

Below is the recursive pre-order Python implementation, using a deque to efficiently consume deserialized values.

```
from typing import Optional
from collections import deque

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None
```

```python
class Codec:
    def serialize(self, root: Optional[TreeNode]) -> str:
        """
        Encodes a BST to a single string using Pre-order traversal.
        Completely omits 'null' markers to ensure it is as compact as possible.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        vals = []

        def pre_order(node: Optional[TreeNode]):
            if not node:
                return
            vals.append(str(node.val))
            pre_order(node.left)
            pre_order(node.right)

        pre_order(root)
        return " ".join(vals)

    def deserialize(self, data: str) -> Optional[TreeNode]:
        """
        Decodes your encoded data to tree.
        Implicitly discovers where 'null' pointers go using mathematical BST bounds.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        if not data:
            return None

        # Convert split strings directly into a queue of integers
        queue = deque(int(x) for x in data.split())

        def build(lower: float, upper: float) -> Optional[TreeNode]:
            if not queue:
                return None

            next_val = queue[0]
            # If the next value does not fit within the BST boundaries,
            # it belongs to another sub-tree (representing a None pointer here).
            if next_val < lower or next_val > upper:
```

```
                return None

            queue.popleft() # Consume the value
            root = TreeNode(next_val)

            root.left = build(lower, next_val)
            root.right = build(next_val, upper)

            return root

        return build(float('-inf'), float('inf'))

# Your Codec object will be instantiated and called as such:
# ser = Codec()
# deser = Codec()
# tree = ser.serialize(root)
# ans = deser.deserialize(tree)
# return ans
```

36.4 Python-Specific Complexities & Implementation Considerations

36.4.1 1. Handling Tree Nulls via Queue vs. Stack Indexing

- In Python, popping elements from the left of a list takes $\mathcal{O}(n)$ time. By wrapping our list in `collections.deque`, we ensure that `popleft()` takes $\mathcal{O}(1)$ time.
- An alternative approach to avoid queue overhead is to pass an index integer wrapped in a list (e.g., `idx = [0]`) to maintain mutable state across recursion levels:

  ```
  def build(lower, upper):
      if idx[0] >= len(vals): return None
      # ... logic ...
      idx[0] += 1
  ```

  Both solutions achieve $\mathcal{O}(n)$ time complexity, but the deque approach is cleaner and more idiomatic.

36.4.2 2. String Concatenation vs. List Join

- Using `+` to concatenate strings inside recursive loops (e.g., `data += str(node.val) + " "`) creates new string instances every time, leading to an $\mathcal{O}(n^2)$ time penalty due to immutable string copying.
- Always collect values in a `list` and use `" ".join(list)` at the end to assemble the string in $\mathcal{O}(n)$ time.

36.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Both serialization and deserialization visit each tree node exactly once.
Space Complexity	$\mathcal{O}(n)$	Preorder serialization list and recursion call stack for both operations.

36.6 Common Follow-Up Questions & How to Answer

36.6.1 Q1: Can we use this exact algorithm for general binary trees?

- **No**. A general binary tree does not satisfy the BST range property ($left < root < right$).
- To serialize a general binary tree, we must store structural metadata:
  1. Include `null` markers (e.g. `1 2 null null 3`).
  2. Provide two traversal lists (e.g., preorder + inorder, or postorder + inorder), provided all node values are guaranteed to be unique.

36.6.2 Q2: How can we serialize the tree into a binary format instead of a string to save more space?

- **The Idea**: String representation takes a variable amount of bytes per number (e.g., `12345` takes 5 bytes for ASCII + 1 byte for space).
- **Solution**: Write the integers as raw binary data. In C++, write the bytes of the integers directly to a char buffer:

```
void serializeBinary(TreeNode* root, std::string& buffer) {
    if (!root) return;
    buffer.append(reinterpret_cast<const char*>(&root->val), sizeof(root->val));
    serializeBinary(root->left, buffer);
    serializeBinary(root->right, buffer);
}
```

  This guarantees exactly 4 bytes per node, eliminating string conversion overhead and space separators.

36.6.3 Q3: How do we deserialize a BST using post-order serialization?

- **The Approach**: Post-order visits nodes in the order: `left`, `right`, `root`. The root node is at the very end of the list.
- **Solution**: We can deserialize post-order by reading the list **backward**. We parse the right child first, then the left child:

```
def deserialize(self, data: str) -> Optional[TreeNode]:
    vals = [int(x) for x in data.split()]
```

```
    def build(lower, upper):
        if not vals:
            return None
        val = vals[-1]
        if val < lower or val > upper:
            return None
        vals.pop()
        root = TreeNode(val)
        root.right = build(val, upper)
        root.left = build(lower, val)
        return root
    return build(float('-inf'), float('inf'))
```

36.7 Pro-Tip: How to Impress the Interviewer

- **Highlight Null Marker Optimization**: Start by noting that standard binary tree serialization requires storing `null` nodes (which makes up roughly half of the data in a full binary tree). Mention that because this is a BST, we can omit `null` entries entirely. This demonstrates immediate optimization intuition.
- **Discuss Varint Encoding**: Mention that if the values in the BST are usually small (e.g., within $[0, 127]$), we can use **Varint** (Variable-length Quantity) encoding (like Protocol Buffers). This allows representing small numbers with a single byte, while larger ones take up to 4 or 5 bytes.

37 Island Perimeter

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 463, Glassdoor
- **Flashcards**: BFS deck

37.1 Question Description

You are given `row x col grid` representing a map where `grid[i][j] = 1` represents land and `grid[i][j] = 0` represents water.

Grid cells are connected horizontally/vertically (not diagonally). The grid is completely surrounded by water, and there is exactly one island (i.e., one or more connected land cells).

The island doesn't have ”lakes”, meaning the water inside isn't connected to the water around the island. One cell is a square with side length 1. The grid is rectangular, width and height don't exceed 100. Determine the perimeter of the island.

37.1.1 Examples

- **Input**: `grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]`
  - **Output**: `16`
- **Input**: `grid = [[1]]`
  - **Output**: `4`
- **Input**: `grid = [[1,0]]`
  - **Output**: `4`

37.2 Detailed Solution (C++)

There are two primary ways to solve this problem: 1. **Graph Traversal (BFS/DFS)**: Locate the single island, perform a traversal, and sum the outer edges (boundaries with water or grid edges). 2. **Mathematical Deduction ($\mathcal{O}(1)$ Space)**: Iterate through the grid. Every land cell initially contributes `4` to the perimeter. If it has a neighbor directly to its right or bottom that is also land, they share an edge, so we subtract `2` from the total perimeter (1 contribution for each cell).

37.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <queue>
#include <utility>

class Solution {
public:
    // BFS Solution: O(R * C) Time and O(R * C) Space
    int islandPerimeterBFS(std::vector<std::vector<int>>& grid) {
        int rows = static_cast<int>(grid.size());
        int cols = static_cast<int>(grid[0].size());
        std::vector<std::vector<bool>> visited(rows, std::vector<bool>(cols, false));
        int perimeter = 0;

        const int dr[] = {0, 0, 1, -1};
        const int dc[] = {1, -1, 0, 0};

        for (int r = 0; r < rows; ++r) {
            for (int c = 0; c < cols; ++c) {
                if (grid[r][c] == 1) {
                    std::queue<std::pair<int, int>> q;
                    q.push({r, c});
                    visited[r][c] = true;

                    while (!q.empty()) {
                        auto [cr, cc] = q.front();
```

```
                    q.pop();

                    for (int d = 0; d < 4; ++d) {
                        int nr = cr + dr[d];
                        int nc = cc + dc[d];

                        // If neighbor is out of bounds or water, it's a boundary edge
                        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || grid[nr][nc] == 0) {
                            perimeter++;
                        } else if (!visited[nr][nc]) {
                            visited[nr][nc] = true;
                            q.push({nr, nc});
                        }
                    }
                }
                return perimeter; // Since there is exactly one island, we are done
            }
        }
    }
    return 0;
}

// Mathematical Deduction: O(R * C) Time and O(1) Space
int islandPerimeter(const std::vector<std::vector<int>>& grid) {
    int rows = static_cast<int>(grid.size());
    int cols = static_cast<int>(grid[0].size());
    int perimeter = 0;

    for (int r = 0; r < rows; ++r) {
        for (int c = 0; c < cols; ++c) {
            if (grid[r][c] == 1) {
                perimeter += 4;
                // If right neighbor is land, subtract 2 (shared boundary)
                if (c + 1 < cols && grid[r][c + 1] == 1) {
                    perimeter -= 2;
                }
                // If bottom neighbor is land, subtract 2 (shared boundary)
                if (r + 1 < rows && grid[r + 1][c] == 1) {
                    perimeter -= 2;
                }
            }
        }
    }
    return perimeter;
```

```
    }
};
```

37.3 Detailed Solution (Python)

37.3.1 Standard Python Production Code

Below are the two approaches implemented in Python.

```python
from typing import List
from collections import deque

class Solution:
    def islandPerimeterBFS(self, grid: List[List[int]]) -> int:
        """
        Calculates the perimeter of the island using BFS.

        Time Complexity: O(R * C)
        Space Complexity: O(R * C)
        """
        rows, cols = len(grid), len(grid[0])
        visited = set()
        perimeter = 0
        directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    queue = deque([(r, c)])
                    visited.add((r, c))

                    while queue:
                        cr, cc = queue.popleft()
                        for dr, dc in directions:
                            nr, nc = cr + dr, cc + dc
                            if nr < 0 or nr >= rows or nc < 0 or nc >= cols or grid[nr][nc] == 0:
                                perimeter += 1
                            elif (nr, nc) not in visited:
                                visited.add((nr, nc))
                                queue.append((nr, nc))
                    return perimeter
        return 0

    def islandPerimeter(self, grid: List[List[int]]) -> int:
```

```
    """
    Calculates the perimeter using O(1) space deduction.

    Time Complexity: O(R * C)
    Space Complexity: O(1)
    """
    perimeter = 0
    rows, cols = len(grid), len(grid[0])

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                perimeter += 4
                # If right neighbor is land, subtract 2 (for the shared edge)
                if c + 1 < cols and grid[r][c + 1] == 1:
                    perimeter -= 2
                # If bottom neighbor is land, subtract 2 (for the shared edge)
                if r + 1 < rows and grid[r + 1][c] == 1:
                    perimeter -= 2
    return perimeter
```

37.4 Python-Specific Complexities & Implementation Considerations

37.4.1 1. Set Lookup vs. In-place Mutation

- In the BFS approach, we tracked visited cells using a set (`visited`). While set operations (`in`, `add`) are average-case $\mathcal{O}(1)$, hash collisions can degrade this, and storing tuples `(r, c)` in a set incurs high object allocation overhead in Python.
- To achieve the absolute fastest execution speed, perform in-place mutation (e.g., mark visited cells as `2`) and restore them back to `1` afterward if the grid needs to remain uncorrupted.

37.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(rows \times cols)$	We scan all cells in the grid. For the mathematical approach, we do a single sweep.

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(1)$	For the mathematical deduction approach, only primitive loop counter variables are used. For BFS, it requires $\mathcal{O}(rows \times cols)$ space for the queue and visited tracking.

37.6 Common Follow-Up Questions & How to Answer

37.6.1 Q1: What if there are multiple disconnected islands, and we want the perimeter of all of them?

- **The Answer**: The mathematical deduction approach is completely local; it does not assume connectivity. It naturally works for any number of disconnected islands, return the exact sum of all island perimeters in $\mathcal{O}(1)$ auxiliary space.

37.6.2 Q2: What if the land cells are represented as a list of coordinates (sparse grid) rather than a 2D matrix?

- **The Solution**: Storing a giant matrix is memory-inefficient for sparse islands. If we have a list of coordinate pairs:
  1. Insert all coordinates into a hash set for $\mathcal{O}(1)$ lookup.
  2. For each coordinate (r, c), check its 4 orthogonal neighbors.
  3. If a neighbor is NOT in the hash set, increment the perimeter count by 1.
- **Python Snippet**:

```
land_set = set(coordinates)
perimeter = 0
for r, c in land_set:
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        if (r + dr, c + dc) not in land_set:
            perimeter += 1
```

- **Complexity**: $\mathcal{O}(L)$ time and $\mathcal{O}(L)$ space, where L is the number of land cells. This is highly optimal for sparse grids where $L \ll rows \times cols$.

37.7 Pro-Tip: How to Impress the Interviewer

- **Highlight the Cache-Friendliness of Row-Major Scans**: In the mathematical deduction approach, the grid is processed in row-major order (`grid[r][c]`). In C++, this aligns perfectly with how 2D arrays are stored contiguously in memory. Scanning row-by-row ensures maximum L1 cache hit

rate because adjacent values are pre-fetched by the CPU cache controller. Scanning column-major instead would result in L1 cache misses.

- **Prevent Double Counting**: Clearly explain the logic behind subtracting `2` for each shared edge: a shared edge between cell A and cell B reduces the perimeter of cell A by 1, and the perimeter of cell B by 1. Checking only right and bottom neighbors guarantees that every shared edge is visited exactly once, avoiding duplicate substractions.

38 Find Bottom Left Tree Value

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 513, Glassdoor
- **Flashcards**: BFS deck

38.1 Question Description

Given the `root` of a binary tree, return the leftmost value in the last row of the tree.

38.1.1 Examples

- **Input**: `root = [2,1,3]`
  - **Output**: `1`
- **Input**: `root = [1,2,3,4,null,5,6,null,null,7]`
  - **Output**: `7`

38.2 Detailed Solution (C++)

While standard BFS level-by-level traversal works, a highly optimized approach is to perform a **Right-to-Left BFS**.

If we traverse the tree level by level starting from the **right** instead of the left (i.e., we push the `right` child before the `left` child to the queue), then the last node popped from the queue is guaranteed to be the leftmost node of the deepest level. This elegant trick simplifies the logic to a single loop, eliminating the need to track level sizes.

38.2.1 Standard C++ Production Code

```cpp
#include <queue>

// Definition for a binary tree node.
#ifdef LOCAL_TESTING
struct TreeNode {
    int val;
```

```
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};
#endif

class Solution {
public:
    int findBottomLeftValue(TreeNode* root) {
        // Guaranteed that root is not null by LeetCode constraints (1 <= nodes <= 10^4)
        std::queue<TreeNode*> q;
        q.push(root);
        TreeNode* node = nullptr;

        while (!q.empty()) {
            node = q.front();
            q.pop();

            // Push right child first, then left child.
            // This ensures that the left-most node of the lowest level is popped last.
            if (node->right) {
                q.push(node->right);
            }
            if (node->left) {
                q.push(node->left);
            }
        }

        return node->val;
    }
};
```

38.3 Detailed Solution (Python)

38.3.1 Standard Python Production Code

Below is the recursive/iterative Python implementation of the Right-to-Left BFS.

```
from typing import Optional
from collections import deque

# Definition for a binary tree node.
```

```python
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findBottomLeftValue(self, root: TreeNode) -> int:
        """
        Finds the leftmost value in the last row of the tree.
        Uses a right-to-left BFS traversal. The last popped node is the answer.

        Time Complexity: O(N)
        Space Complexity: O(W) where W is the maximum width of the tree
        """
        # Root is guaranteed to be non-null
        queue = deque([root])
        node = root

        while queue:
            node = queue.popleft()

            # Queue right first, then left.
            if node.right:
                queue.append(node.right)
            if node.left:
                queue.append(node.left)

        return node.val
```

38.4 Python-Specific Complexities & Implementation Considerations

38.4.1 1. BFS vs. DFS Recursion Stack Limit

- A DFS implementation uses the Python call stack, which has a default recursion limit of 1000. If the tree is highly skewed (e.g., 10^4 nodes aligned in a single line), a DFS solution will raise a `RecursionError`.
- The BFS solution runs iteratively and does not create call stack frames. The space is allocated on the heap inside `collections.deque`, which is bounded only by the machine's physical memory.

38.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Every node in the binary tree is visited exactly once.
Space Complexity	$\mathcal{O}(w)$	The queue holds at most $\mathcal{O}(w)$ elements, where w is the maximum width of the tree. In the worst-case of a complete binary tree, $w \approx n/2$.

38.6 Common Follow-Up Questions & How to Answer

38.6.1 Q1: How can you implement this using DFS?

- **The Approach**: Use a pre-order traversal (which visits the left child before the right child). Keep track of both the maximum depth seen so far (`max_depth`) and the value of the leftmost node at that depth. Because we traverse left-to-right, the first node encountered at any new depth is guaranteed to be the leftmost node for that depth level.
- **Python Code**:

```python
def findBottomLeftValueDFS(root: TreeNode) -> int:
    max_depth = -1
    leftmost_val = root.val

    def dfs(node: Optional[TreeNode], depth: int):
        nonlocal max_depth, leftmost_val
        if not node:
            return

        if depth > max_depth:
            max_depth = depth
            leftmost_val = node.val

        dfs(node.left, depth + 1)
        dfs(node.right, depth + 1)

    dfs(root, 0)
    return leftmost_val
```

38.6.2 Q2: Which is better, BFS or DFS, for a very wide tree vs. a very deep tree?

- **Wide Tree (large width w, small height h)**: DFS is superior because DFS space complexity is $\mathcal{O}(h)$ (small stack), while BFS space complexity is $\mathcal{O}(w)$ (large queue).

- **Deep Tree (small width w, large height h)**: BFS is superior because BFS space complexity is $\mathcal{O}(w)$ (small queue) and avoids stack overflow risks, while DFS space complexity is $\mathcal{O}(h)$ (large stack).

38.7 Pro-Tip: How to Impress the Interviewer

- **Offer Right-to-Left BFS Immediately**: Many candidates implement standard level-by-level BFS because they memorize it. Proposing the Right-to-Left BFS immediately signals a deep, intuitive understanding of queue ordering properties rather than simple rote memorization.
- **Explain the Queue Invariant**: Explain *why* the Right-to-Left BFS works: "At any point in a BFS queue, the elements represent at most two adjacent levels, sorted from right to left. Therefore, the very last element in the queue must be the leftmost node of the deepest level."

39 Find Largest Value in Each Tree Row

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 515, Glassdoor
- **Flashcards**: BFS deck

39.1 Question Description

Given the `root` of a binary tree, return an array of the largest value in each row of the tree (0-indexed).

39.1.1 Examples

- **Input**: `root = [1,3,2,5,3,null,9]`
  - **Output**: `[1,3,9]`
- **Input**: `root = [1,2,3]`
  - **Output**: `[1,3]`

39.2 Detailed Solution (C++)

The standard solution uses **Breadth-First Search (BFS)**. We traverse the tree level by level. For each level, we find the maximum node value by iterating through the current queue elements, and then push its children to the queue for the next level.

39.2.1 Standard C++ Production Code

```
#include <vector>
#include <queue>
#include <climits>
```

```cpp
#include <algorithm>

// Definition for a binary tree node.
#ifdef LOCAL_TESTING
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode() : val(0), left(nullptr), right(nullptr) {}
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
    TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
};
#endif

class Solution {
public:
    std::vector<int> largestValues(TreeNode* root) {
        std::vector<int> result;
        if (!root) {
            return result;
        }

        std::queue<TreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int level_size = static_cast<int>(q.size());
            int max_val = INT_MIN; // Initialize with the lowest possible signed 32-bit integer

            for (int i = 0; i < level_size; ++i) {
                TreeNode* node = q.front();
                q.pop();

                max_val = std::max(max_val, node->val);

                if (node->left) {
                    q.push(node->left);
                }
                if (node->right) {
                    q.push(node->right);
                }
            }
            result.push_back(max_val);
        }
```

```
        return result;
    }
};
```

39.3 Detailed Solution (Python)

39.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using `collections.deque`.

```python
from typing import List, Optional
from collections import deque

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def largestValues(self, root: Optional[TreeNode]) -> List[int]:
        """
        Finds the largest value in each row of a binary tree.
        Uses BFS to process the tree level by level.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        if not root:
            return []

        result: List[int] = []
        queue = deque([root])

        while queue:
            level_size = len(queue)
            max_val = float('-inf')

            for _ in range(level_size):
                node = queue.popleft()

                # Check for new max value
```

```
            if node.val > max_val:
                max_val = node.val

            # Push children
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

        result.append(max_val)

    return result
```

39.4 Python-Specific Complexities & Implementation Considerations

39.4.1 1. Initializing Negative Infinity in Python

- In Python, do not use a hardcoded value like `-999999` to initialize a minimum value checker.
- Use `float('-inf')`, which is mathematically guaranteed to be smaller than any numeric value. Alternatively, initialize `max_val` with the first element of the level: `max_val = queue[0].val`.

39.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Every node is visited exactly once.
Space Complexity	$\mathcal{O}(n)$	The queue size peaks at the tree's maximum width w, which is at most $\lceil n/2 \rceil$ nodes (for a full binary tree).

39.6 Common Follow-Up Questions & How to Answer

39.6.1 Q1: How can you implement this using DFS?

- **The Approach**: We can perform a preorder DFS traversal while tracking the current level `depth`. If the depth is equal to the size of the results list, it means this is our first time reaching this row, so we append the current node's value. Otherwise, we update the existing value at index `depth` with the maximum of its current value and the new node's value.
- **Python Code**:

```python
def largestValuesDFS(root: Optional[TreeNode]) -> List[int]:
    result = []

    def dfs(node: Optional[TreeNode], depth: int):
        if not node:
            return

        if depth == len(result):
            result.append(node.val)
        else:
            result[depth] = max(result[depth], node.val)

        dfs(node.left, depth + 1)
        dfs(node.right, depth + 1)

    dfs(root, 0)
    return result
```

- **Complexity**: Time complexity is $\mathcal{O}(n)$. Space complexity is $\mathcal{O}(h)$ (height of the tree, due to recursion stack frames). This makes DFS more space-efficient for balanced trees.

39.6.2 Q2: How can we implement this without using `std::max` in C++ for branchless optimization?

- **The Approach**: We can use ternary operators, which compilers optimize into branchless conditional moves (`CMOV`).
- **C++ Snippet**:

```cpp
max_val = (node->val > max_val) ? node->val : max_val;
```

39.7 Pro-Tip: How to Impress the Interviewer

- **Mention INT_MIN Bound Safety**: Point out that you initialized the maximum checker with `INT_MIN` in C++ and `float('-inf')` in Python because binary tree node values can legally be negative (down to -2^{31}). Using `0` as an initial value would fail on trees consisting entirely of negative numbers.
- **Double-Buffering for High Performance**: Discuss that for real-time systems, avoiding dynamic memory allocations inside the `while` loop is crucial. Instead of standard `std::queue`, using two pre-allocated vectors (`std::vector<TreeNode*>`) and swapping them eliminates heap allocations during traversal.

40 Rotting Oranges

- **Category**: Coding
- **Difficulty**: Medium

- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 994, Glassdoor
- **Flashcards**: BFS deck

40.1 Question Description

You are given an `m x n grid` where each cell can have one of three values: * `0` representing an empty cell, * `1` representing a fresh orange, or * `2` representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return `-1`.

40.1.1 Examples

- **Input**: `grid = [[2,1,1],[1,1,0],[0,1,1]]`
  - **Output**: `4`
- **Input**: `grid = [[2,1,1],[0,1,1],[1,0,1]]`
  - **Output**: `-1`
  - **Explanation**: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.
- **Input**: `grid = [[0,2]]`
  - **Output**: `0`
  - **Explanation**: Since there are already no fresh oranges at minute 0, the answer is just 0.

40.2 Detailed Solution (C++)

This problem represents a classic propagation process on a grid. Because multiple rotten oranges can rot adjacent fresh oranges simultaneously, we must perform a **Multi-Source BFS**.

1. **Initialization**: Sweep the grid to:
   - Push all coordinates of initial rotten oranges (`2`) into the queue.
   - Count the total number of fresh oranges (`fresh`).
2. **Edge Case**: If there are no fresh oranges, return `0` immediately.
3. **Propagation**: Perform BFS. In each iteration, pop all nodes currently in the queue (representing one minute of rotting spread). If any fresh orange is turned rotten, push it to the queue, decrement `fresh`, and set a flag `rotted_any = true`.
4. **Time Accumulation**: Increment `minutes` only if `rotted_any` was true during the level traversal.
5. **Result**: If `fresh == 0`, return `minutes`; otherwise, return `-1`.

40.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <queue>
```

```cpp
#include <utility>

class Solution {
public:
    int orangesRotting(std::vector<std::vector<int>>& grid) {
        if (grid.empty() || grid[0].empty()) {
            return 0;
        }

        int rows = static_cast<int>(grid.size());
        int cols = static_cast<int>(grid[0].size());

        std::queue<std::pair<int, int>> q;
        int fresh = 0;

        // Initialize queue with all source nodes (initial rotten oranges)
        for (int r = 0; r < rows; ++r) {
            for (int c = 0; c < cols; ++c) {
                if (grid[r][c] == 2) {
                    q.push({r, c});
                } else if (grid[r][c] == 1) {
                    fresh++;
                }
            }
        }

        // If there are no fresh oranges to rot, 0 minutes are needed
        if (fresh == 0) {
            return 0;
        }

        int minutes = 0;
        const int dr[] = {0, 0, 1, -1};
        const int dc[] = {1, -1, 0, 0};

        while (!q.empty()) {
            int sz = static_cast<int>(q.size());
            bool rotted_any = false;

            for (int i = 0; i < sz; ++i) {
                auto [r, c] = q.front();
                q.pop();

                for (int d = 0; d < 4; ++d) {
```

```
                int nr = r + dr[d];
                int nc = c + dc[d];

                // Check bounds and if neighbor is a fresh orange
                if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && grid[nr][nc] == 1) {
                    grid[nr][nc] = 2; // Rot the fresh orange in-place
                    fresh--;
                    q.push({nr, nc});
                    rotted_any = true;
                }
            }
        }

        if (rotted_any) {
            minutes++;
        }
    }

    // If fresh is 0, all oranges have rotted; otherwise, some were unreachable
    return fresh == 0 ? minutes : -1;
  }
};
```

40.3 Detailed Solution (Python)

40.3.1 Standard Python Production Code

Below is the multi-source BFS implementation using `collections.deque`.

```
from typing import List
from collections import deque

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        """
        Finds the minimum time for all oranges to rot.
        Uses a multi-source BFS.

        Time Complexity: O(R * C)
        Space Complexity: O(R * C)
        """
        if not grid or not grid[0]:
            return 0
```

```
rows, cols = len(grid), len(grid[0])
queue = deque()
fresh = 0

# Step 1: Scan grid for rotten oranges (sources) and count fresh oranges
for r in range(rows):
    for c in range(cols):
        if grid[r][c] == 2:
            queue.append((r, c))
        elif grid[r][c] == 1:
            fresh += 1

# If there are no fresh oranges, return 0 minutes immediately
if fresh == 0:
    return 0

minutes = 0
directions = [(0, 1), (0, -1), (1, 0), (-1, 0)]

# Step 2: Multi-source BFS propagation
while queue:
    rotted_any = False

    # Process all currently rotten oranges for this minute level
    for _ in range(len(queue)):
        r, c = queue.popleft()

        for dr, dc in directions:
            nr, nc = r + dr, c + dc

            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2  # Rot the orange in-place
                fresh -= 1
                queue.append((nr, nc))
                rotted_any = True

    if rotted_any:
        minutes += 1

# Return minutes if all fresh oranges rotted, else -1 (isolated components)
return minutes if fresh == 0 else -1
```

40.4 Python-Specific Complexities & Implementation Considerations

40.4.1 1. BFS Level-by-Level Incrementing Gotcha

- A common mistake is incrementing `minutes` unconditionally inside the `while queue:` loop. If the final level does not rot any new oranges, it should not contribute to the minute count.
- By tracking whether at least one orange rotted using a boolean flag (`rotted_any`), we avoid off-by-one errors (e.g. returning `5` instead of `4` because we incremented for the final queue clear).

40.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(rows \times cols)$	We scan the grid initially, and each grid cell is queued/visited at most once.
Space Complexity	$\mathcal{O}(rows \times cols)$	In the worst-case, the queue holds $\mathcal{O}(rows \times cols)$ coordinates (e.g., if the entire grid starts as rotten).

40.6 Common Follow-Up Questions & How to Answer

40.6.1 Q1: What is the difference between running Multi-Source BFS vs. multiple Single-Source BFS traversals?

- **Multi-Source BFS**: We insert all source nodes into the queue at level 0. The wave spreads outwards simultaneously. The time complexity is $\mathcal{O}(R \times C)$.
- **Multiple Single-Source BFS**: Running BFS from each rotten orange one-by-one to calculate the shortest path to every fresh orange, and then calculating the maximum of the minimums. This would take $\mathcal{O}(K \times R \times C)$ time (where K is the number of initial rotten oranges), which is highly inefficient for dense grids.

40.6.2 Q2: What if we are not allowed to modify the input grid?

- **The Solution**: If input modification is forbidden (e.g., for concurrency safety), maintain a separate `visited` set containing coordinates of newly rotten oranges, or copy the grid. However, copying the grid or keeping a `visited` set changes auxiliary space from $\mathcal{O}(1)$ to $\mathcal{O}(R \times C)$.

40.6.3 Q3: How do we solve this if the grid size is massive (e.g. $10^9 \times 10^9$) but the number of oranges is very small (sparse grid)?

- **The Solution**: Storing a $10^9 \times 10^9$ array is impossible. Instead, represent the grid as a hash map of coordinates to state: `hash_map: Coordinate -> State`.

- We only store the coordinates of actual oranges. The BFS will query neighbors using hash map lookups.
- This drops the space and time complexity to $\mathcal{O}(O)$ (where O is the total number of oranges), making it independent of grid size.

40.7 Pro-Tip: How to Impress the Interviewer

- **Explain Multi-Source Initialization**: Emphasize that the key to multi-source BFS is that **all** source nodes must be in the queue at the start (minute 0). Adding them sequentially or processing them in separate BFS executions changes the correctness of the traversal.
- **Mention Connectivity & Disjoint Sets**: Note that if the final fresh orange count is greater than 0, it indicates the graph has multiple disconnected components (islands), and at least one component containing fresh oranges had no initial source (rotten orange).